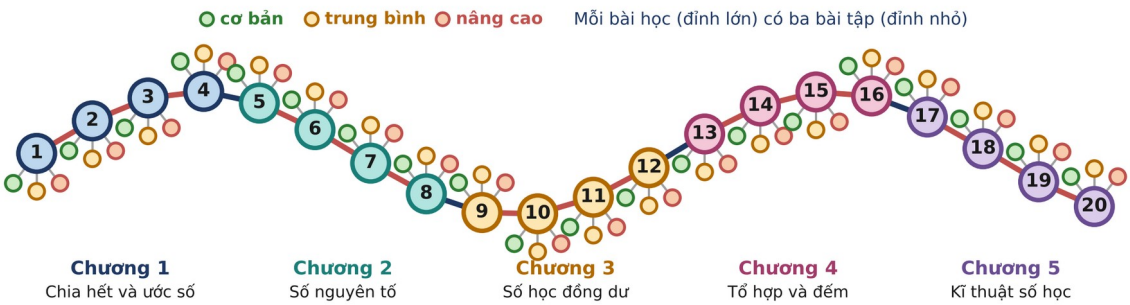
Tổng hợp và biên soạn: Lê Thành Nam

TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

SỐ HỌC

Từ cơ bản đến nâng cao

TẬP 2 · BÀI TẬP



Chương 1. CHIA HẾT VÀ ƯỚC SỐ
Chương 2. SỐ NGUYÊN TỐ
Chương 3. SỐ HỌC ĐỒNG DƯ
Chương 4. TỔ HỢP VÀ ĐẾM
Chương 5. KĨ THUẬT SỐ HỌC TRONG LẬP TRÌNH

60 bài tập · ràng buộc theo subtask · ví dụ có giải thích và hình minh họa

LỜI NÓI ĐẦU

Học số học mà không làm bài tập cũng giống như học bơi trên cạn. Ở Tập 1, các em đã được dẫn từng bước từ ý tưởng ngây thơ tới thuật toán tốt, đã chạy tay trên những ví dụ nhỏ và hiểu **vì sao** mỗi công thức đúng. Tập 2 này là "bể bơi": nơi các em tự mình gặp đề bài mới, tự nhận ra số học ẩn sau câu chuyện, tự chọn công cụ và tự xử lí những chi tiết chỉ lộ ra khi bắt tay vào code: tràn số, số dư âm, chia trong đồng dư, trường hợp n = 1...

Bộ tài liệu gồm ba tập đi cùng nhau:

- **Tập 1: Lý thuyết** gồm 20 bài học trong 5 chương.
- **Tập 2: Bài tập** (cuốn sách này) gồm **60 bài**: mỗi bài học của Tập 1 có đúng ba bài tập ở ba mức độ **cơ bản**, **trung bình** và **nâng cao**. Tên và ý tưởng các bài trùng với mục "Luyện tập" ở cuối mỗi bài học của Tập 1.
- **Tập 3: Hướng dẫn giải** phân tích từng bài từ cách ngây thơ đến cách tối ưu, kèm chương trình hoàn chỉnh.

Mỗi bài tập được trình bày như một đề thi thật: có tên tệp, giới hạn thời gian và bộ nhớ, ràng buộc chia theo **subtask** để em biết với mỗi thuật toán mình nghĩ ra sẽ được bao nhiêu điểm. Mọi ví dụ đều có **giải thích** và phần lớn có **hình minh họa** vẽ cùng bảng màu với Tập 1, để em hình dung đề bài trước khi bắt tay vào code. Cuối mỗi bài là khung **CHÚ Ý** chỉ ra những bẫy thường gặp và khung **HƯỚNG SUY NGHĨ** gợi mở hướng đi mà không "lộ" lời giải.

Để đảm bảo độ tin cậy, mỗi bài tập đều có một chương trình giải mẫu; mọi kết quả trong các ví dụ được sinh ra bằng cách **chạy chính chương trình đó**, và chương trình giải mẫu lại được đối chiếu với một chương trình "vét cạn" (hoặc phép tính số lớn chính xác) trên hàng trăm bộ dữ liệu ngẫu nhiên, cũng như được chạy thử trên dữ liệu lớn nhất để kiểm tra thời gian.

Lời khuyên quan trọng nhất: **hãy tự làm trước khi đọc gợi ý**. Một bài tự nghĩ ra sau hai giờ có giá trị hơn mười bài đọc lời giải. Khi bế tắc thật sự, hãy đọc lại phần lí thuyết tương ứng ở Tập 1, rồi mới đọc khung "Hướng suy nghĩ", và chỉ mở Tập 3 khi đã thử hết sức.

Chúc các em luyện tập bền bỉ và đạt kết quả cao!

Lê Thành Nam

CÁCH SỬ DỤNG TẬP BÀI TẬP

Mỗi bài tập trong sách đều được trình bày theo cùng một cấu trúc, để em quen mắt và tìm thông tin thật nhanh:

Thành phần	Nội dung
Thanh tiêu đề	Mã bài (số bài học . thứ tự), tên bài và mức độ khó.
Dải thông tin	Tên tệp chương trình, tệp dữ liệu vào, tệp kết quả; giới hạn thời gian và bộ nhớ.
Kiến thức	Kiến thức cần dùng và bài học tương ứng ở Tập 1.
Đề bài	Câu chuyện và yêu cầu của bài toán.
Dữ liệu vào / Kết quả	Khuôn dạng chính xác của tệp vào và tệp ra.
Ràng buộc	Giới hạn chung và bảng **subtask**: mỗi subtask có thêm ràng buộc riêng và chiếm một tỉ lệ điểm.
Ví dụ	Một hoặc vài bộ dữ liệu mẫu, kèm cột giải thích và hình minh họa.
CHÚ Ý	Những bẫy làm mất điểm: tràn số, tràn ngăn xếp, trường hợp đặc biệt...
HƯỚNG SUY NGHĨ	Vài câu gợi mở để đọc khi đã thật sự bế tắc.

Ba mức độ của bài tập



Mức độ	Ý nghĩa
★☆☆ CƠ BẢN	Áp dụng gần như trực tiếp thuật toán của bài học; trọng tâm là cài đặt đúng, đủ nhanh, xử lí cẩn thận dữ liệu vào ra.
★★☆ TRUNG BÌNH	Cần thêm một quan sát, một bước biến đổi hoặc một chi tiết cài đặt quan trọng trước khi dùng được thuật toán.
★★★ NÂNG CAO	Kết hợp nhiều kĩ thuật, mô hình hóa khó, hoặc phải dựng lại đáp án; tương đương bài khó của kì thi học sinh giỏi cấp tỉnh, cấp quốc gia.

Subtask và chiến lược lấy điểm

Bảng subtask cho biết với ràng buộc nhỏ hơn em được bao nhiêu phần trăm số điểm. Chẳng hạn ở bài 2.2, chương trình thử mọi số từ 1 tới n (O(n)) đã được một phần số điểm, dù n tới 10^{12} đòi hỏi O(√n). Trong phòng thi, hãy luôn:

1) Viết lời giải ngây thơ cho subtask nhỏ nhất trước: vừa lấy điểm chắc chắn, vừa có công cụ để kiểm tra lời giải tốt.
2) Đọc kĩ các subtask ở giữa: chúng thường gợi ý từng bước tiến tới lời giải đầy đủ.
3) Khi làm xong lời giải tốt, đối chiếu với lời giải ngây thơ trên nhiều bộ dữ liệu nhỏ (xem Phụ lục B).

Quy ước chung

- Các số trên cùng một dòng cách nhau bởi dấu cách. "Mô-đun $10^9 + 7$" nghĩa là in số dư (không âm) khi chia cho 1 000 000 007.
- Nếu không ghi khác, giới hạn thời gian là 1 giây và bộ nhớ là 256 MB cho mỗi test.
- Với bài có **nhiều đáp án đúng**, đề bài ghi rõ "in cách bất kì"; bài làm được chấm bằng chương trình kiểm tra (checker).
- Chương trình đọc và ghi theo kiểu "có tệp thì đọc tệp" như ở Tập 1, để dùng được cả với phần mềm chấm Themis lẫn các trang chấm trực tuyến:

NGAYTHU.CPP: khung chương trình dùng cho mọi bài trong sách

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    // Có tệp thì đọc tệp, không có thì đọc từ bàn phím
    if (fopen("NGAYTHU.INP", "r")) {
        freopen("NGAYTHU.INP", "r", stdin);
        freopen("NGAYTHU.OUT", "w", stdout);
    }
    long long n;
    scanf("%lld", &n);
    // ... phần giải bài toán ...
    return 0;
}
```

Quy ước màu trong hình vẽ

Hình minh họa dùng chung bảng màu với Tập 1:

Màu	Ý nghĩa
Ô **vàng** viền đỏ	Giá trị cần chú ý: đáp số, số đang xét, phần tử thuộc đáp án.
Ô **xanh dương**	Giá trị đã tính xong, hoặc phần tử "bình thường".
Ô **cam nhạt**	Dữ liệu vào: chữ số, số hạng, phần tử của dãy.
Ô **xám**	Giá trị bị loại (hợp số đã gạch...) hoặc khởi tạo.
Chữ **đỏ**	Kết luận, đáp số, chỗ dễ sai.
Đỉnh, cạnh (nếu có)	Giống Tập 1 của bộ Lý thuyết đồ thị.

LỘ TRÌNH LUYỆN TẬP GỢI Ý

- **Vòng 1:** làm 20 bài ★☆☆ theo thứ tự các bài học, ngay sau khi học xong bài lí thuyết tương ứng.
- **Vòng 2:** làm 20 bài ★★☆; mỗi bài cố gắng tự tìm ra "bước biến đổi" trước khi đọc Hướng suy nghĩ.
- **Vòng 3:** làm 20 bài ★★★ như một kì thi thử: bấm giờ, tự lấy điểm từng subtask.
- Ghi lại tiến độ vào bảng ở Phụ lục C để biết mình còn yếu ở chương nào.

MỤC LỤC

LỜI NÓI ĐẦU **2**

CÁCH SỬ DỤNG TẬP BÀI TẬP **3**

BẢNG TỔNG HỢP 60 BÀI TẬP **6**

CHƯƠNG 1. CHIA HẾT VÀ ƯỚC SỐ **11**

Bài 1. Phép chia có dư và đồng dư (bài tập 1.1, 1.2, 1.3) 12
Bài 2. Ước số: liệt kê, đếm và tính tổng (bài tập 2.1, 2.2, 2.3) 19
Bài 3. Ước chung lớn nhất và bội chung nhỏ nhất (bài tập 3.1, 3.2, 3.3) 26
Bài 4. Euclid mở rộng và phương trình ax + by = c (bài tập 4.1, 4.2, 4.3) 33

CHƯƠNG 2. SỐ NGUYÊN TỐ **40**

Bài 5. Số nguyên tố và kiểm tra nguyên tố (bài tập 5.1, 5.2, 5.3) 41
Bài 6. Sàng Eratosthenes (bài tập 6.1, 6.2, 6.3) 48
Bài 7. Phân tích thừa số nguyên tố (bài tập 7.1, 7.2, 7.3) 55
Bài 8. Sàng trên đoạn (bài tập 8.1, 8.2, 8.3) 62

CHƯƠNG 3. SỐ HỌC ĐỒNG DƯ **69**

Bài 9. Lũy thừa nhanh (bài tập 9.1, 9.2, 9.3) 70
Bài 10. Nghịch đảo mô-đun (bài tập 10.1, 10.2, 10.3) 77
Bài 11. Hàm phi Euler và định lí Euler (bài tập 11.1, 11.2, 11.3) 84
Bài 12. Định lí số dư Trung Hoa (bài tập 12.1, 12.2, 12.3) 91

CHƯƠNG 4. TỔ HỢP VÀ ĐẾM **99**

Bài 13. Tổ hợp và hệ số nhị thức (bài tập 13.1, 13.2, 13.3) 100
Bài 14. Nguyên lí bù trừ (bài tập 14.1, 14.2, 14.3) 107
Bài 15. Dãy truy hồi và lũy thừa ma trận (bài tập 15.1, 15.2, 15.3) 114
Bài 16. Kĩ thuật chia khối ⌊n / i⌋ (bài tập 16.1, 16.2, 16.3) 121

CHƯƠNG 5. KĨ THUẬT SỐ HỌC TRONG LẬP TRÌNH **127**

Bài 17. Số nguyên lớn (bài tập 17.1, 17.2, 17.3) 128
Bài 18. Hệ cơ số và biểu diễn nhị phân (bài tập 18.1, 18.2, 18.3) 135
Bài 19. Tổng tiền tố và số dư (bài tập 19.1, 19.2, 19.3) 142
Bài 20. Điểm nguyên và hình học số nguyên (bài tập 20.1, 20.2, 20.3) 149

PHỤ LỤC A. KIỂM TRA TRƯỚC KHI NỘP BÀI **156**

PHỤ LỤC B. TỰ KIỂM TRA BẰNG CÁCH ĐỐI CHIẾU **158**

PHỤ LỤC C. BẢNG THEO DÕI TIẾN ĐỘ **160**

BẢNG TỔNG HỢP 60 BÀI TẬP

Bảng dưới đây liệt kê toàn bộ các bài tập của sách, kèm tên tệp, mức độ, kiến thức chính và số trang, để em tiện chọn bài theo chủ đề hoặc theo mức độ.

Chương 1. CHIA HẾT VÀ ƯỚC SỐ

Bài	Tên bài	Tệp	Mức độ	Kĩ năng rèn luyện	Trang
1.1	**NGÀY TRONG TUẦN**	`NGAYTHU`	★☆☆ CƠ BẢN	Lấy số dư không âm; tránh vòng lặp đếm từng ngày.	12
1.2	**CHỮ SỐ CUỐI**	`CHUSOCUOI`	★★☆ TRUNG BÌNH	Tìm chu kì của dãy số dư; xử lí số cho dưới dạng xâu.	15
1.3	**VIẾT LIỀN**	`VIETLIEN`	★★★ NÂNG CAO	Mở rộng sơ đồ Horner cho những "chữ số" có độ dài thay đổi; tiết kiệm bộ nhớ.	17
2.1	**XẾP HÀNG**	`XEPHANG`	★☆☆ CƠ BẢN	Duyệt ước tới √n; xử lí trường hợp suy biến n = 1 và số nguyên tố.	19
2.2	**ƯỚC TĂNG DẦN**	`UOCTANG`	★★☆ TRUNG BÌNH	Ghép hai dãy ước nhỏ, ước lớn thành dãy tăng; trả lời nhanh nhiều truy vấn trên dãy đã sắp.	22
2.3	**SỐ BẠN BÈ**	`BANBE`	★★★ NÂNG CAO	Đổi chiều câu hỏi "k có những ước nào" thành "d là ước của những số nào"; tiết kiệm bộ nhớ.	24
3.1	**CẮT GẠCH**	`CATGACH`	★☆☆ CƠ BẢN	Dùng gcd cho số tới 10^{18}; nhận ra đáp số vượt `long long` và in số `__int128`.	26
3.2	**BỎ MỘT SỐ**	`BOMOTSO`	★★☆ TRUNG BÌNH	Tính trước tiền tố, hậu tố để "bỏ một phần tử" trong O(1).	29
3.3	**ĐÈN NHÁY**	`DENNHAY`	★★★ NÂNG CAO	Khai thác "chỉ có ít giá trị khác nhau" để trả lời truy vấn trên đoạn; BCNN không tràn số.	31
4.1	**ĐONG NƯỚC**	`DONGNUOC`	★☆☆ CƠ BẢN	Áp dụng Bézout cho nhiều số; trả lời truy vấn trong O(1).	33
4.2	**MUA HÀNG**	`MUAHANG`	★★☆ TRUNG BÌNH	Tìm nghiệm có x nhỏ nhất không âm rồi đếm số nghiệm trong một đoạn; tránh tràn số.	36

Bài	Tên bài	Tệp	Mức độ	Kĩ năng rèn luyện	Trang
4.3	**ĐỒNG HỒ HỎNG**	`DONGHO`	★★★ NÂNG CAO	Đưa bài toán về phương trình đồng dư; xử lí trường hợp suy biến và tràn số với mô-đun tới 10^{18}.	38

Chương 2. SỐ NGUYÊN TỐ

Bài	Tên bài	Tệp	Mức độ	Kĩ năng rèn luyện	Trang
5.1	**NGUYÊN TỐ GẦN NHẤT**	`NTGANNHAT`	★☆☆ CƠ BẢN	Cài đặt đúng và nhanh phép kiểm tra nguyên tố; duyệt "loang" ra hai phía.	41
5.2	**TỔNG HAI SỐ NGUYÊN TỐ**	`GOLDBACH`	★★☆ TRUNG BÌNH	Dùng tính chẵn lẻ để thu hẹp tìm kiếm; sắp xếp các phép kiểm tra rẻ trước, đắt sau.	44
5.3	**NGUYÊN TỐ 64 BIT**	`NT64`	★★★ NÂNG CAO	Cài đặt Miller–Rabin không tràn số cho mọi số không dấu 64 bit.	46
6.1	**ĐẾM SỐ NGUYÊN TỐ**	`DEMNGUYENTO`	★☆☆ CƠ BẢN	Sàng một lần, trả lời hai loại truy vấn bằng hai mảng.	48
6.2	**SỐ NGUYÊN TỐ SINH ĐÔI**	`SINHDOI`	★★☆ TRUNG BÌNH	Thiết kế mảng tiền tố cho một "cặp" thay vì một số; xử lí biên của đoạn.	51
6.3	**SỐ CÓ ĐÚNG BA ƯỚC**	`BAUOC`	★★★ NÂNG CAO	Nhận dạng số có đúng ba ước; chỉ cần thử chia tới căn bậc ba thay vì căn bậc hai.	53
7.1	**SỐ ƯỚC**	`SOUOC`	★☆☆ CƠ BẢN	Dùng sàng spf cho rất nhiều truy vấn; tính đồng thời số ước và tổng ước.	55
7.2	**CHỮ SỐ 0 CỦA GIAI THỪA**	`CHUSO0`	★★☆ TRUNG BÌNH	Đưa bài toán "chữ số 0 tận cùng" trong cơ số bất kì về số mũ của từng ước nguyên tố.	58
7.3	**BCNN CỦA DÃY**	`BCNNDAY`	★★★ NÂNG CAO	Làm việc với BCNN khổng lồ qua bảng số mũ; đếm phần tử "không thể thay thế".	60
8.1	**NGUYÊN TỐ TRÊN ĐOẠN**	`NTDOAN`	★☆☆ CƠ BẢN	Sàng nhỏ một lần, dùng lại cho nhiều cửa sổ; tính tổng không tràn số.	62
8.2	**KHOẢNG CÁCH LỚN NHẤT**	`KHOANGCACH`	★★☆ TRUNG BÌNH	Duyệt các số nguyên tố liên tiếp trong cửa sổ; xử lí hòa và trường hợp suy biến.	65

Bài	Tên bài	Tệp	Mức độ	Kĩ năng rèn luyện	Trang
8.3	**SỐ ƯỚC TRÊN ĐOẠN**	`UOCDOAN`	★★★ NÂNG CAO	Phân tích cùng lúc mọi số trong cửa sổ; nhận ra phần còn lại là số nguyên tố.	67

Chương 3. SỐ HỌC ĐỒNG DƯ

Bài	Tên bài	Tệp	Mức độ	Kĩ năng rèn luyện	Trang
9.1	**LŨY THỪA**	`LUYTHUAQ`	★☆☆ CƠ BẢN	Cài đặt lũy thừa nhanh chuẩn; lấy dư cơ số trước khi nhân.	70
9.2	**SỐ MŨ KHỔNG LỒ**	`SOMULON`	★★☆ TRUNG BÌNH	Áp dụng sơ đồ Horner cho số mũ; không lạm dụng định lí Fermat khi mô-đun là hợp số.	73
9.3	**TỔNG LŨY THỪA**	`TONGLT`	★★★ NÂNG CAO	Chia để trị trên số mũ khi không chia được theo mô-đun; chống tràn số $10^{18} \cdot 10^{18}$.	75
10.1	**XÁC SUẤT**	`XACSUAT`	★☆☆ CƠ BẢN	Biểu diễn phân số theo mô-đun $10^9 + 7$; trừ theo mô-đun không ra số âm.	77
10.2	**NGHỊCH ĐẢO DÃY**	`NGHICHDAO`	★★☆ TRUNG BÌNH	Công thức truy hồi cho nghịch đảo; tiết kiệm thời gian và bộ nhớ với $n = 10^7$.	80
10.3	**PHƯƠNG TRÌNH ĐỒNG DƯ**	`PTDONGDU`	★★★ NÂNG CAO	Giải $a \cdot x \equiv c \pmod m$ tổng quát; đếm nghiệm trên đoạn với số âm và số cỡ 10^{18}.	82
11.1	**PHI**	`PHI`	★☆☆ CƠ BẢN	Nhận ra hàm φ trong một câu chuyện; cài đặt $O(\sqrt{n})$ không sai ước lớn còn lại.	84
11.2	**PHÂN SỐ TỐI GIẢN**	`PHANSO`	★★☆ TRUNG BÌNH	Sàng một hàm số học tới 10^7; trả lời truy vấn đoạn bằng tổng tiền tố.	87
11.3	**THÁP LŨY THỪA**	`THAPLT`	★★★ NÂNG CAO	Rút gọn số mũ với mô-đun bất kì; biết khi nào được rút gọn bằng cách chặn trên giá trị thật của tháp.	89
12.1	**ĐIỂM BINH**	`DIEMBINH`	★☆☆ CƠ BẢN	Cài đặt gộp từng cặp phương trình bằng Euclid mở rộng; tìm nghiệm nhỏ nhất không nhỏ hơn một ngưỡng.	91

Bài	Tên bài	Tệp	Mức độ	Kĩ năng rèn luyện	Trang
12.2	HAI NGỌN ĐÈN	HAIDEN	★★☆ TRUNG BÌNH	Gộp hai phương trình tổng quát; xử lí điều kiện "đã bắt đầu" ngoài đồng dư.	94
12.3	LỊCH CAN CHI	CANCHI	★★★ NÂNG CAO	Gộp nhiều phương trình khi lcm vượt xa 10^{18}; tính lcm theo mô-đun qua phân tích thừa số.	96

Chương 4. TỔ HỢP VÀ ĐẾM

Bài	Tên bài	Tệp	Mức độ	Kĩ năng rèn luyện	Trang
13.1	CHIA KẸO	CHIAKEO	★☆☆ CƠ BẢN	Đưa bài đếm về một số C(a, b); chuẩn bị bảng giai thừa đủ lớn.	100
13.2	ĐƯỜNG ĐI TRÊN LƯỚI	DUONGLUOI	★★☆ TRUNG BÌNH	Tách đường đi qua một điểm thành hai đoạn; bù trừ khi có hai điểm cấm.	103
13.3	TỔ HỢP KHỔNG LỒ	TOHOPLON	★★★ NÂNG CAO	Tính C(n, k) theo mô-đun không nguyên tố: tách theo từng ước nguyên tố rồi ghép lại.	105
14.1	BỘI SỐ	BOISO	★☆☆ CƠ BẢN	Đổi điều kiện "hoặc", "nhưng không" thành công thức bù trừ; chống tràn số.	107
14.2	NGUYÊN TỐ CÙNG NHAU	NTCN	★★☆ TRUNG BÌNH	Phân tích nhiều số tới 10^{12} đủ nhanh nhờ danh sách số nguyên tố sàng sẵn.	110
14.3	TẶNG QUÀ	TANGQUA	★★★ NÂNG CAO	Phối hợp quy tắc nhân, truy hồi D(n) và tổ hợp với n vượt mô-đun.	112
15.1	LÁT GẠCH	LATGACH	★☆☆ CƠ BẢN	Lập hệ thức truy hồi bằng cách nhìn vào phần cuối của hình.	114
15.2	TRIBONACCI	TRIBO	★★☆ TRUNG BÌNH	Ngắt dãy truy hồi tại các vị trí đặc biệt: lũy thừa ma trận theo từng đoạn.	117
15.3	ĐƯỜNG ĐI ĐỘ DÀI K	DUONGK	★★★ NÂNG CAO	Tính sẵn các lũy thừa A^1, A^2, A^4, ...; nhân vectơ với ma trận thay cho ma trận với ma trận.	119
16.1	TỔNG ƯỚC	TONGSIGMA	★☆☆ CƠ BẢN	Cộng theo khối với trọng số là cấp số cộng; chống tràn bằng `__int128`.	121
16.2	TỔNG DƯ	TONGDU	★★☆ TRUNG BÌNH	Chia khối trên đoạn [1, min(n, k)]; xử lí riêng phần i > n.	123

Bài	Tên bài	Tệp	Mức độ	Kĩ năng rèn luyện	Trang
16.3	**HAI THƯƠNG**	`HAITHUONG`	★★★ NÂNG CAO	Khai triển thành các tổng $\Sigma\ 1$, $\Sigma\ i$, $\Sigma\ i^2$ trên mỗi khối chung của hai thương.	125

Chương 5. KĨ THUẬT SỐ HỌC TRONG LẬP TRÌNH

Bài	Tên bài	Tệp	Mức độ	Kĩ năng rèn luyện	Trang
17.1	**TỔNG LỚN**	`TONGLON`	★☆☆ CƠ BẢN	Cộng, trừ số lớn; xử lí dấu, số 0 thừa và kết quả bằng 0.	128
17.2	**LŨY THỪA ĐẦY ĐỦ**	`LUYTHUALON`	★★☆ TRUNG BÌNH	Nhân số lớn với số nhỏ trong cơ số 10^9; in đúng định dạng.	131
17.3	**TÍCH LỚN**	`TICHLON`	★★★ NÂNG CAO	Nhân hai số lớn $O(L_1 \cdot L_2)$; chọn cơ số vừa nhanh vừa không tràn.	133
18.1	**ĐỔI CƠ SỐ**	`DOICOSO`	★☆☆ CƠ BẢN	Đổi qua lại giữa hai cơ số bất kì; xử lí chữ cái, số 0 và chữ số 0 thừa.	135
18.2	**BIT TRONG ĐOẠN**	`BITDOAN`	★★☆ TRUNG BÌNH	Trả lời truy vấn đoạn bằng hiệu hai "tiền tố"; xử lí đáp số vượt 2^{64}.	138
18.3	**SỐ CÓ K BIT 1**	`KBIT`	★★★ NÂNG CAO	Đếm số có tính chất về chữ số không vượt quá n; tìm số thứ m theo thứ tự.	140
19.1	**TỔNG BẰNG KHÔNG**	`TONG0`	★☆☆ CƠ BẢN	Đổi "đếm đoạn" thành "đếm cặp"; gom nhóm bằng sắp xếp.	142
19.2	**ĐOẠN DÀI NHẤT**	`DOANDAI`	★★☆ TRUNG BÌNH	Tối ưu độ dài đoạn bằng vị trí xuất hiện đầu tiên; số dư không âm; chọn đáp án theo luật phụ.	145
19.3	**HÌNH CHỮ NHẬT CHIA HẾT**	`HCNCHIAHET`	★★★ NÂNG CAO	Hạ bài toán 2 chiều xuống 1 chiều; xóa mảng đếm thông minh.	147
20.1	**TRỒNG CÂY**	`TRONGCAY`	★☆☆ CƠ BẢN	Áp dụng công thức gcd cho từng đoạn; không đếm trùng đỉnh chung.	149
20.2	**ĐA GIÁC LƯỚI**	`DAGIAC`	★★☆ TRUNG BÌNH	Kết hợp diện tích và số điểm trên biên để đếm điểm bên trong.	152
20.3	**TAM GIÁC NGUYÊN**	`TAMGIAC`	★★★ NÂNG CAO	Ước lượng mọi đại lượng trung gian; tính toán và in số `__int128`.	154

CHƯƠNG 1. CHIA HẾT VÀ ƯỚC SỐ

Mười hai bài tập đầu tiên rèn luyện những kĩ năng nền móng của số học trong lập trình: lấy số dư (kể cả với số âm và số rất lớn), duyệt ước tới √n, thuật toán Euclid và Euclid mở rộng. Các bài không khó về ý tưởng, nhưng rất nhiều "bẫy" nhỏ đang chờ: tràn số, số dư âm, đếm trùng, trường hợp n = 1... Hãy đọc kĩ khung CHÚ Ý của từng bài.

TRONG CHƯƠNG NÀY

Bài 1. Phép chia có dư và đồng dư

- 1.1 NGÀY TRONG TUẦN — ★☆☆ CƠ BẢN
- 1.2 CHỮ SỐ CUỐI — ★★☆ TRUNG BÌNH
- 1.3 VIẾT LIỀN — ★★★ NÂNG CAO

Bài 2. Ước số: liệt kê, đếm và tính tổng

- 2.1 XẾP HÀNG — ★☆☆ CƠ BẢN
- 2.2 ƯỚC TĂNG DẦN — ★★☆ TRUNG BÌNH
- 2.3 SỐ BẠN BÈ — ★★★ NÂNG CAO

Bài 3. Ước chung lớn nhất và bội chung nhỏ nhất

- 3.1 CẮT GẠCH — ★☆☆ CƠ BẢN
- 3.2 BỎ MỘT SỐ — ★★☆ TRUNG BÌNH
- 3.3 ĐÈN NHÁY — ★★★ NÂNG CAO

Bài 4. Euclid mở rộng và phương trình ax + by = c

- 4.1 ĐONG NƯỚC — ★☆☆ CƠ BẢN
- 4.2 MUA HÀNG — ★★☆ TRUNG BÌNH
- 4.3 ĐỒNG HỒ HỎNG — ★★★ NÂNG CAO

LỜI KHUYÊN CHO CHƯƠNG NÀY

- Trước khi code, hãy ghi ra giá trị lớn nhất của mọi đại lượng (tích, tổng, BCNN...) để chọn đúng kiểu dữ liệu.
- Số cho dưới dạng xâu thì xử lí theo từng chữ số; đừng cố đổi ra kiểu số.
- Với mỗi bài, tự tạo vài test "góc": n = 1, số nguyên tố, số chính phương, số âm, số 0.

BÀI 1 Phép chia có dư và đồng dư

ÔN NHANH KIẾN THỨC · Tập 1, Bài 1

- **Phép chia có dư:** $a = b \cdot q + r$ với $0 \le r < b$. Trong C++, `a % b` mang dấu của a; số dư không âm là `((a % m) + m) % m`.
- **Đồng dư:** $a \equiv b \pmod m$ khi m chia hết a − b. Đồng dư giữ nguyên qua cộng, trừ, nhân và lũy thừa: được thay một số bằng số dư của nó ở bất kì bước nào.
- **Sơ đồ Horner:** đọc số từ trái sang phải, $r \leftarrow (r \cdot 10 + c) \bmod m$ cho số dư của số rất lớn trong O(độ dài).
- Chỉ có hữu hạn số dư, nên mọi dãy số dư "tính từ số dư trước" sớm muộn đều **lặp lại tuần hoàn**.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
1.1	**NGÀY TRONG TUẦN**	★☆☆ CƠ BẢN	Lấy số dư không âm; tránh vòng lặp đếm từng ngày.
1.2	**CHỮ SỐ CUỐI**	★★☆ TRUNG BÌNH	Tìm chu kì của dãy số dư; xử lí số cho dưới dạng xâu.
1.3	**VIẾT LIỀN**	★★★ NÂNG CAO	Mở rộng sơ đồ Horner cho những "chữ số" có độ dài thay đổi; tiết kiệm bộ nhớ.

BÀI 1.1 NGÀY TRONG TUẦN ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`NGAYTHU.CPP`	`NGAYTHU.INP`	`NGAYTHU.OUT`	1 giây · 256 MB

Kiến thức: *Phép chia có dư với số âm, đồng dư theo mô-đun 7 (Tập 1, Bài 1).*

Người Việt gọi các ngày trong tuần là thứ Hai, thứ Ba, ..., thứ Bảy và Chủ nhật. Trong bài này ta viết chúng lần lượt bằng các số **2, 3, 4, 5, 6, 7** và **8** (số 8 là Chủ nhật).

Bạn An có Q câu hỏi. Câu hỏi thứ i cho biết hôm nay là thứ t và một số nguyên k: nếu $k > 0$, An muốn biết **k ngày nữa** là thứ mấy; nếu $k < 0$, An muốn biết **|k| ngày trước** là thứ mấy; nếu $k = 0$ thì đó chính là hôm nay.

Hãy trả lời mọi câu hỏi của An.

Dữ liệu vào

Dòng đầu chứa số nguyên Q. Mỗi dòng trong Q dòng tiếp theo chứa hai số nguyên t và k.

Kết quả

Gồm Q dòng, dòng thứ i là thứ (một số từ 2 tới 8) ứng với câu hỏi thứ i.

Ràng buộc

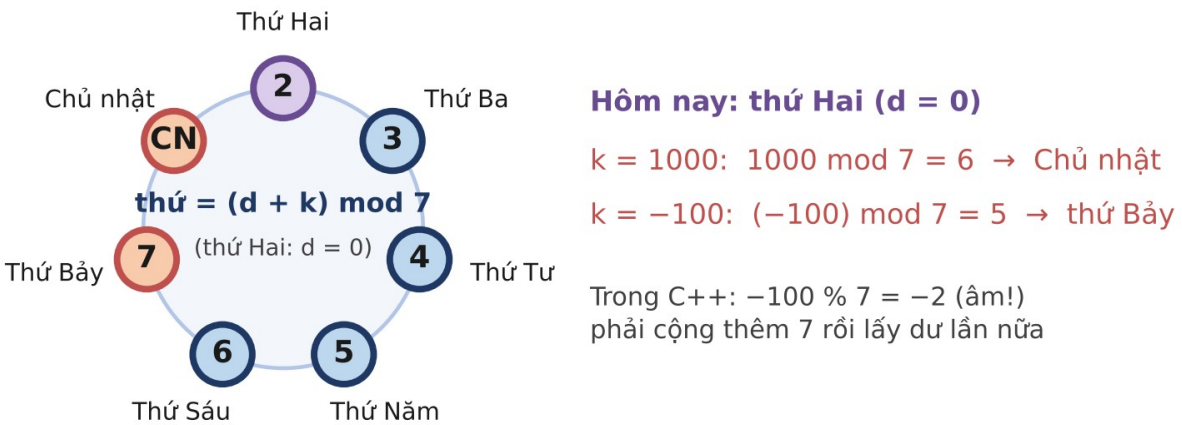
$1 \le Q \le 10^5$; $2 \le t \le 8$; $|k| \le 10^{18}$.

Subtask	Điểm	Ràng buộc thêm
1	30%	$0 \le k \le 10^6$.
2	30%	$k \ge 0$.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

NGAYTHU.INP	NGAYTHU.OUT	Giải thích
3 2 1000 2 -100 2 1000000000000000000	8 7 3	1000 = 7 · 142 + 6: sau 1000 ngày, thứ Hai tiến thêm 6 ngày là **Chủ nhật** (8). −100 = 7 · (−15) + 5: 100 ngày trước thứ Hai là **thứ Bảy** (7). 10^{18} chia 7 dư 1: **thứ Ba** (3).

NGAYTHU.INP	NGAYTHU.OUT	Giải thích
2 8 0 5 -7	8 5	k = 0: chính hôm nay, Chủ nhật. Một tuần trước thứ Năm vẫn là **thứ Năm**.



Hình 1.1. *Mỗi ngày là một "ngăn" trên vòng tròn 7 ngăn; đi k bước theo chiều kim đồng hồ (k < 0: ngược chiều).*

CHÚ Ý

Với k tới 10^{18}, vòng lặp "đếm từng ngày" chạy cả nghìn năm. Chỉ có 7 kết quả có thể xảy ra: hãy dùng phép chia có dư.

Trong C++, `-100 % 7` bằng −2. Nếu em in thẳng (t − 2 + k) % 7 + 2 thì có thể ra số 0 hoặc số âm.

Biểu thức t − 2 + k vẫn nằm trong phạm vi `long long`, nhưng nếu em cộng thêm $7 \cdot 10^{18}$ "cho chắc chắn dương" thì sẽ tràn số.

HƯỚNG SUY NGHĨ

- Đánh số lại các ngày: thứ Hai là 0, ..., Chủ nhật là 6. Khi đó câu trả lời là (d + k) mod 7, rồi đổi ngược lại.
- Số dư không âm trong C++: `((x % 7) + 7) % 7`.

BÀI 1.2	CHỮ SỐ CUỐI	★★☆ TRUNG BÌNH	
Chương trình **CHUSOCUOI.CPP**	Dữ liệu vào **CHUSOCUOI.INP**	Kết quả **CHUSOCUOI.OUT**	Giới hạn **1 giây · 256 MB**

Kiến thức: *Đồng dư theo mô-đun 10, tính tuần hoàn của lũy thừa, số dư của số rất lớn (Tập 1, Bài 1).*

Thầy giáo viết lên bảng hai số tự nhiên a và b **rất lớn**, mỗi số có tới một trăm nghìn chữ số, rồi hỏi: "Chữ số tận cùng của a^b là chữ số nào?"

Không máy tính nào tính nổi a^b (nó có thể có tới 10^{100000} chữ số!). Nhưng chỉ cần chữ số cuối, em có trả lời được không? Quy ước $0^0 = 1$.

Dữ liệu vào

Dòng đầu chứa số nguyên T là số câu hỏi. Mỗi dòng trong T dòng tiếp theo chứa hai số tự nhiên a và b, viết trong hệ thập phân, không có chữ số 0 thừa ở đầu.

Kết quả

Gồm T dòng, mỗi dòng một chữ số là chữ số tận cùng của a^b.

Ràng buộc

$1 \le T \le 10$; a và b có không quá 10^5 chữ số.

Subtask	Điểm	Ràng buộc thêm
1	20%	a, b ≤ 10^6.
2	30%	a, b ≤ 10^{18}.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

CHUSOCUOI.INP	CHUSOCUOI.OUT	Giải thích
3 2 2026 7 3 0 0	4 3 1	2^{2026}: dãy chữ số cuối của 2^b là 2, 4, 8, 6 lặp lại; 2026 chia 4 dư 2 nên chữ số cuối giống $2^2 = 4$. $7^3 = 343$. Quy ước $0^0 = 1$.

CHUSOCUOI.INP	CHUSOCUOI.OUT	Giải thích
1 123456789123456789 123456789 987654321098765432 10	1	Chữ số cuối của a là 9; dãy 9, 1, 9, 1, ... có chu kì 2. Số mũ chẵn nên đáp số là **1**.



Hình 1.2. *Chữ số tận cùng của a^b chỉ phụ thuộc vào chữ số cuối của a và lặp lại tuần hoàn theo b; ô vàng là một chu kì.*

CHÚ Ý

a và b phải đọc vào **xâu**: 10^5 chữ số vượt xa mọi kiểu số nguyên.

Cẩn thận khi b chia hết cho 4: số dư là 0 nhưng chữ số cuối ứng với số mũ 4 (chẳng hạn $2^4 = 16$ tận cùng 6, không phải $2^0 = 1$). Riêng b = 0 thì đáp số là 1.

Đừng quên trường hợp a = 0 với b > 0 (đáp số 0).

HƯỚNG SUY NGHĨ

- Chữ số tận cùng của a^b là $a^b \bmod 10$, và theo tính chất đồng dư, chỉ phụ thuộc vào a mod 10.
- Nhìn bảng ở hình: mọi chu kì đều là ước của 4. Vậy chỉ cần biết b mod 4, mà số dư của một số rất lớn thì tính được bằng sơ đồ Horner (hoặc chỉ cần hai chữ số cuối, vì 100 chia hết cho 4).

BÀI 1.3	VIẾT LIỀN	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
VIETLIEN.CPP	VIETLIEN.INP	VIETLIEN.OUT	1 giây · 64 MB

Kiến thức: *Sơ đồ Horner theo từng khối chữ số, lũy thừa của 10 theo mô-đun (Tập 1, Bài 1).*

Bé Na tập viết số. Bé viết liền nhau các số tự nhiên từ 1 tới n, không có dấu cách, được một số rất dài. Chẳng hạn với n = 12, bé viết được số **123456789101112**.

Hãy cho biết số mà bé Na viết được chia cho m dư bao nhiêu.

Dữ liệu vào

Một dòng chứa hai số nguyên n và m.

Kết quả

Một số nguyên là số dư cần tìm.

Ràng buộc

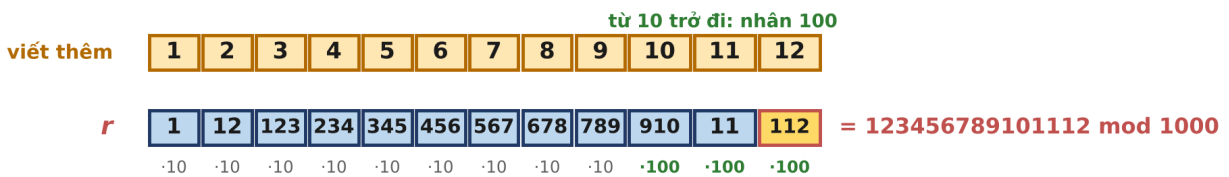
$1 \le n \le 10^7$; $1 \le m \le 10^9$.

Subtask	Điểm	Ràng buộc thêm
1	30%	$n \le 9$.
2	30%	$n \le 1000$.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

VIETLIEN.INP	VIETLIEN.OUT	Giải thích
12 1000	112	123456789101112 = 1000 · 123456789101 + 112.

VIETLIEN.INP	VIETLIEN.OUT	Giải thích
15 7	2	Số 123456789101112131415 chia 7 dư 2.



Hình 1.3. *Viết thêm số i vào bên phải giống như "dịch trái" phần đã viết đi số chữ số của i rồi cộng i.*

CHÚ Ý

Với $n = 10^7$, số bé Na viết có 68 888 897 chữ số. Lưu cả xâu ấy đã tốn khoảng 69 MB, vượt giới hạn bộ nhớ 64 MB của bài.

$r \cdot 10^k$ với $r < m \le 10^9$ và $10^k \bmod m < 10^9$ có thể tới 10^{18}: vẫn vừa `long long`, nhưng không vừa `int`.

Đừng tính lũy thừa của 10 bằng `pow`: gọi k là số chữ số của i, hãy duy trì $10^k \bmod m$ và chỉ cập nhật khi i tăng thêm một chữ số (i = 10, 100, 1000, ...).

HƯỚNG SUY NGHĨ

- Ở bài SODU (Tập 1, Bài 1), mỗi bước thêm **một** chữ số: $r \leftarrow r \cdot 10 + c$. Ở đây mỗi bước thêm cả **một số** i có k chữ số: $r \leftarrow r \cdot 10^k + i$.
- Từ 1 tới 9 ta nhân 10, từ 10 tới 99 nhân 100, ... Số chữ số chỉ thay đổi 7 lần với $n \le 10^7$.

BÀI 2 Ước số: liệt kê, đếm và tính tổng

ÔN NHANH KIẾN THỨC · Tập 1, Bài 2

- **Cặp ước:** nếu d là ước của n thì n / d cũng là ước, và trong hai số ấy có một số ≤ √n. Duyệt `for (d = 1; d * d <= n; d++)`, mỗi ước d ghi nhận cả n / d: O(√n), chỉ 10^6 bước với n = 10^{12}.
- **Số chính phương:** ước √n tự bắt cặp với chính nó, chỉ được đếm một lần (n có số ước lẻ ⇔ n là số chính phương).
- **Liệt kê tăng dần không cần sắp xếp:** các ước nhỏ d xuất hiện theo thứ tự tăng, các ước lớn n / d theo thứ tự giảm; in mảng thứ nhất rồi mảng thứ hai theo chiều ngược lại.
- **Sàng điều hòa:** với mỗi d, cộng d vào d, 2d, 3d, ...: có σ(k) cho mọi k ≤ N trong O(N log N). Một số ≤ 10^{12} có không quá 6720 ước.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
2.1	**XẾP HÀNG**	★☆☆ CƠ BẢN	Duyệt ước tới √n; xử lí trường hợp suy biến n = 1 và số nguyên tố.
2.2	**ƯỚC TĂNG DẦN**	★★☆ TRUNG BÌNH	Ghép hai dãy ước nhỏ, ước lớn thành dãy tăng; trả lời nhanh nhiều truy vấn trên dãy đã sắp.
2.3	**SỐ BẠN BÈ**	★★★ NÂNG CAO	Đổi chiều câu hỏi "k có những ước nào" thành "d là ước của những số nào"; tiết kiệm bộ nhớ.

BÀI 2.1 XẾP HÀNG ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`XEPHANG.CPP`	`XEPHANG.INP`	`XEPHANG.OUT`	1 giây · 256 MB

Kiến thức: *Đếm ước bằng cặp ước (d, n / d), duyệt tới √n (Tập 1, Bài 2).*

Thầy Tổng phụ trách chuẩn bị cho lễ khai giảng. Có n học sinh cần được xếp thành một **khối hình chữ nhật**: r hàng, mỗi hàng đúng c bạn, với r · c = n và không bạn nào đứng lẻ.

Để khối trông thật sự là "khối", thầy yêu cầu **r ≥ 2 và c ≥ 2** (không xếp thành một hàng ngang hay một hàng dọc). Hai cách xếp được coi là khác nhau nếu số hàng khác nhau; chẳng hạn với n = 12, xếp 3 hàng × 4 bạn và xếp 4 hàng × 3 bạn là hai cách khác nhau.

Thầy muốn biết có bao nhiêu cách xếp, và trong các cách ấy, cách nào **cân đối nhất**, tức là có hiệu c − r nhỏ nhất với r ≤ c.

Thầy cần chuẩn bị cho T khối lớp khác nhau. Hãy trả lời giúp thầy.

Dữ liệu vào

Dòng đầu chứa số nguyên T. Mỗi dòng trong T dòng tiếp theo chứa một số nguyên n.

Kết quả

Gồm T dòng. Với mỗi n: nếu không có cách xếp nào, in số 0; ngược lại in ba số nguyên k, r, c, trong đó k là số cách xếp, còn r hàng × c bạn (r ≤ c) là cách xếp cân đối nhất.

Ràng buộc

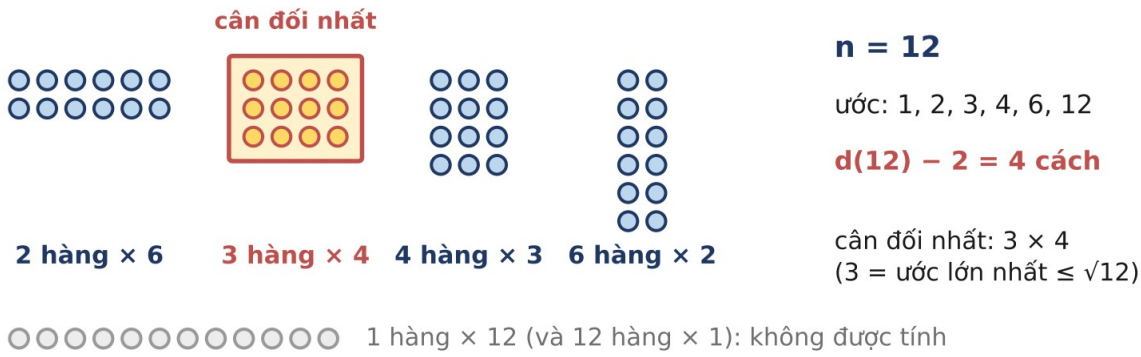
$1 \le T \le 10$; $1 \le n \le 10^{12}$.

Subtask	Điểm	Ràng buộc thêm
1	30%	$n \le 1000$.
2	30%	$n \le 10^7$.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

XEPHANG.INP	XEPHANG.OUT	Giải thích
3 12 36 13	4 3 4 7 6 6 0	12 có 6 ước 1, 2, 3, 4, 6, 12; bỏ hai cách 1 × 12 và 12 × 1 còn **4** cách (Hình 2.1). Cân đối nhất là 3 hàng × 4 bạn. 36 có 9 ước nên có 9 − 2 = 7 cách; cân đối nhất là 6 × 6. 13 là số nguyên tố: chỉ xếp được 1 × 13 hoặc 13 × 1.

XEPHANG.INP	XEPHANG.OUT	Giải thích
4 1 4 999999999989 1000000000000	0 1 2 2 0 167 1000000 1000000	n = 1: chỉ có cách 1 × 1, không hợp lệ. n = 4: duy nhất 2 × 2. 999 999 999 989 là số nguyên tố. 10^{12} có 169 ước, nên có 167 cách; cân đối nhất là $10^6 \times 10^6$.



Hình 2.1. *Bốn cách xếp 12 học sinh thành khối hình chữ nhật; ước lớn nhất không vượt quá √12 cho cách xếp cân đối nhất.*

CHÚ Ý

Công thức "số ước trừ 2" cho kết quả **−1** khi n = 1 (số 1 chỉ có một ước). Hãy xử lí riêng.

Với $n = 10^{12}$, vòng lặp thử mọi d từ 1 tới n chạy cả giờ đồng hồ; chỉ được duyệt tới $\sqrt{n} = 10^6$. Viết điều kiện `d * d <= n` với d kiểu `long long`, đừng dùng `sqrt` số thực.

Số chính phương: ước √n chỉ đếm một lần (với n = 36, cách xếp 6 × 6 chỉ là **một** cách).

HƯỚNG SUY NGHĨ

- Mỗi cách xếp ứng với một ước r của n với $2 \le r \le n / 2$. Có bao nhiêu ước bị loại?
- Cách xếp cân đối nhất có r là ước lớn nhất của n không vượt quá √n. Khi duyệt d tăng dần tới √n, ước cuối cùng gặp được chính là r.

BÀI 2.2 ƯỚC TĂNG DẦN

★★☆ TRUNG BÌNH

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
UOCTANG.CPP	UOCTANG.INP	UOCTANG.OUT	1 giây · 256 MB

Kiến thức: *Liệt kê ước theo thứ tự tăng dần không cần sắp xếp (Tập 1, Bài 2, Mở rộng 1); tìm kiếm nhị phân.*

Cửa hàng bánh có n chiếc bánh quy giống nhau và muốn đóng tất cả vào các hộp, mỗi hộp **cùng số bánh**, không thừa chiếc nào. Như vậy số bánh mỗi hộp phải là một ước của n.

Trước hết, cửa hàng muốn có bảng liệt kê mọi lựa chọn: tất cả các ước của n theo thứ tự tăng dần.

Sau đó có Q khách hàng. Khách thứ i muốn mỗi hộp có **ít nhất x_i** chiếc bánh, nhưng hộp càng nhỏ càng tốt. Với mỗi khách, hãy cho biết số bánh mỗi hộp: đó là **ước nhỏ nhất của n không nhỏ hơn x_i**, hoặc −1 nếu không có ước nào như vậy.

Dữ liệu vào

Dòng đầu chứa hai số nguyên n và Q. Dòng thứ hai chứa Q số nguyên $x_1, x_2, ..., x_Q$.

Kết quả

Dòng 1: số ước của n. Dòng 2: các ước của n theo thứ tự tăng dần, cách nhau một dấu cách. Tiếp theo là Q dòng, dòng thứ i là câu trả lời cho khách thứ i.

Ràng buộc

$1 \le n \le 10^{12}$; $1 \le Q \le 10^5$; $1 \le x_i \le 10^{12}$.

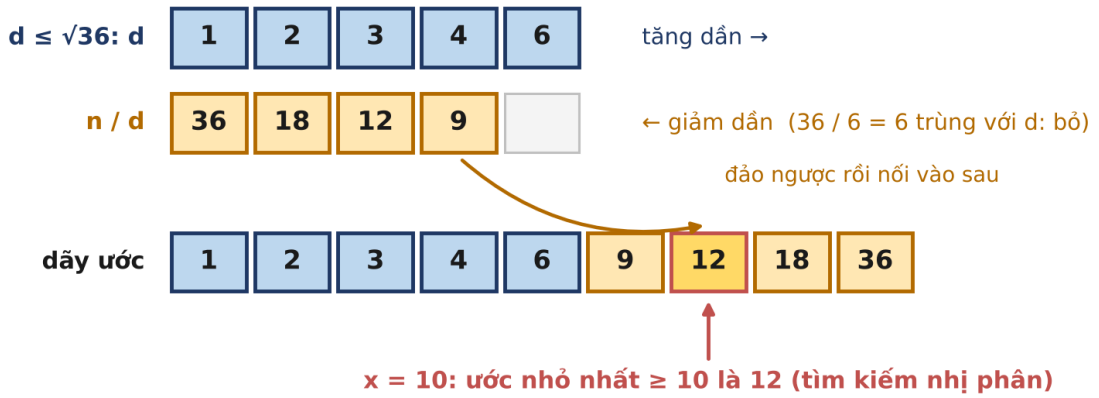
Subtask	Điểm	Ràng buộc thêm
1	20%	$n \le 10^6$, $Q \le 100$.
2	30%	$Q \le 100$.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

UOCTANG.INP	UOCTANG.OUT	Giải thích
36 4 5 10 36 40	9 1 2 3 4 6 9 12 18 36 6 12 36 -1	36 có 9 ước. x = 5: ước nhỏ nhất ≥ 5 là 6; x = 10: là 12 (Hình 2.2); x = 36: chính là 36; x = 40 lớn hơn n nên in −1.

UOCTANG.INP	UOCTANG.OUT	Giải thích
999999999989 2 2 1000000000000	2 1 999999999989 999999999989 -1	999 999 999 989 là số nguyên tố, chỉ có hai ước. x = 10^{12} lớn hơn n.

UOCTANG.INP	UOCTANG.OUT	Giải thích
1 2 1 2	1 1 1 -1	n = 1 chỉ có một ước là 1.



Hình 2.2. *Ước nhỏ tăng dần, ước lớn giảm dần: đảo ngược dãy ước lớn rồi nối vào sau là có dãy ước tăng dần, không cần sắp xếp.*

CHÚ Ý

Số 963 761 198 400 ≤ 10^{12} có tới **6720** ước. Nếu với mỗi khách em duyệt cả dãy ước thì tốn $10^5 \cdot 6720 \approx 6{,}7 \cdot 10^8$ phép so sánh: quá chậm. Hãy tìm kiếm nhị phân (khoảng 13 bước mỗi khách).

Khi n là số chính phương, đừng in ước √n hai lần.

Câu trả lời có thể là chính n (khi x_i lớn hơn mọi ước khác), và là −1 khi $x_i > n$.

HƯỚNG SUY NGHĨ

- Duyệt d tới √n, cho d vào mảng thứ nhất và n / d vào mảng thứ hai. Thứ tự của mỗi mảng là gì?
- Trên một dãy tăng, "phần tử nhỏ nhất ≥ x" tìm được bằng tìm kiếm nhị phân; trong C++ có sẵn hàm `lower_bound`.

BÀI 2.3	SỐ BẠN BÈ	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
BANBE.CPP	BANBE.INP	BANBE.OUT	2 giây · 64 MB

Kiến thức: *Sàng điều hòa tính tổng ước của mọi số tới N (Tập 1, Bài 2, Mở rộng 2).*

Gọi s(n) là tổng các **ước thực sự** của n, tức là các ước dương khác chính n. Chẳng hạn s(12) = 1 + 2 + 3 + 4 + 6 = 16 và s(1) = 0.

Từ thời Pythagoras, người ta đã biết cặp số kì lạ 220 và 284: s(220) = 284 và s(284) = 220. Hai số như vậy gọi là một cặp **số bạn bè**. Các nhà toán học còn xét cặp **số hứa hôn**: s(48) = 76 = 75 + 1 và s(75) = 49 = 48 + 1.

Ta gộp hai khái niệm ấy lại. Cho số nguyên k ≥ 0. Hai số nguyên dương a < b gọi là **bạn bè bậc k** nếu

$$\mathbf{s(a) = b + k \quad và \quad s(b) = a + k.}$$

Như vậy bạn bè bậc 0 là số bạn bè, bạn bè bậc 1 là số hứa hôn. Cho N và k, hãy liệt kê mọi cặp bạn bè bậc k (a, b) với a < b ≤ N.

Dữ liệu vào

Một dòng chứa hai số nguyên N và k.

Kết quả

Dòng đầu chứa P là số cặp tìm được. P dòng tiếp theo, mỗi dòng chứa hai số a, b của một cặp; các cặp được in theo thứ tự a tăng dần.

Ràng buộc

$1 \le N \le 10^7$; $0 \le k \le 10^7$.

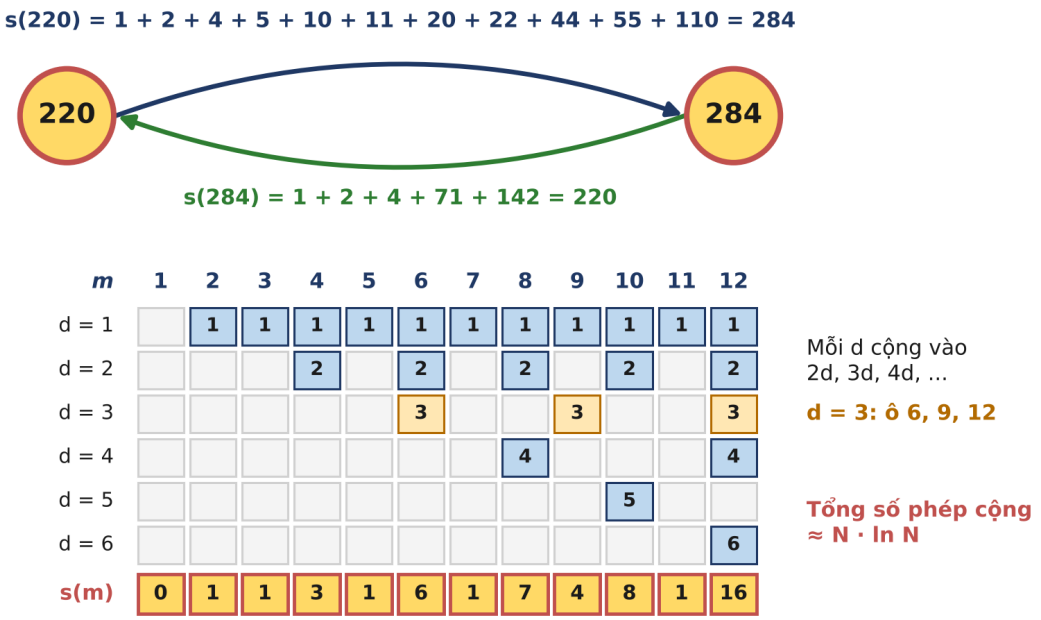
Subtask	Điểm	Ràng buộc thêm
1	20%	$N \le 2000$.
2	30%	$N \le 2 \cdot 10^5$.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

BANBE.INP	BANBE.OUT	Giải thích
300 0	1 220 284	Cặp số bạn bè nhỏ nhất (Hình 2.3).

BANBE.INP	BANBE.OUT	Giải thích
200 1	2 48 75 140 195	s(48) = 76 = 75 + 1, s(75) = 49 = 48 + 1; s(140) = 196 = 195 + 1, s(195) = 141 = 140 + 1.

BANBE.INP	BANBE.OUT	Giải thích
283 0	0	s(220) = 284 nhưng 284 > N = 283, nên cặp (220, 284) không được tính.



Hình 2.3. *Trên: cặp số bạn bè 220 và 284. Dưới: sàng điều hòa với N = 12, mỗi d cộng vào các bội 2d, 3d, ... của nó; dòng cuối là s(m).*

CHÚ Ý

Tính s(n) riêng cho từng n bằng cách duyệt tới √n tốn khoảng $N\sqrt{N} \approx 3{,}2 \cdot 10^{10}$ phép tính với $N = 10^7$: quá chậm. Sàng điều hòa chỉ cần 152 725 364 ≈ $1{,}5 \cdot 10^8$ phép cộng.

Giới hạn bộ nhớ chỉ 64 MB. Mảng 10^7 số `long long` đã tốn 80 MB! Với $n \le 10^7$, s(n) không vượt quá $4 \cdot 10^7$ (σ(n) lớn nhất là 45 732 192 tại n = 9 979 200), nên mảng `int` (40 MB) là đủ.

b = s(a) − k có thể ≤ a, có thể âm, cũng có thể **lớn hơn N**: khi đó không được truy cập s[b] (tràn mảng) và cặp ấy không được tính.

HƯỚNG SUY NGHĨ

- Đừng hỏi "n có những ước nào?" mà hãy hỏi "d là ước thực sự của những số nào?". Câu trả lời: 2d, 3d, 4d, ...
- Có mảng s rồi, với mỗi a chỉ có **đúng một** ứng viên b = s(a) − k. Chỉ cần kiểm tra điều kiện của b.

BÀI 3 Ước chung lớn nhất và bội chung nhỏ nhất

ÔN NHANH KIẾN THỨC · Tập 1, Bài 3

- **Thuật toán Euclid:** gcd(a, b) = gcd(b, a mod b), gcd(a, 0) = a: O(log min(a, b)), chỉ khoảng 85 bước với số ≤ 10^{18} (xấu nhất là hai số Fibonacci liên tiếp).
- **BCNN:** gcd(a, b) · lcm(a, b) = a · b. Chia trước, nhân sau: lcm = a / g · b, và kiểm tra `a / g > LIM / b` trước khi nhân để phát hiện vượt giới hạn.
- **Gộp dần:** gcd(a_1, ..., a_n) = gcd(gcd(a_1, a_2), a_3, ...). Khi gộp dần, gcd chỉ giảm, lcm chỉ tăng, và mỗi lần thay đổi thì gcd giảm ít nhất một nửa, lcm tăng ít nhất gấp đôi.
- **Tiền tố, hậu tố:** L[i] = gcd(a_1..a_i), R[i] = gcd(a_i..a_n); bỏ phần tử i thì gcd còn lại là gcd(L[i − 1], R[i + 1]).

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
3.1	**CẮT GẠCH**	★☆☆ CƠ BẢN	Dùng gcd cho số tới 10^{18}; nhận ra đáp số vượt `long long` và in số `__int128`.
3.2	**BỎ MỘT SỐ**	★★☆ TRUNG BÌNH	Tính trước tiền tố, hậu tố để "bỏ một phần tử" trong O(1).
3.3	**ĐÈN NHÁY**	★★★ NÂNG CAO	Khai thác "chỉ có ít giá trị khác nhau" để trả lời truy vấn trên đoạn; BCNN không tràn số.

BÀI 3.1 CẮT GẠCH ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
CATGACH.CPP	**CATGACH.INP**	**CATGACH.OUT**	**1 giây · 256 MB**

Kiến thức: *Thuật toán Euclid, số nguyên 128 bit (Tập 1, Bài 3).*

Xưởng gốm có những tấm gạch hình chữ nhật kích thước a × b. Mỗi tấm cần được cắt thành các miếng **hình vuông bằng nhau**, cạnh là số nguyên, không thừa mẩu nào. Để đỡ tốn công, xưởng muốn miếng vuông **lớn nhất** có thể.

Với mỗi tấm gạch, hãy cho biết cạnh của miếng vuông và số miếng cắt được.

Dữ liệu vào

Dòng đầu chứa số nguyên T là số tấm gạch. Mỗi dòng trong T dòng tiếp theo chứa hai số nguyên a, b.

Kết quả

Gồm T dòng, mỗi dòng hai số nguyên: cạnh miếng vuông lớn nhất và số miếng cắt được.

Ràng buộc

$1 \le T \le 10^5$; $1 \le a, b \le 10^{18}$.

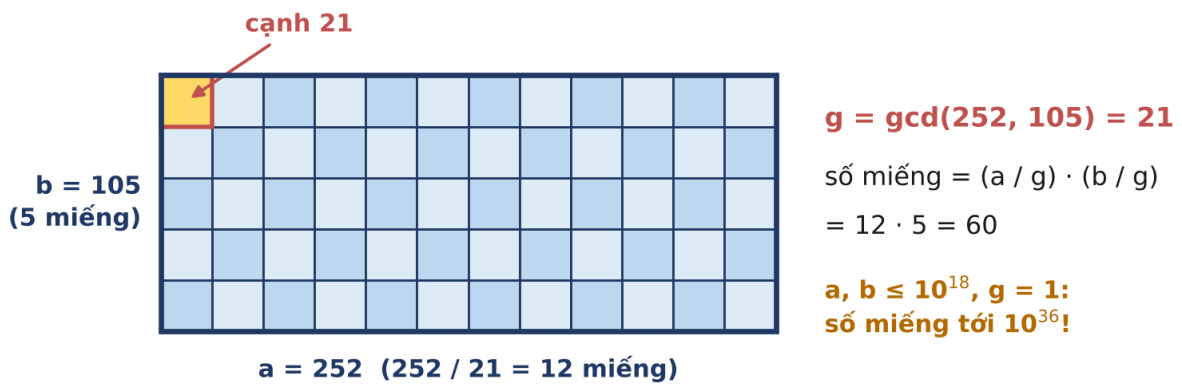
Subtask	Điểm	Ràng buộc thêm
1	30%	$T \le 10$; $a, b \le 10^6$.
2	30%	$a, b \le 10^9$.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

CATGACH.INP	CATGACH.OUT	Giải thích
2 252 105 6 4	21 60 2 6	gcd(252, 105) = 21: tấm gạch chia thành 12 · 5 = **60** miếng (Hình 3.1). gcd(6, 4) = 2: được 3 · 2 = 6 miếng.

CATGACH.INP	CATGACH.OUT
2 1000000000000000000 999999 999999999999999999 3	1 999999000000000000000000 3 333333333333333333

Giải thích ví dụ 2. $10^{18} = 2^{18} \cdot 5^{18}$ và $999\,999 = 3^3 \cdot 7 \cdot 11 \cdot 13 \cdot 37$ nguyên tố cùng nhau, nên chỉ cắt được miếng cạnh 1, gồm $10^{18} \cdot 999\,999 \approx 10^{24}$ miếng: vượt xa `long long`. Số 999 999 999 999 999 999 chia hết cho 3 (tổng chữ số chia hết cho 3), nên tấm 999 999 999 999 999 999 × 3 cắt được 333 333 333 333 333 333 miếng cạnh 3.



Hình 3.1. *Tấm 252 × 105 cắt thành các miếng vuông cạnh gcd(252, 105) = 21: 12 miếng theo chiều dài, 5 miếng theo chiều rộng.*

CHÚ Ý

Số miếng là (a / g) · (b / g), có thể tới gần 10^{36} khi a, b nguyên tố cùng nhau. Kiểu `long long` chỉ chứa tới khoảng $9{,}2 \cdot 10^{18}$; hãy dùng `unsigned __int128` (tới khoảng $3{,}4 \cdot 10^{38}$) và tự viết hàm in, vì `printf` và `cout` không in được kiểu này.

Đừng tính a · b / (g · g): tích a · b có thể tràn ngay cả với `__int128` khi chưa chia. Chia trước: (a / g) · (b / g).

Thử giảm dần cạnh từ min(a, b) chỉ chạy được với số nhỏ; với 10^5 tấm gạch cỡ 10^{18} phải dùng thuật toán Euclid.

HƯỚNG SUY NGHĨ

- Cạnh miếng vuông phải là ước chung của a và b. Lớn nhất là bao nhiêu?
- Số miếng theo chiều dài là a / g, theo chiều rộng là b / g. Để in số 128 bit, hãy tách lần lượt từng chữ số bằng phép chia cho 10.

BÀI 3.2	BỎ MỘT SỐ	★★☆ TRUNG BÌNH	
Chương trình **BOMOTSO.CPP**	Dữ liệu vào **BOMOTSO.INP**	Kết quả **BOMOTSO.OUT**	Giới hạn **1 giây · 256 MB**

Kiến thức: *ƯCLN của dãy, gcd tiền tố và hậu tố (Tập 1, Bài 3, Mở rộng 2).*

Cho dãy n số nguyên dương $a_1, a_2, ..., a_n$. Em được **bỏ đi đúng một** phần tử của dãy. Hãy chọn phần tử bỏ đi sao cho ƯCLN của n − 1 phần tử còn lại là **lớn nhất**.

Nếu có nhiều cách bỏ cùng cho ƯCLN lớn nhất, hãy chọn cách bỏ phần tử có **chỉ số nhỏ nhất**.

Dữ liệu vào

Dòng đầu chứa số nguyên n. Dòng thứ hai chứa n số nguyên $a_1, a_2, ..., a_n$.

Kết quả

Hai số nguyên: ƯCLN lớn nhất đạt được và chỉ số của phần tử bị bỏ.

Ràng buộc

$2 \le n \le 5 \cdot 10^5$; $1 \le a_i \le 10^{18}$.

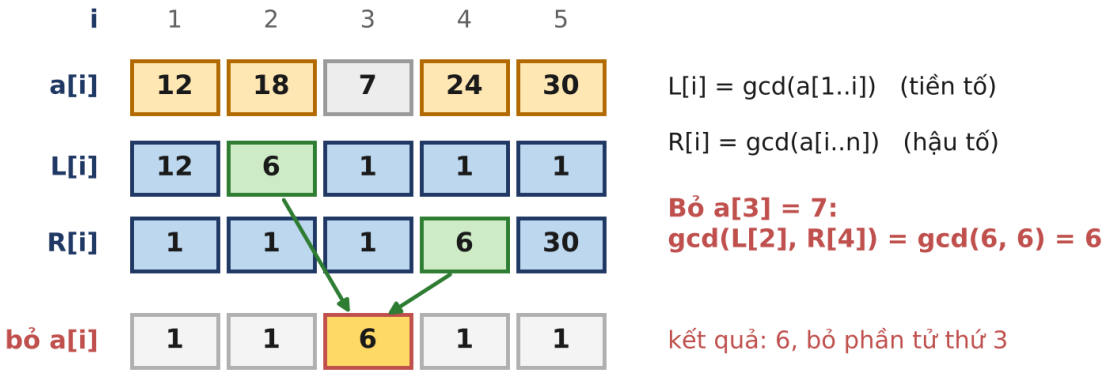
Subtask	Điểm	Ràng buộc thêm
1	40%	$n \le 3000$.
2	60%	Không có ràng buộc gì thêm.

Ví dụ

BOMOTSO.INP	BOMOTSO.OUT	Giải thích
5 12 18 7 24 30	6 3	Bỏ số 7 (chỉ số 3), bốn số còn lại 12, 18, 24, 30 có ƯCLN là **6** (Hình 3.2). Bỏ bất kì số nào khác thì 7 còn lại và ƯCLN chỉ là 1.

BOMOTSO.INP	BOMOTSO.OUT	Giải thích
2 4 6	6 1	n = 2: bỏ 4 còn 6, bỏ 6 còn 4. ƯCLN của dãy chỉ gồm một số là chính số đó, nên bỏ phần tử thứ 1.

BOMOTSO.INP	BOMOTSO.OUT	Giải thích
3 5 5 5	5 1	Bỏ số nào cũng được ƯCLN 5; chọn chỉ số nhỏ nhất là 1.



Hình 3.2. *Ví dụ 1: gcd tiền tố L, gcd hậu tố R; bỏ a[i] thì ƯCLN còn lại là gcd(L[i − 1], R[i + 1]).*

CHÚ Ý

Với mỗi i lại tính ƯCLN của n − 1 số còn lại thì tốn $O(n^2)$ lần gọi gcd: khoảng $2{,}5 \cdot 10^{11}$ với $n = 5 \cdot 10^5$.

Ở hai đầu dãy: bỏ a_1 thì còn R[2], bỏ a_n thì còn L[n − 1]. Đặt L[0] = R[n + 1] = 0 và dùng gcd(0, x) = x để khỏi phải xét riêng.

a_i tới 10^{18}: dùng `long long` cho cả dãy lẫn các mảng L, R.

HƯỚNG SUY NGHĨ

- ƯCLN của dãy sau khi bỏ a_i là ƯCLN của hai "khúc": $a_1..a_{i-1}$ và $a_{i+1}..a_n$.
- Mỗi khúc là một tiền tố hoặc một hậu tố của dãy. Có thể tính trước ƯCLN của mọi tiền tố và mọi hậu tố trong O(n log max) không?

BÀI 3.3	ĐÈN NHÁY	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
DENNHAY.CPP	DENNHAY.INP	DENNHAY.OUT	1 giây · 256 MB

Kiến thức: *BCNN của dãy, phát hiện vượt 10^{18}; BCNN tăng ít nhất gấp đôi mỗi khi thay đổi (Tập 1, Bài 3).*

Trên một con phố có n ngọn đèn trang trí, đánh số từ 1 tới n. Đèn thứ i nháy sáng vào các thời điểm t_i, $2t_i$, $3t_i$, ... (tính bằng giây, kể từ lúc khởi động).

Ban tổ chức muốn thử nhiều kịch bản. Kịch bản thứ j chỉ bật các đèn có số hiệu từ l_j tới r_j. Hỏi sau khi khởi động, thời điểm **đầu tiên** mà mọi đèn đang bật cùng nháy là giây thứ mấy?

Thời điểm ấy có thể rất lớn. Nếu nó **vượt quá 10^{18}**, hãy in −1.

Dữ liệu vào

Dòng đầu chứa hai số nguyên n và Q. Dòng thứ hai chứa n số nguyên t_1, t_2, ..., t_n. Mỗi dòng trong Q dòng tiếp theo chứa hai số nguyên l, r mô tả một kịch bản.

Kết quả

Gồm Q dòng, dòng thứ j là câu trả lời cho kịch bản thứ j.

Ràng buộc

$1 \le n, Q \le 2 \cdot 10^5$; $1 \le t_i \le 10^{18}$; $1 \le l \le r \le n$.

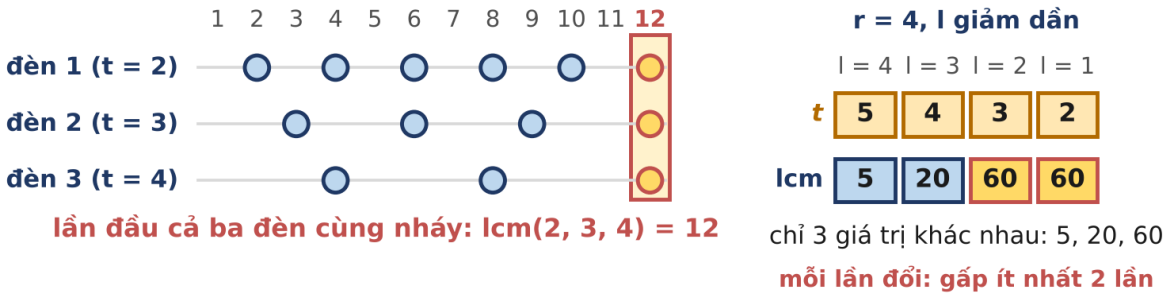
Subtask	Điểm	Ràng buộc thêm
1	20%	n, Q ≤ 2000.
2	30%	l = 1 trong mọi kịch bản.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

DENNHAY.INP	DENNHAY.OUT	Giải thích
4 4 2 3 4 5 1 3 2 2 3 4 1 4	12 3 20 60	Đèn 1, 2, 3 nháy cùng lúc lần đầu ở giây lcm(2, 3, 4) = 12 (Hình 3.3). Chỉ đèn 2: giây thứ 3. lcm(4, 5) = 20. lcm(2, 3, 4, 5) = 60.

DENNHAY.INP	DENNHAY.OUT
4 4 1000000000 1000000000 999999999 1000000000000000000 1 2 1 3 2 4 4 4	1000000000 999999999000000000 -1 1000000000000000000

Giải thích ví dụ 2. $\text{lcm}(10^9, 10^9) = 10^9$. Vì 10^9 và 999 999 999 nguyên tố cùng nhau nên lcm của ba đèn đầu là $999\,999\,999 \cdot 10^9 = 999\,999\,999\,000\,000\,000$, vẫn $\le 10^{18}$. Kịch bản 3 (đèn 2, 3, 4) có BCNN là $999\,999\,999 \cdot 10^{18}$, vượt 10^{18}: in −1. Riêng đèn 4 nháy lần đầu ở giây 10^{18}, đúng bằng giới hạn nên vẫn in ra.



Hình 3.3. *Trái: ba đèn với chu kì 2, 3, 4 cùng nháy lần đầu ở giây 12. Phải: với r = 4 cố định, khi l giảm dần, lcm(t[l..r]) chỉ nhận vài giá trị khác nhau.*

CHÚ Ý

BCNN của hai số $\le 10^{18}$ có thể tới gần 10^{36}. Tính "chia trước, nhân sau" và kiểm tra `x > LIM / y` trước khi nhân; một khi đã vượt 10^{18} thì thêm đèn nào cũng vẫn vượt, nên có thể coi như một giá trị đặc biệt "vô cùng".

Tính lại từ đầu cho mỗi kịch bản tốn tới $n \cdot Q = 4 \cdot 10^{10}$ lần gọi gcd: quá chậm.

Với r cố định, khi l giảm dần, mỗi lần $\text{lcm}(t_l..t_r)$ đổi giá trị thì nó tăng **ít nhất gấp đôi** (bội mới khác bội cũ). Vì $2^{60} > 10^{18}$, chỉ có không quá 60 giá trị khác nhau không vượt 10^{18}.

HƯỚNG SUY NGHĨ

- Subtask l = 1: chỉ cần BCNN của mọi tiền tố, tính dần từ trái sang phải.
- Duyệt r từ 1 tới n và giữ danh sách các đoạn l liên tiếp có cùng giá trị $\text{lcm}(t_l..t_r)$. Khi chuyển sang r + 1, mỗi giá trị v trong danh sách trở thành $\text{lcm}(v, t_{r+1})$; các đoạn kề nhau có cùng giá trị thì gộp lại.
- Đọc hết các kịch bản rồi sắp xếp theo r, để trả lời mỗi kịch bản đúng lúc danh sách ứng với r của nó đang có sẵn.

BÀI 4 Euclid mở rộng và phương trình ax + by = c

ÔN NHANH KIẾN THỨC · Tập 1, Bài 4

- **Bézout:** ax + by = c có nghiệm nguyên ⇔ g = gcd(a, b) chia hết c. Số dương nhỏ nhất viết được dưới dạng ax + by chính là g.
- **Euclid mở rộng:** ở đáy (g, 0) có x = 1, y = 0; lần ngược $x = y_1$, $y = x_1 - \lfloor a / b \rfloor \cdot y_1$. Độ phức tạp O(log min(a, b)).
- **Họ nghiệm:** $x = x_0 + (b / g)t$, $y = y_0 - (a / g)t$. x nhỏ nhất không âm là $x_0 \bmod (b / g)$, lấy theo nghĩa không âm.
- **Phương trình đồng dư** $ax \equiv c \pmod m$ chính là ax + my = c. Cẩn thận tràn số khi nhân x_0 với c / g: dùng `__int128` hoặc lấy dư trước.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
4.1	**ĐONG NƯỚC**	★☆☆ CƠ BẢN	Áp dụng Bézout cho nhiều số; trả lời truy vấn trong O(1).
4.2	**MUA HÀNG**	★★☆ TRUNG BÌNH	Tìm nghiệm có x nhỏ nhất không âm rồi đếm số nghiệm trong một đoạn; tránh tràn số.
4.3	**ĐỒNG HỒ HỎNG**	★★★ NÂNG CAO	Đưa bài toán về phương trình đồng dư; xử lí trường hợp suy biến và tràn số với mô-đun tới 10^{18}.

BÀI 4.1 ĐONG NƯỚC ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`DONGNUOC.CPP`	`DONGNUOC.INP`	`DONGNUOC.OUT`	1 giây · 256 MB

Kiến thức: *Định lí Bézout, ƯCLN của dãy (Tập 1, Bài 3 và Bài 4).*

Một cửa hàng có n loại can, loại thứ i có dung tích đúng a_i lít (số can mỗi loại không hạn chế). Người bán có một chiếc bể rất lớn, ban đầu rỗng. Mỗi thao tác, người bán chọn một loại can và **đổ vào** bể một can đầy nước, hoặc **múc ra** khỏi bể đúng một can đầy.

Một lượng nước c lít gọi là **đong được** nếu sau một số thao tác (có thể không có thao tác nào), bể chứa đúng c lít. Vì bể đủ lớn, người bán có thể đổ vào hết rồi mới múc ra, nên không bao giờ phải múc khi bể không đủ nước.

Hãy cho biết lượng nước **dương nhỏ nhất** đong được, rồi trả lời Q câu hỏi: lượng c_j lít có đong được không?

Dữ liệu vào

Dòng đầu chứa hai số nguyên n và Q. Dòng thứ hai chứa n số nguyên a_1, ..., a_n. Dòng thứ ba chứa Q số nguyên c_1, ..., c_Q.

Kết quả

Dòng đầu là lượng nước dương nhỏ nhất đong được. Tiếp theo là Q dòng, dòng thứ j ghi `CO` nếu đong được c_j lít, ngược lại ghi `KHONG`.

Ràng buộc

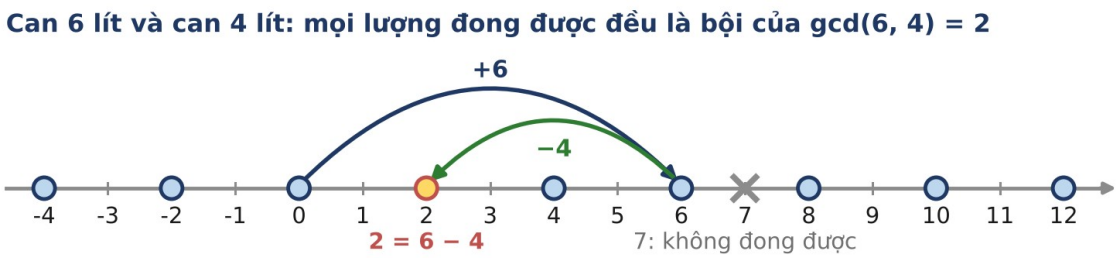
$1 \le n, Q \le 10^5$; $1 \le a_i \le 10^{18}$; $0 \le c_j \le 10^{18}$.

Subtask	Điểm	Ràng buộc thêm
1	30%	$n = 2$; $a_i, c_j \le 1000$.
2	30%	$n = 2$.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

DONGNUOC.INP	DONGNUOC.OUT	Giải thích
2 3 6 4 7 2 10	2 KHONG CO CO	gcd(6, 4) = 2: mọi lượng đong được đều chẵn, nên 7 lít là không thể (Hình 4.1). 2 = 6 − 4: đổ vào một can 6 lít, múc ra một can 4 lít. 10 = 6 + 4.

DONGNUOC.INP	DONGNUOC.OUT	Giải thích
3 3 6 10 15 1 0 1000000000000000000	1 CO CO CO	Từng cặp can có ƯCLN lớn hơn 1 (gcd(6, 10) = 2, gcd(6, 15) = 3, gcd(10, 15) = 5), nhưng cả ba can thì có ƯCLN bằng 1: 6 + 10 − 15 = 1. 0 lít: không làm gì cả. 10^{18} là bội của 1.



Hình 4.1. *Mỗi thao tác là một bước nhảy ±6 hoặc ±4 trên trục số; chỉ tới được các bội của 2.*

CHÚ Ý

Dù có thể múc ra, lượng nước đong được vẫn không âm (đề bài chỉ hỏi $c \geq 0$); thứ tự thao tác không quan trọng, chỉ có **số lần** đổ vào và múc ra của mỗi loại can là đáng kể.

Định lí Bézout mở rộng cho nhiều số: tập các tổng $a_1x_1 + ... + a_nx_n$ (x_i nguyên) đúng bằng tập các bội của $\gcd(a_1, ..., a_n)$.

$c = 0$ luôn đong được. Giá trị tới 10^{18}: dùng `long long`.

HƯỚNG SUY NGHĨ

- Với hai loại can, lượng nước đong được có dạng $a_1x + a_2y$ với x, y nguyên (âm nghĩa là múc ra). Định lí Bézout nói gì?
- Thêm loại can thứ ba a_3 thì $a_1x + a_2y$ "đi được" mọi bội của $g = \gcd(a_1, a_2)$, nên bài toán trở thành $gx' + a_3z$: tiếp tục gộp dần.

BÀI 4.2	MUA HÀNG	★★☆ TRUNG BÌNH	
Chương trình **MUAHANG.CPP**	Dữ liệu vào **MUAHANG.INP**	Kết quả **MUAHANG.OUT**	Giới hạn **1 giây · 256 MB**

Kiến thức: *Euclid mở rộng, họ nghiệm, đếm nghiệm tự nhiên (Tập 1, Bài 4, Mở rộng 1).*

Siêu thị bán hai loại hàng: mỗi gói kẹo giá a đồng và mỗi hộp bánh giá b đồng. Em có đúng c đồng và muốn **tiêu hết** số tiền, không thừa đồng nào.

Hỏi có bao nhiêu cách mua? Mỗi cách mua là một cặp số nguyên (x, y) với $\mathbf{x \geq 0, y \geq 0}$ và $a \cdot x + b \cdot y = c$, trong đó x là số gói kẹo và y là số hộp bánh. Em có Q câu hỏi như vậy.

Dữ liệu vào

Dòng đầu chứa số nguyên Q. Mỗi dòng trong Q dòng tiếp theo chứa ba số nguyên a, b, c.

Kết quả

Gồm Q dòng, dòng thứ i là số cách mua của câu hỏi thứ i.

Ràng buộc

$1 \leq Q \leq 10^5$; $1 \leq a, b \leq 10^9$; $0 \leq c \leq 10^{18}$.

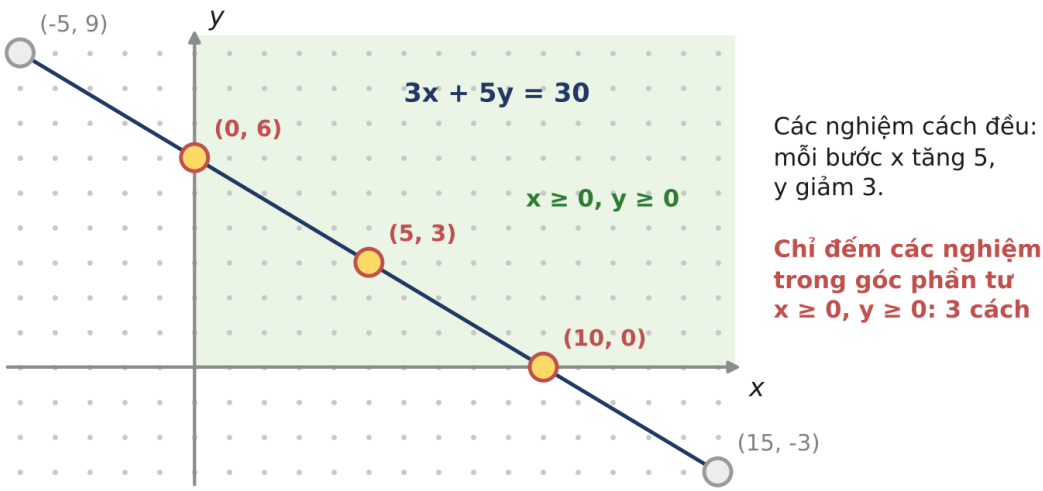
Subtask	Điểm	Ràng buộc thêm
1	30%	$Q \leq 10$; $c \leq 10^7$.
2	20%	$Q \leq 10$; $a, b \leq 10^6$.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

MUAHANG.INP	MUAHANG.OUT	Giải thích
3 3 5 30 6 4 7 3 5 7	3 0 0	$3x + 5y = 30$ có ba nghiệm tự nhiên (0, 6), (5, 3), (10, 0) (Hình 4.2). gcd(6, 4) = 2 không chia hết 7. $3x + 5y = 7$ có nghiệm nguyên (4, −1), nhưng không có nghiệm nào với x, y ≥ 0: $7 = 3 \cdot 5 - 3 - 5$ là số lớn nhất không mua được.

MUAHANG.INP	MUAHANG.OUT
3 1 1 1000000000000000000 1000000000 1000000000 1000000000000000000 2 3 0	1000000000000000001 1000000001 1

Giải thích ví dụ 2. Với a = b = 1, mọi x từ 0 tới 10^{18} đều được: $10^{18} + 1$ cách, đáp số vẫn vừa `long long`. Với $a = b = 10^9$, phương trình trở thành $x + y = 10^9$: $10^9 + 1$ cách. Với c = 0 có đúng một cách: không mua gì.



Hình 4.2. *Các nghiệm nguyên của 3x + 5y = 30 nằm cách đều trên đường thẳng; chỉ ba nghiệm nằm trong góc $x \ge 0, y \ge 0$.*

CHÚ Ý

Thử mọi x từ 0 tới c / a tốn tới 10^{18} bước. Kể cả khi chỉ thử x trong một chu kì b / g (để tìm nghiệm đầu tiên) cũng tốn tới 10^{9} bước cho mỗi câu hỏi.

Sau khi chia cho g, gọi a' = a / g, b' = b / g, c' = c / g. Các giá trị x của nghiệm cách nhau đúng b'. Hãy chắc chắn rằng $a' \cdot x_0$ và a' · b' không tràn `long long` (cả hai không vượt 10^{18} khi $0 \le x_0 < b'$).

Phép nhân $x_0 \cdot (c / g)$ để có một nghiệm có thể lên tới 10^{27}: lấy dư c' theo b' trước khi nhân, hoặc dùng `__int128`.

Đáp số có thể là 0, và có thể lên tới $10^{18} + 1$.

HƯỚNG SUY NGHĨ

- Mọi nghiệm có dạng $x = x_0 + b't$, $y = y_0 - a't$. Nghiệm có x nhỏ nhất không âm là bao nhiêu? Nếu ở nghiệm ấy y đã âm thì sao?
- Từ nghiệm (x_{min}, y), mỗi bước x tăng b' và y giảm a'. Đếm số bước cho tới khi y còn không âm: một phép chia nguyên.

BÀI 4.3	ĐỒNG HỒ HỎNG	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
DONGHO.CPP	DONGHO.INP	DONGHO.OUT	1 giây · 256 MB

Kiến thức: *Phương trình đồng dư ax ≡ c (mod m), Euclid mở rộng với số 64 bit (Tập 1, Bài 4, Mở rộng 3).*

Một chiếc đồng hồ kì lạ có mặt chia thành m nấc, đánh số 0, 1, ..., m − 1 theo chiều kim đồng hồ. Đồng hồ có hai kim A và B, nhưng cả hai đều đã hỏng:

- Lúc khởi động (giây thứ 0), kim A chỉ nấc p, kim B chỉ nấc q.
- Cứ mỗi giây, kim A nhảy a nấc và kim B nhảy b nấc theo chiều kim đồng hồ (hai kim nhảy cùng lúc). Như vậy ở giây thứ t, kim A chỉ nấc $(p + a \cdot t) \bmod m$ và kim B chỉ nấc $(q + b \cdot t) \bmod m$.

Hai kim gọi là **trùng nhau** ở giây thứ t nếu chúng chỉ cùng một nấc. Hãy tìm giây $t \geq 0$ **nhỏ nhất** mà hai kim trùng nhau, và đếm xem trong các giây 0, 1, 2, ..., T có bao nhiêu giây hai kim trùng nhau.

Dữ liệu vào

Dòng đầu chứa số nguyên Q là số đồng hồ. Mỗi dòng trong Q dòng tiếp theo chứa sáu số nguyên m, p, a, q, b, T mô tả một đồng hồ.

Kết quả

Gồm Q dòng, mỗi dòng hai số nguyên: giây đầu tiên hai kim trùng nhau và số giây trùng nhau trong đoạn [0, T]. Nếu hai kim không bao giờ trùng nhau, in `-1 0`.

Ràng buộc

$1 \leq Q \leq 10^5$; $1 \leq m \leq 10^{18}$; $0 \leq p, a, q, b < m$; $0 \leq T \leq 10^{18}$.

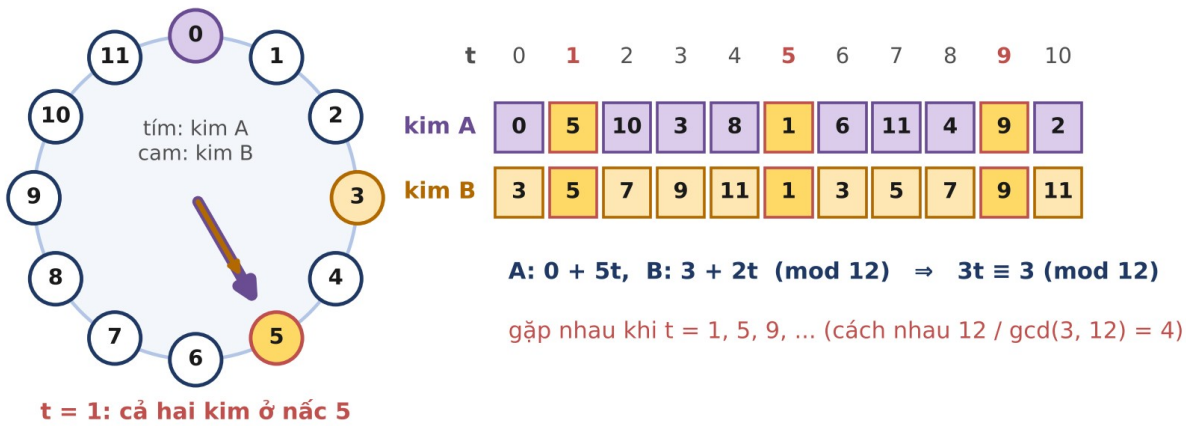
Subtask	Điểm	Ràng buộc thêm
1	20%	$Q \leq 10$; $m \leq 1000$; $T \leq 10^6$.
2	30%	$m \leq 10^9$.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

DONGHO.INP	DONGHO.OUT	Giải thích
3 12 0 5 3 2 10 12 0 4 1 0 100 60 7 13 7 13 5	1 3 -1 0 0 6	Hai kim trùng nhau ở các giây 1, 5, 9, ...; trong [0, 10] có 3 giây (Hình 4.3). Kim A chỉ đi qua các nấc 0, 4, 8, còn kim B đứng yên ở nấc 1: không bao giờ trùng. Hai kim xuất phát cùng nấc, nhảy cùng tốc độ: trùng nhau ở cả 6 giây 0, 1, ..., 5.

DONGHO.INP	DONGHO.OUT
2 1000000000000000000 0 3 1 0 1000000000000000000 1 0 0 0 0 1000000000000000000	666666666666666667 1 0 1000000000000000001

Giải thích ví dụ 2. Đồng hồ thứ nhất: kim B đứng yên ở nấc 1, cần $3t \equiv 1 \pmod{10^{18}}$. Vì $3 \cdot 666\,666\,666\,666\,666\,667 = 2 \cdot 10^{18} + 1$, giây đầu tiên là t = 666 666 666 666 666 667; giây tiếp theo đã vượt T. Đồng hồ thứ hai chỉ có một nấc nên hai kim luôn trùng nhau: $10^{18} + 1$ giây.



Hình 4.3. *Ví dụ 1, đồng hồ thứ nhất: m = 12, kim A từ nấc 0 nhảy 5 nấc, kim B từ nấc 3 nhảy 2 nấc mỗi giây.*

CHÚ Ý

Hai kim trùng nhau $\Leftrightarrow (a - b) \cdot t \equiv q - p \pmod{m}$. Hãy đưa a − b và q − p về số dư không âm trước khi giải.

Khi a = b, hệ số bằng 0: hoặc hai kim luôn trùng nhau (p = q: in 0 và T + 1), hoặc không bao giờ. Thuật toán Euclid mở rộng với gcd(0, m) = m vẫn xử lí đúng trường hợp này nếu em cài đặt cẩn thận.

Với m tới 10^{18}, tích của hai số dư theo mô-đun m có thể tới 10^{36}: phải nhân bằng `__int128`. Số giây trùng nhau có thể tới $10^{18} + 1$, vẫn vừa `long long`.

Mô phỏng từng giây chỉ dùng được cho subtask 1: các vị trí lặp lại sau m giây, nhưng m có thể tới 10^{18}.

HƯỚNG SUY NGHĨ

- Đặt $d = (a - b) \bmod m$, $e = (q - p) \bmod m$. Theo Bézout, $d \cdot t \equiv e \pmod{m}$ có nghiệm khi và chỉ khi g = gcd(d, m) chia hết e.
- Nếu có nghiệm, các nghiệm t cách nhau đúng m / g. Nghiệm nhỏ nhất không âm tìm được bằng Euclid mở rộng; số nghiệm trong [0, T] là một phép chia nguyên.

CHƯƠNG 2. SỐ NGUYÊN TỐ

Mười hai bài tập của chương này luyện bốn công cụ làm việc với số nguyên tố: thử chia tới $\sqrt{n}$ và Miller–Rabin để kiểm tra một số, sàng Eratosthenes cho mọi số tới 10^7, sàng ước nguyên tố nhỏ nhất để phân tích hàng triệu số, và sàng trên đoạn để "nhìn" vào vùng số cỡ 10^{12}. Bẫy thường gặp nhất là tràn số ($i \cdot i$, tích hai số 64 bit, tổng các số nguyên tố) và các trường hợp biên quen thuộc: số 1, số 2, bình phương của một số nguyên tố, đoạn chứa chính "thợ gạch" p.

TRONG CHƯƠNG NÀY

Bài 5. Số nguyên tố và kiểm tra nguyên tố

- **5.1 NGUYÊN TỐ GẦN NHẤT** ★☆☆ CƠ BẢN
- **5.2 TỔNG HAI SỐ NGUYÊN TỐ** ★★☆ TRUNG BÌNH
- **5.3 NGUYÊN TỐ 64 BIT** ★★★ NÂNG CAO

Bài 6. Sàng Eratosthenes

- **6.1 ĐẾM SỐ NGUYÊN TỐ** ★☆☆ CƠ BẢN
- **6.2 SỐ NGUYÊN TỐ SINH ĐÔI** ★★☆ TRUNG BÌNH
- **6.3 SỐ CÓ ĐÚNG BA ƯỚC** ★★★ NÂNG CAO

Bài 7. Phân tích thừa số nguyên tố

- **7.1 SỐ ƯỚC** ★☆☆ CƠ BẢN
- **7.2 CHỮ SỐ 0 CỦA GIAI THỪA** ★★☆ TRUNG BÌNH
- **7.3 BCNN CỦA DÃY** ★★★ NÂNG CAO

Bài 8. Sàng trên đoạn

- **8.1 NGUYÊN TỐ TRÊN ĐOẠN** ★☆☆ CƠ BẢN
- **8.2 KHOẢNG CÁCH LỚN NHẤT** ★★☆ TRUNG BÌNH
- **8.3 SỐ ƯỚC TRÊN ĐOẠN** ★★★ NÂNG CAO

LỜI KHUYÊN CHO CHƯƠNG NÀY

- Trước khi chọn thuật toán, hãy ước lượng: có bao nhiêu số cần xét, mỗi số lớn tới đâu? $n \le 10^7$ thì sàng; $n \le 10^{12}$ thì thử chia tới $\sqrt{n}$ hoặc sàng đoạn; n tới 10^{18} hoặc 2^{64} thì Miller–Rabin.
- Một mảng sàng hay mảng spf có thể chiếm hàng chục MB: tính trước bộ nhớ và khai báo mảng lớn ở phạm vi toàn cục.
- Luôn thử chương trình với 1, 2, 4, p^2 (chẳng hạn $999\,966\,000\,289 = 999\,983^2$) và các đoạn có $L = 1$.

BÀI 5 Số nguyên tố và kiểm tra nguyên tố

ÔN NHANH KIẾN THỨC · Tập 1, Bài 5

- **Số nguyên tố** là số $p > 1$ chỉ có hai ước dương 1 và p. Số 0 và 1 không nguyên tố; 2 là số nguyên tố chẵn duy nhất; mọi số nguyên tố $p > 3$ có dạng $6k \pm 1$.
- **Thử chia tới √n:** hợp số n luôn có ước nguyên tố $\le \sqrt{n}$. Thử 2, 3 rồi các cặp i, i + 2 (i = 5, 11, 17, ...) trong khi `i * i <= n`, với i kiểu `long long`: khoảng $\sqrt{n}/3$ phép thử, tức 333 334 với $n \approx 10^{12}$.
- **Miller–Rabin:** viết $n - 1 = d \cdot 2^s$ (d lẻ). Nếu n nguyên tố thì dãy $a^d, a^{2d}, ..., a^{n-1}$ (mod n) bắt đầu bằng 1 hoặc chứa $n - 1$ trước số 1 đầu tiên. Thử 12 cơ sở 2, 3, 5, ..., 37 là đúng cho **mọi** $n < 2^{64}$; nhân theo mô-đun bằng `__int128`.
- Bẫy kinh điển: n = 1, bình phương của số nguyên tố ($999\,966\,000\,289 = 999\,983^2$), số Carmichael $561 = 3 \cdot 11 \cdot 17$ lừa được phép thử Fermat.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
5.1	**NGUYÊN TỐ GẦN NHẤT**	★☆☆ CƠ BẢN	Cài đặt đúng và nhanh phép kiểm tra nguyên tố; duyệt "loang" ra hai phía.
5.2	**TỔNG HAI SỐ NGUYÊN TỐ**	★★☆ TRUNG BÌNH	Dùng tính chẵn lẻ để thu hẹp tìm kiếm; sắp xếp các phép kiểm tra rẻ trước, đắt sau.
5.3	**NGUYÊN TỐ 64 BIT**	★★★ NÂNG CAO	Cài đặt Miller–Rabin không tràn số cho mọi số không dấu 64 bit.

BÀI 5.1 NGUYÊN TỐ GẦN NHẤT ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`NTGANNHAT.CPP`	`NTGANNHAT.INP`	`NTGANNHAT.OUT`	1 giây · 256 MB

Kiến thức: *Kiểm tra nguyên tố bằng thử chia tới √n với các số $6k \pm 1$ (Tập 1, Bài 5).*

Câu lạc bộ Tin học chơi trò "đoán số nguyên tố". Quản trò đọc một số tự nhiên n; ai nói ra được **số nguyên tố gần n nhất** sẽ thắng. Số nguyên tố p gần n nhất là số nguyên tố làm cho $|p - n|$ nhỏ nhất. Nếu có hai số nguyên tố cách đều n (một số nhỏ hơn n, một số lớn hơn n), ta chọn **số nhỏ hơn**. Nếu chính n là số nguyên tố thì đáp số là n.

Em hãy viết chương trình làm trọng tài cho Q lượt chơi.

Dữ liệu vào

Dòng đầu chứa số nguyên Q. Mỗi dòng trong Q dòng tiếp theo chứa một số nguyên n.

Kết quả

Gồm Q dòng, dòng thứ i là số nguyên tố gần nhất với số n của lượt thứ i.

Ràng buộc

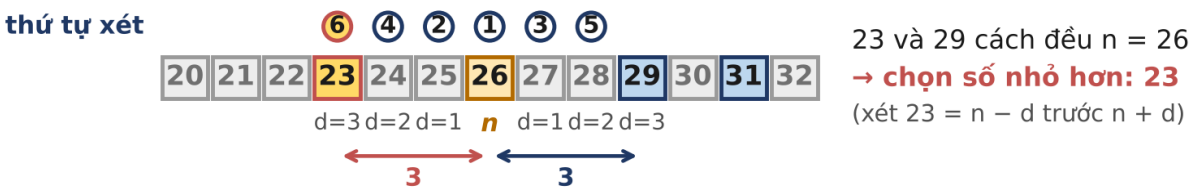
$1 \le Q \le 100$; $1 \le n \le 10^{12}$.

Subtask	Điểm	Ràng buộc thêm
1	30%	$n \le 1000$.
2	30%	$n \le 10^9$.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

NTGANNHAT.INP	NTGANNHAT.OUT	Giải thích
5 1 10 12 26 97	2 11 11 23 97	n = 1: số nguyên tố duy nhất gần nhất là 2 (0 và 1 không nguyên tố). 10 cách 11 một đơn vị, còn $9 = 3^2$ là hợp số: đáp số **11**. 12 cách đều 11 và 13: chọn số nhỏ hơn, **11**. Tương tự 26 cách đều 23 và 29. 97 là số nguyên tố.

NTGANNHAT.INP	NTGANNHAT.OUT	Giải thích
3 999999549106 1000000000000 999999999989	999999548953 999999999989 999999999989	999 999 549 106 nằm chính giữa hai số nguyên tố liên tiếp 999 999 548 953 và 999 999 549 259 (khoảng trống dài 306): cách đều 153, chọn số nhỏ hơn. Số nguyên tố lớn nhất không vượt quá 10^{12} là 999 999 999 989 (cách 11), còn số nguyên tố kế tiếp là $10^{12} + 39$.



Hình 5.1. *Loang ra hai phía từ n = 26: xét lần lượt d = 0, 1, 2, 3 và ở mỗi d xét n − d trước n + d. Số nguyên tố gặp đầu tiên là đáp số.*

CHÚ Ý

Với $n \le 10^{12}$ thì $\sqrt{n} \le 10^6$; nếu biến chạy i là `int` thì `i * i` bị tràn số trước khi vượt n. Hãy khai báo `long long`.

Đừng quên n = 1 và n = 2. Hàm kiểm tra phải trả lời "không" với 0 và 1: nếu không, n = 1 sẽ cho đáp số 1, thậm chí 0.

Khoảng cách giữa hai số nguyên tố liên tiếp gần 10^{12} chỉ cỡ vài trăm (trong đoạn $[10^{12} - 10^7, 10^{12}]$, khoảng trống dài nhất là 306), nên số lần gọi hàm kiểm tra rất ít; phần lớn số bị loại ngay ở phép chia cho 2, 3, 5, 7.

HƯỚNG SUY NGHĨ

- Gọi d là khoảng cách cần tìm. Thử d = 0, 1, 2, ...: với mỗi d, kiểm tra n − d rồi mới tới n + d. Thứ tự ấy tự động chọn số nhỏ hơn khi cách đều.
- Mỗi lần kiểm tra một số nguyên tố cỡ 10^{12} tốn khoảng 333 334 phép chia. Mỗi lượt chơi chỉ có **đúng một** lần kiểm tra trả lời "có", nên tổng thời gian chủ yếu là Q lần kiểm tra như thế.

BÀI 5.2	TỔNG HAI SỐ NGUYÊN TỐ	★★☆ TRUNG BÌNH

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
GOLDBACH.CPP	GOLDBACH.INP	GOLDBACH.OUT	1 giây · 256 MB

Kiến thức: *Kiểm tra nguyên tố, số nguyên tố chẵn duy nhất (Tập 1, Bài 5).*

Năm 1742, Christian Goldbach viết thư cho Euler nêu một giả thuyết mà tới nay vẫn chưa ai chứng minh được: **mọi số chẵn lớn hơn 2 đều là tổng của hai số nguyên tố**. Bằng máy tính, người ta đã kiểm chứng giả thuyết này cho mọi số chẵn tới $4 \cdot 10^{18}$.

Cho Q số tự nhiên n (chẵn hoặc lẻ). Với mỗi số, hãy viết n = p + q với p, q là hai số nguyên tố, p ≤ q. Nếu có nhiều cách, chọn cách có **p nhỏ nhất**. Nếu không có cách nào, ghi −1.

Dữ liệu vào

Dòng đầu chứa số nguyên Q. Mỗi dòng trong Q dòng tiếp theo chứa một số nguyên n.

Kết quả

Gồm Q dòng. Dòng thứ i ghi hai số p, q tìm được cho số n thứ i, cách nhau một dấu cách; hoặc ghi −1.

Ràng buộc

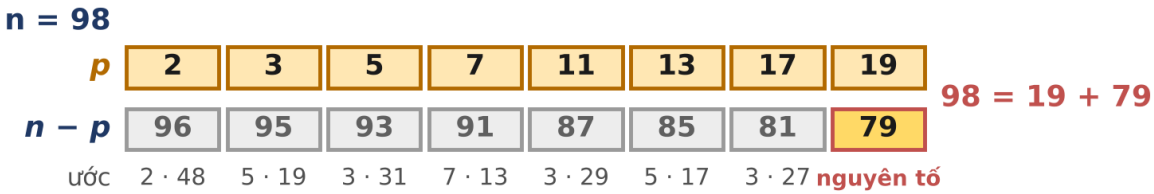
$1 \le Q \le 100$; $2 \le n \le 10^{12}$.

Subtask	Điểm	Ràng buộc thêm
1	30%	$n \le 10^6$ và n chẵn.
2	30%	n chẵn.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

GOLDBACH.INP	GOLDBACH.OUT	Giải thích
5 4 10 98 9 11	2 2 3 7 19 79 2 7 -1	4 = 2 + 2; 10 = 3 + 7 (cách 5 + 5 có p lớn hơn). 98 = 19 + 79: phải thử tới p = 19 (Hình 5.2). 9 lẻ: 9 = 2 + 7. Còn 11 = 2 + 9 mà 9 không nguyên tố, nên ghi −1.

GOLDBACH.INP	GOLDBACH.OUT	Giải thích
4 2 999999634148 1000000000000 999999999991	-1 1669 999999632479 11 999999999989 2 999999999989	n = 2 không viết được (2 = 1 + 1, mà 1 không nguyên tố). Trong mọi số chẵn của đoạn $[10^{12} - 2 \cdot 10^6, 10^{12}]$, số 999 999 634 148 bắt ta thử lâu nhất: p = 1669 là số nguyên tố thứ 263. 10^{12} = 11 + 999 999 999 989. 999 999 999 991 lẻ và 999 999 999 989 nguyên tố.



Hình 5.2. *Thử p = 2, 3, 5, ... cho n = 98. Ô xám: n − p là hợp số (ghi một ước để kiểm chứng). Lần đầu tiên n − p nguyên tố là ở p = 19.*

CHÚ Ý

Nếu n **lẻ** thì p và q khác tính chẵn lẻ, nên một trong hai phải là số chẵn, tức là bằng 2. Đừng chạy vòng lặp tới n / 2 với n lẻ cỡ 10^{12}: chương trình sẽ không bao giờ dừng khi n − 2 là hợp số.

Kiểm tra p trước (p nhỏ, rất nhanh), chỉ khi p nguyên tố mới kiểm tra n − p (cỡ 10^{12}, đắt hơn nhiều). Toán tử `&&` của C++ tự bỏ qua vế phải khi vế trái sai.

n = 2 và n = 3 đều cho −1; n = 4 cho đáp số 2 2 (p được phép bằng q).

HƯỚNG SUY NGHĨ

- Với n chẵn, thử p = 2, 3, 5, 7, ... cho tới khi n − p cũng nguyên tố. Thực nghiệm cho thấy p cần thử rất nhỏ: với mọi số chẵn trong $[10^{12} - 2 \cdot 10^6, 10^{12}]$, p lớn nhất cần thử là 1669.
- Đa số các số n − p là hợp số có ước nhỏ, nên hàm kiểm tra dừng ngay sau vài phép chia; chỉ lần cuối cùng (n − p nguyên tố) mới phải chạy đủ √n / 3 lượt.

BÀI 5.3	NGUYÊN TỐ 64 BIT	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`NT64.CPP`	`NT64.INP`	`NT64.OUT`	2 giây · 256 MB

Kiến thức: *Kiểm tra Miller–Rabin tất định, lũy thừa nhanh theo mô-đun, nhân* `__int128` *(Tập 1, Bài 5, Mở rộng 1).*

Kiểu `unsigned long long` của C++ lưu được mọi số nguyên từ 0 tới $2^{64} - 1 = 18\,446\,744\,073\,709\,551\,615$. Cho Q số trong phạm vi đó, hãy cho biết số nào là số nguyên tố.

Với những số lớn như vậy, thử chia tới $\sqrt{n}$ (tới hơn 4 tỉ) là quá chậm. Em cần cài đặt phép kiểm tra **Miller–Rabin** một cách chính xác tuyệt đối: bộ test có nhiều "kẻ giả danh", những hợp số qua mặt được Miller–Rabin với rất nhiều cơ sở.

Dữ liệu vào

Dòng đầu chứa số nguyên Q. Mỗi dòng trong Q dòng tiếp theo chứa một số nguyên n.

Kết quả

Gồm Q dòng, dòng thứ i ghi `YES` nếu số thứ i là số nguyên tố, ngược lại ghi `NO`.

Ràng buộc

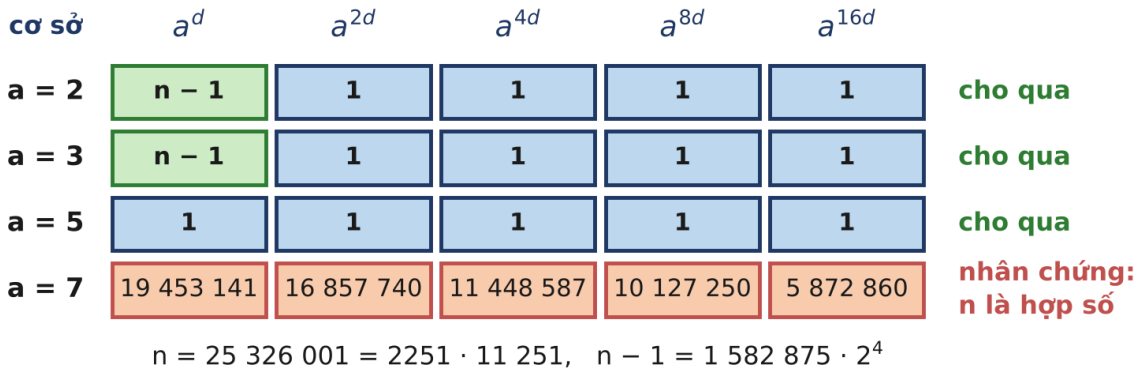
$1 \le Q \le 10^5$; $1 \le n \le 2^{64} - 1$.

Subtask	Điểm	Ràng buộc thêm
1	20%	$Q \le 100$, $n \le 10^{12}$.
2	50%	$n \le 10^{18}$.
3	30%	Không có ràng buộc gì thêm.

Ví dụ

NT64.INP	NT64.OUT	Giải thích
6 1 2 97 561 1000000007 999966000289	NO YES YES NO YES NO	1 không nguyên tố; 2 và 97 nguyên tố. 561 = 3 · 11 · 17 là số Carmichael: phép thử Fermat bị lừa với mọi cơ sở nguyên tố cùng nhau với 561. $10^9 + 7$ nguyên tố; $999\,966\,000\,289 = 999\,983^2$.

NT64.INP	NT64.OUT	Giải thích
4 25326001 3825123056546413051 18446744073709551557 18446744073709551615	NO NO YES NO	25 326 001 = 2251 · 11 251 qua mặt các cơ sở 2, 3, 5 và bị cơ sở 7 vạch mặt (Hình 5.3). 3 825 123 056 546 413 051 = 149 491 · 747 451 · 34 233 211 qua mặt **11 cơ sở** 2, 3, 5, ..., 31; chỉ cơ sở 37 phát hiện được. $2^{64} - 59$ là số nguyên tố lớn nhất nhỏ hơn 2^{64}; $2^{64} - 1 = 3 \cdot 5 \cdot 17 \cdot 257 \cdot 641 \cdot 65\,537 \cdot 6\,700\,417$.



Hình 5.3. *Dãy a^d, a^{2d}, ..., $a^{16d} = a^{n-1} \pmod n$ với $n = 25\,326\,001$, $n - 1 = 1\,582\,875 \cdot 2^4$. Ba cơ sở đầu "đúng mực" như với một số nguyên tố; cơ sở 7 không bao giờ chạm $n - 1$: n là hợp số.*

CHÚ Ý

n tới $2^{64} - 1$ vượt quá `long long` (lớn nhất $2^{63} - 1 \approx 9{,}22 \cdot 10^{18}$): hãy đọc bằng `unsigned long long` và `%llu`.

Tích hai số dư cỡ 2^{64} có tới 128 bit. Ép kiểu `(unsigned __int128)a * b % n` trước khi nhân; viết `a * b % n` với kiểu 64 bit sẽ tràn số mà không báo lỗi.

Bớt cơ sở để chạy nhanh hơn là bẫy chết người: ví dụ 2 cho thấy một hợp số qua mặt 11 cơ sở nguyên tố đầu tiên. Với $n < 2^{64}$, dùng đủ 12 cơ sở 2, 3, 5, ..., 37.

Xử lí riêng $n < 2$ và n chia hết cho một cơ sở ($n = 2, 3, ..., 37$ là số nguyên tố; bội thật sự của chúng là hợp số), nếu không thì $a^d \bmod n$ có thể bằng 0 và kết luận sai.

HƯỚNG SUY NGHĨ

- Lũy thừa nhanh (sẽ học kĩ ở Bài 9): $a^e = (a^{e/2})^2$ khi e chẵn, $= a \cdot a^{e-1}$ khi e lẻ; chỉ cần khoảng 64 phép bình phương.
- Với mỗi cơ sở a: tính $x = a^d \bmod n$. Nếu $x = 1$ hoặc $x = n - 1$ thì a "cho qua". Nếu không, bình phương x tối đa $s - 1$ lần; gặp $n - 1$ thì cho qua, không gặp thì n chắc chắn là hợp số.

BÀI 6 Sàng Eratosthenes

ÔN NHANH KIẾN THỨC · Tập 1, Bài 6

- **Sàng Eratosthenes:** với mỗi p chưa bị gạch mà $p \cdot p \le N$, gạch p^2, $p^2 + p$, $p^2 + 2p$, ... $\le N$. Các số ≥ 2 còn lại là số nguyên tố. Độ phức tạp $O(N \log \log N)$: với $N = 10^7$ chỉ khoảng $2{,}3 \cdot 10^7$ phép gạch.
- **Mảng tiền tố:** c[x] = c[x − 1] + [x nguyên tố]; số số nguyên tố trong [a, b] là c[b] − c[a − 1], mỗi truy vấn O(1).
- $\pi(10^7) = 664\,579$; $\pi(N)$ xấp xỉ $N / \ln N$. Hợp số x có ước nguyên tố $\le \sqrt{x}$, nên muốn kiểm tra mọi số tới 10^{12} chỉ cần các số nguyên tố tới 10^6.
- Bộ nhớ: mảng `char` 10^7 phần tử là 10 MB, mảng `int` là 40 MB. Với 10^6 dòng vào/ra, dùng `scanf`/`printf` hoặc gom kết quả vào một xâu rồi in một lần.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
6.1	**ĐẾM SỐ NGUYÊN TỐ**	★☆☆ CƠ BẢN	Sàng một lần, trả lời hai loại truy vấn bằng hai mảng.
6.2	**SỐ NGUYÊN TỐ SINH ĐÔI**	★★☆ TRUNG BÌNH	Thiết kế mảng tiền tố cho một "cặp" thay vì một số; xử lí biên của đoạn.
6.3	**SỐ CÓ ĐÚNG BA ƯỚC**	★★★ NÂNG CAO	Nhận dạng số có đúng ba ước; chỉ cần thử chia tới căn bậc ba thay vì căn bậc hai.

BÀI 6.1 ĐẾM SỐ NGUYÊN TỐ ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
DEMNGUYENTO.CPP	**DEMNGUYENTO.INP**	**DEMNGUYENTO.OUT**	**1 giây · 256 MB**

Kiến thức: *Sàng Eratosthenes, mảng tiền tố, danh sách số nguyên tố (Tập 1, Bài 6).*

Kí hiệu $\pi(n)$ là số các số nguyên tố không vượt quá n, và p_k là số nguyên tố thứ k khi xếp tăng dần ($p_1 = 2$, $p_2 = 3$, $p_3 = 5$, ...).

Em nhận được Q truy vấn thuộc một trong hai loại:

- `1 n`: hãy cho biết $\pi(n)$;
- `2 k`: hãy cho biết p_k.

Dữ liệu vào

Dòng đầu chứa số nguyên Q. Mỗi dòng trong Q dòng tiếp theo chứa hai số nguyên t và x mô tả một truy vấn: t = 1 thì x là n, t = 2 thì x là k.

Kết quả

Gồm Q dòng, mỗi dòng là câu trả lời của một truy vấn, theo thứ tự trong dữ liệu vào.

Ràng buộc

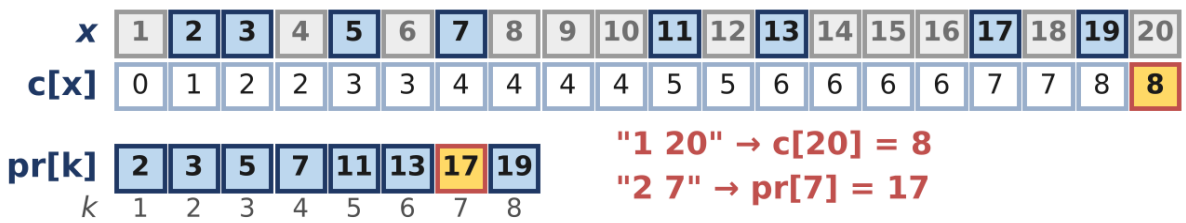
$1 \le Q \le 10^6$; $t \in \{1, 2\}$; với loại 1: $1 \le n \le 10^7$; với loại 2: $1 \le k \le 664\,579$ (tức $p_k \le 10^7$).

Subtask	Điểm	Ràng buộc thêm
1	30%	$Q \le 100$, $n \le 10^4$, $k \le 1000$.
2	30%	Chỉ có truy vấn loại 1.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

DEMNGUYENTO.INP	DEMNGUYENTO.OUT	Giải thích
6 1 10 1 100 2 1 2 25 1 1 1 2	4 25 2 97 0 1	$\pi(10) = 4$ (các số 2, 3, 5, 7); $\pi(100) = 25$. $p_1 = 2$; số nguyên tố thứ 25 là 97, số lớn nhất dưới 100. $\pi(1) = 0$ vì 1 không nguyên tố; $\pi(2) = 1$.

DEMNGUYENTO.INP	DEMNGUYENTO.OUT	Giải thích
4 1 10000000 2 664579 2 100000 1 9999990	664579 9999991 1299709 664578	Có 664 579 số nguyên tố không vượt quá 10^7, số lớn nhất là 9 999 991. $p_{100000} = 1\,299\,709$. $\pi(9\,999\,990) = 664\,578$: thiếu đúng số 9 999 991.



Hình 6.1. *Sau khi sàng tới 20: mảng c[x] = π(x) trả lời loại 1, danh sách pr[1..8] trả lời loại 2. Truy vấn "1 20" và "2 7" được trả lời bằng một lần đọc mảng.*

CHÚ Ý

Nếu quên gạch số 1, mọi π(n) đều lớn hơn đáp số đúng 1 đơn vị và danh sách bị lệch.

Danh sách đánh số từ 0 trong C++: số nguyên tố thứ k là `pr[k - 1]`.

Với Q = 10^6, đếm lại từ đầu cho mỗi truy vấn là quá chậm, kể cả khi đã sàng. Mọi truy vấn phải được trả lời trong O(1).

HƯỚNG SUY NGHĨ

- Sàng một lần tới 10^7. Trong lúc duyệt x = 1, 2, ..., 10^7 để dựng mảng c, đồng thời đẩy các số nguyên tố vào danh sách: danh sách ấy tự động tăng dần.
- Hai loại truy vấn là hai câu hỏi ngược nhau: c "đếm số nguyên tố tới n", pr "số nguyên tố ở vị trí k". Thật ra c[pr[k − 1]] = k.

BÀI 6.2 SỐ NGUYÊN TỐ SINH ĐÔI

★★☆ TRUNG BÌNH

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
SINHDOI.CPP	SINHDOI.INP	SINHDOI.OUT	2 giây · 256 MB

Kiến thức: *Sàng Eratosthenes, mảng tiền tố trên một dấu hiệu tự chọn (Tập 1, Bài 6).*

Hai số nguyên tố p và p + 2 được gọi là một **cặp sinh đôi**, chẳng hạn (3, 5), (5, 7), (11, 13), (17, 19). Người ta tin rằng có vô số cặp sinh đôi, nhưng đó vẫn là một bài toán mở.

Cho Q truy vấn, mỗi truy vấn là một đoạn [a, b]. Hãy đếm số cặp sinh đôi (p, p + 2) **nằm trọn** trong đoạn, tức là $a \le p$ và $p + 2 \le b$.

Dữ liệu vào

Dòng đầu chứa số nguyên Q. Mỗi dòng trong Q dòng tiếp theo chứa hai số nguyên a, b.

Kết quả

Gồm Q dòng, mỗi dòng là đáp số của một truy vấn.

Ràng buộc

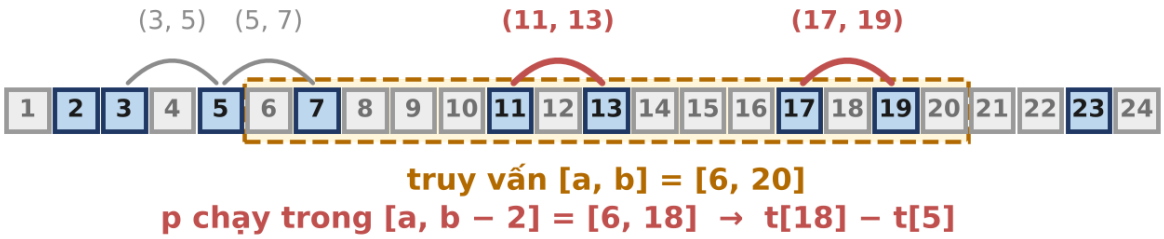
$1 \le Q \le 10^6$; $1 \le a \le b \le 10^7$.

Subtask	Điểm	Ràng buộc thêm
1	30%	$Q \le 100$, $b \le 10^4$.
2	30%	$Q \le 100$.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

SINHDOI.INP	SINHDOI.OUT	Giải thích
4 1 20 5 13 6 12 3 5	4 2 0 1	[1, 20] có 4 cặp: (3, 5), (5, 7), (11, 13), (17, 19). Số 5 thuộc hai cặp, mỗi cặp vẫn được đếm. [5, 13] có (5, 7) và (11, 13). [6, 12]: cặp (11, 13) "thò" ra ngoài vì 13 > 12, cặp (5, 7) thò ra vì 5 < 6: đáp số 0. [3, 5] có đúng cặp (3, 5).

SINHDOI.INP	SINHDOI.OUT	Giải thích
3 1 10000000 9999000 10000000 4 4	58980 4 0	Có 58 980 cặp sinh đôi với $p + 2 \le 10^7$. Đoạn [9 999 000, 10^7] có 4 cặp, cặp cuối cùng là (9 999 971, 9 999 973). Đoạn một phần tử không chứa cặp nào.



Hình 6.2. *Mỗi cặp sinh đôi được "ghi danh" tại số nhỏ p của nó. Truy vấn [6, 20] đếm các p trong [6, 18] = [a, b − 2]: có (11, 13) và (17, 19).*

CHÚ Ý

Lấy t[b] − t[a − 1] như bài DEMNT là **sai**: nó đếm cả cặp có $p \le b$ nhưng $p + 2 > b$. Ở ví dụ 1, đoạn [6, 12] sẽ cho 1 thay vì 0.

Khi $b - 2 < a$ (đoạn có ít hơn 3 số), đáp số là 0; hãy kiểm tra trước để không truy cập t[−1] hay trả về số âm.

Sàng phải đi tới $10^7 + 2$ nếu em ghi danh mọi $p \le 10^7$ (cần biết $p + 2$ có nguyên tố không); hoặc chỉ ghi danh $p \le 10^7 - 2$.

HƯỚNG SUY NGHĨ

- Đặt s[p] = 1 nếu cả p và p + 2 đều nguyên tố, ngược lại s[p] = 0. Một cặp nằm trọn trong [a, b] khi và chỉ khi $p \in [a, b - 2]$.
- Vậy đáp số là tổng s[a] + ... + s[b − 2]: dựng mảng tiền tố của s.

BÀI 6.3 SỐ CÓ ĐÚNG BA ƯỚC

★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
BAUOC.CPP	BAUOC.INP	BAUOC.OUT	1 giây · 256 MB

Kiến thức: *Sàng Eratosthenes, ước nguyên tố ≤ √n, căn bậc hai nguyên (Tập 1, Bài 5–6).*

Số 4 có ba ước 1, 2, 4; số 9 có ba ước 1, 3, 9; còn số 6 có bốn ước. Ta gọi một số nguyên dương là **số ba ước** nếu nó có **đúng ba ước dương**.

Cho N số nguyên dương $a_1, a_2, ..., a_N$. Với mỗi số a_i, hãy đếm xem trong các ước dương của a_i có bao nhiêu số ba ước.

Dữ liệu vào

Dòng đầu chứa số nguyên N. Mỗi dòng trong N dòng tiếp theo chứa một số nguyên a_i.

Kết quả

Gồm N dòng, dòng thứ i là số ước của a_i là số ba ước.

Ràng buộc

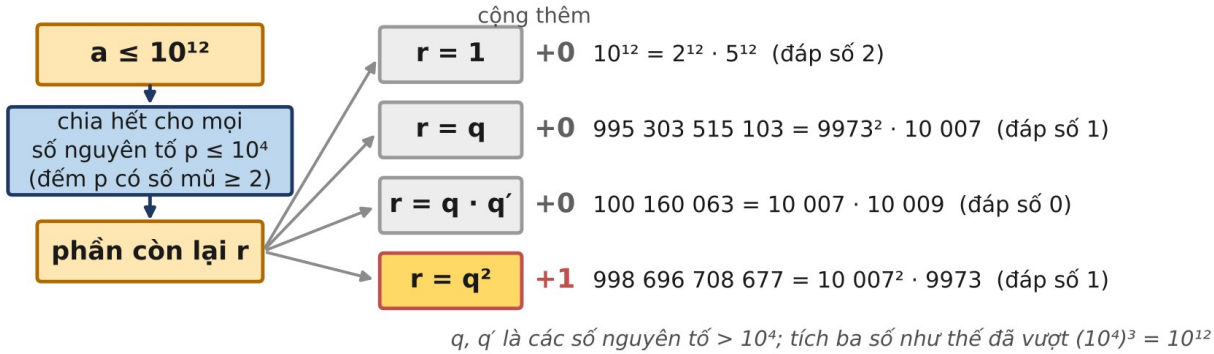
$1 \le N \le 10^5$; $1 \le a_i \le 10^{12}$.

Subtask	Điểm	Ràng buộc thêm
1	20%	$a_i \le 10^6$.
2	30%	$N \le 100$.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

BAUOC.INP	BAUOC.OUT	Giải thích
6 4 36 360 30 1 3600	1 2 2 0 0 3	Các số ba ước nhỏ nhất là 4, 9, 25, 49, 121, ... Ước của 36 có 4 và 9; ước của $360 = 2^3 \cdot 3^2 \cdot 5$ cũng chỉ có 4 và 9 (25 không chia hết 360). 30 và 1 không có ước nào như vậy. $3600 = 2^4 \cdot 3^2 \cdot 5^2$ có ba: 4, 9, 25.

BAUOC.INP	BAUOC.OUT	Giải thích
5 999966000289 1000000000000 995303515103 998696708677 100160063	1 2 1 1 0	$999\,966\,000\,289 = 999\,983^2$ chính là một số ba ước. $10^{12} = 2^{12} \cdot 5^{12}$: hai số 4 và 25. $995\,303\,515\,103 = 9973^2 \cdot 10\,007$ và $998\,696\,708\,677 = 10\,007^2 \cdot 9973$: mỗi số có đúng một. $100\,160\,063 = 10\,007 \cdot 10\,009$: không có.



Hình 6.3. *Bóc hết các thừa số nguyên tố ≤ 10^4. Phần còn lại r chỉ có ước nguyên tố > 10^4 nên có tối đa hai thừa số, và chỉ rơi vào bốn trường hợp.*

CHÚ Ý

Thử chia mỗi a_i tới $\sqrt{a_i} = 10^6$ (78 498 số nguyên tố) cho $N = 10^5$ số là gần $8 \cdot 10^9$ phép chia: quá chậm.

Tính căn bậc hai nguyên của r bằng `sqrtl` rồi chỉnh lại bằng phép nhân số nguyên (giảm khi $s \cdot s > r$, tăng khi $(s+1)^2 \le r$): `sqrt` kiểu `double` có thể sai ở chữ số cuối.

r = 1 là số chính phương nhưng không ứng với số ba ước nào: đừng đếm nó.

HƯỚNG SUY NGHĨ

- Một số có đúng ba ước khi và chỉ khi nó bằng p^2 với p nguyên tố (vì sao? dùng công thức số ước của Bài 7, hoặc ghép cặp ước d với x / d). Vậy đáp số là số các số nguyên tố p mà p^2 chia hết a_i.
- Chia a_i cho mọi số nguyên tố $p \le 10^4$ tới hết. Vì $(10^4)^3 = 10^{12}$, phần còn lại r có nhiều nhất **hai** thừa số nguyên tố, đều lớn hơn 10^4. Khi nào r cho thêm một số ba ước?

BÀI 7 Phân tích thừa số nguyên tố

ÔN NHANH KIẾN THỨC · Tập 1, Bài 7

- **Định lí cơ bản của số học:** mọi $n > 1$ viết duy nhất được thành $n = p_1^{e_1} \cdot p_2^{e_2} \cdot \ldots \cdot p_k^{e_k}$ với $p_1 < \ldots < p_k$ nguyên tố. Với $n \le 10^7$: tổng số mũ ≤ 23, số ước nguyên tố khác nhau ≤ 8.
- **Sàng ước nguyên tố nhỏ nhất** spf: lần gạch đầu tiên của p cho spf[j] = p. Phân tích bằng $n \leftarrow n / \text{spf}[n]$ trong $O(\log n)$. Thử chia tới $\sqrt{n}$: $O(\sqrt{n})$, phần còn lại > 1 là số nguyên tố.
- **Số ước và tổng ước:** $d(n) = (e_1 + 1) \cdot \ldots \cdot (e_k + 1)$; $\sigma(n) = \Pi\,(1 + p_i + \ldots + p_i^{e_i})$. ƯCLN lấy số mũ nhỏ nhất, BCNN lấy số mũ lớn nhất của từng số nguyên tố.
- **Công thức Legendre:** số mũ của p trong n! là $\lfloor n / p \rfloor + \lfloor n / p^2 \rfloor + \ldots$; tính bằng vòng `while (n) { n /= p; s += n; }` để không tràn số.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
7.1	**SỐ ƯỚC**	★☆☆ CƠ BẢN	Dùng sàng spf cho rất nhiều truy vấn; tính đồng thời số ước và tổng ước.
7.2	**CHỮ SỐ 0 CỦA GIAI THỪA**	★★☆ TRUNG BÌNH	Đưa bài toán "chữ số 0 tận cùng" trong cơ số bất kì về số mũ của từng ước nguyên tố.
7.3	**BCNN CỦA DÃY**	★★★ NÂNG CAO	Làm việc với BCNN khổng lồ qua bảng số mũ; đếm phần tử "không thể thay thế".

BÀI 7.1 SỐ ƯỚC ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
SOUOC.CPP	**SOUOC.INP**	**SOUOC.OUT**	**2 giây · 256 MB**

Kiến thức: *Sàng spf, phân tích thừa số trong $O(\log n)$, công thức $d(n)$ và $\sigma(n)$ (Tập 1, Bài 7).*

Với mỗi số nguyên dương n, kí hiệu d(n) là số ước dương của n và σ(n) là tổng các ước dương của n. Chẳng hạn 12 có các ước 1, 2, 3, 4, 6, 12 nên $d(12) = 6$ và $\sigma(12) = 28$.

Cho Q số nguyên dương, hãy tính d(n) và σ(n) cho từng số.

Dữ liệu vào

Dòng đầu chứa số nguyên Q. Mỗi dòng trong Q dòng tiếp theo chứa một số nguyên n.

Kết quả

Gồm Q dòng, dòng thứ i ghi hai số d(n) và σ(n) của số thứ i, cách nhau một dấu cách.

Ràng buộc

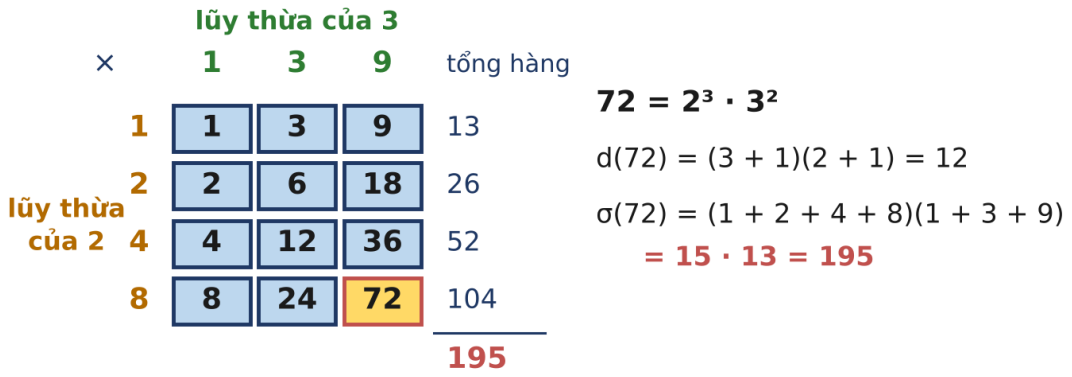
$1 \le Q \le 10^6$; $1 \le n \le 10^7$.

Subtask	Điểm	Ràng buộc thêm
1	30%	$Q \le 1000$, $n \le 10^4$.
2	30%	$Q \le 1000$.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

SOUOC.INP	SOUOC.OUT	Giải thích
4 1 12 360 97	1 1 6 28 24 1170 2 98	n = 1 chỉ có một ước là chính nó. $360 = 2^3 \cdot 3^2 \cdot 5$: d = 4 · 3 · 2 = 24, σ = 15 · 13 · 6 = 1170. 97 nguyên tố: hai ước 1 và 97.

SOUOC.INP	SOUOC.OUT	Giải thích
3 10000000 8648640 9999991	64 24902280 448 40965120 2 9999992	$10^7 = 2^7 \cdot 5^7$: d = 8 · 8 = 64, σ = 255 · 97 656 = 24 902 280. 8 648 640 = $2^6 \cdot 3^3 \cdot 5 \cdot 7 \cdot 11 \cdot 13$ có 7 · 4 · 2 · 2 · 2 · 2 = 448 ước. 9 999 991 là số nguyên tố lớn nhất không vượt quá 10^7.



Hình 7.1. *Các ước của $72 = 2^3 \cdot 3^2$ xếp thành bảng 4 × 3: hàng là lũy thừa của 2, cột là lũy thừa của 3. Có 4 · 3 = 12 ước; tổng mỗi hàng là tích của tổng hai "trục".*

CHÚ Ý

Với $n \le 10^7$, $\sigma(n)$ lớn nhất là 45 732 192 (tại $n = 9\,979\,200$) nên vẫn vừa kiểu `int`; nhưng cứ khai báo `long long` cho các tích trung gian để khỏi phải lo.

$n = 1$ không có thừa số nguyên tố nào: vòng phân tích không chạy lần nào, $d = \sigma = 1$, đúng với quy ước tích rỗng.

Mảng spf kiểu `int` tới 10^7 chiếm 40 MB; khai báo toàn cục, đừng khai báo trong hàm `main` (dễ tràn bộ nhớ ngăn xếp).

HƯỚNG SUY NGHĨ

- Sàng spf một lần. Với mỗi n, gom các lần chia cho cùng một $p = \text{spf}[n]$ để biết số mũ e.
- Trong lúc chia, duy trì lũy thừa p^j và cộng dồn $1 + p + \ldots + p^e$: không cần hàm `pow` hay công thức chia $(p^{e+1} - 1) / (p - 1)$.

BÀI 7.2	CHỮ SỐ 0 CỦA GIAI THỪA	★★☆ TRUNG BÌNH

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
CHUSO0.CPP	CHUSO0.INP	CHUSO0.OUT	1 giây · 256 MB

Kiến thức: *Phân tích thừa số bằng thử chia, công thức Legendre (Tập 1, Bài 7, Mở rộng 3).*

Ai cũng biết 10! = 3 628 800 có hai chữ số 0 tận cùng. Nhưng nếu viết 10! trong hệ cơ số 12 thì sao? Trong hệ cơ số b, số chữ số 0 tận cùng của x là số k lớn nhất mà b^k chia hết x.

Cho T cặp số (n, b), hãy cho biết n! viết trong hệ cơ số b có bao nhiêu chữ số 0 tận cùng. Quy ước 0! = 1.

Dữ liệu vào

Dòng đầu chứa số nguyên T. Mỗi dòng trong T dòng tiếp theo chứa hai số nguyên n và b.

Kết quả

Gồm T dòng, mỗi dòng một số nguyên là số chữ số 0 tận cùng của n! trong hệ cơ số b.

Ràng buộc

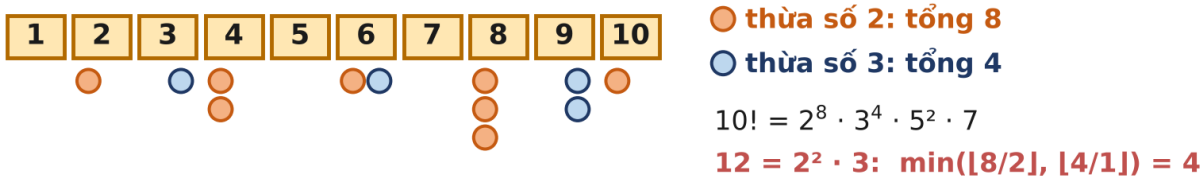
$1 \le T \le 100$; $0 \le n \le 10^{18}$; $2 \le b \le 10^{12}$.

Subtask	Điểm	Ràng buộc thêm
1	20%	$n \le 20$, $b \le 100$.
2	30%	$b = 10$.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

CHUSO0.INP	CHUSO0.OUT	Giải thích
5 10 10 10 12 5 2 0 7 100 10	2 4 3 0 24	10! có 2 chữ số 0 trong hệ thập phân. $10! = 2^8 \cdot 3^4 \cdot 5^2 \cdot 7$ và $12 = 2^2 \cdot 3$: 2^8 "đủ" cho 4 lần 12, 3^4 cũng đủ cho 4 lần: đáp số 4 (Hình 7.2). 5! = 120 = 1111000 trong hệ nhị phân. 0! = 1 không có chữ số 0 nào. 100! có 24 chữ số 0.

CHUSO0.INP	CHUSO0.OUT	Giải thích
3 1000000000000000000 2 1000000000000000000 999999999989 2026 360	999999999999999976 1000000 505	Số mũ của 2 trong n! bằng n trừ số chữ số 1 trong biểu diễn nhị phân của n; 10^{18} có 24 chữ số 1. b = 999 999 999 989 nguyên tố: $\lfloor n / b \rfloor = 10^6$, còn $b^2 > n$. $360 = 2^3 \cdot 3^2 \cdot 5$: 2026! chứa 2^{2018}, 3^{1010}, 5^{505}; đáp số min(672, 505, 505) = 505.



Hình 7.2. *Đếm số thừa số 2 và 3 trong 10! (mỗi chấm là một thừa số). $12 = 2^2 \cdot 3$ nên mỗi chữ số 0 "tiêu" hai chấm cam và một chấm xanh.*

CHÚ Ý

Không có ước nguyên tố nào của b luôn "quyết định" đáp số. Với b = 12: 8! = $2^7 \cdot 3^2 \cdot 5 \cdot 7$ cho min(⌊7/2⌋, 2) = 2, do số 3 quyết định; 9! = $2^7 \cdot 3^4 \cdot 5 \cdot 7$ cho min(3, 4) = 3, do số 2 quyết định. Phải lấy min trên **mọi** ước nguyên tố của b.

Nhớ chia số mũ $e_p(n!)$ cho số mũ k của p trong b (lấy phần nguyên) trước khi lấy min.

Tính $\lfloor n / p^j \rfloor$ bằng cách chia dần n cho p; tính lũy thừa p^j trực tiếp sẽ tràn số khi $p \approx 10^{12}$.

Đáp số có thể gần 10^{18} (ví dụ 2): dùng `long long`.

HƯỚNG SUY NGHĨ

- Phân tích $b = q_1^{k_1} \cdot \ldots \cdot q_m^{k_m}$ bằng thử chia tới $\sqrt{b} \le 10^6$. b^t chia hết n! khi và chỉ khi với mọi i, $t \cdot k_i$ không vượt quá số mũ của q_i trong n!.
- Vậy đáp số là min theo i của $\lfloor e_{q_i}(n!) / k_i \rfloor$, với $e_q(n!)$ tính bằng công thức Legendre.

BÀI 7.3	BCNN CỦA DÃY	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
BCNNDAY.CPP	BCNNDAY.INP	BCNNDAY.OUT	1 giây · 256 MB

Kiến thức: *Sàng spf, BCNN theo số mũ lớn nhất, đồng dư (Tập 1, Bài 7, Mở rộng 2; Bài 1).*

Cho dãy n số nguyên dương $a_1, a_2, ..., a_n$ và gọi L là bội chung nhỏ nhất của cả dãy. Số L có thể rất lớn: chẳng hạn BCNN của các số từ 1 tới 20 đã là 232 792 560.

Hãy tính L mod $(10^9 + 7)$. Ngoài ra, phần tử a_i gọi là **cần thiết** nếu khi xóa riêng a_i khỏi dãy thì BCNN của các số còn lại **nhỏ hơn** L (quy ước BCNN của dãy rỗng bằng 1). Hãy đếm số phần tử cần thiết.

Dữ liệu vào

Dòng đầu chứa số nguyên n. Dòng thứ hai chứa n số nguyên $a_1, a_2, ..., a_n$.

Kết quả

Dòng thứ nhất ghi L mod $(10^9 + 7)$. Dòng thứ hai ghi số phần tử cần thiết.

Ràng buộc

$1 \le n \le 10^5$; $1 \le a_i \le 10^7$.

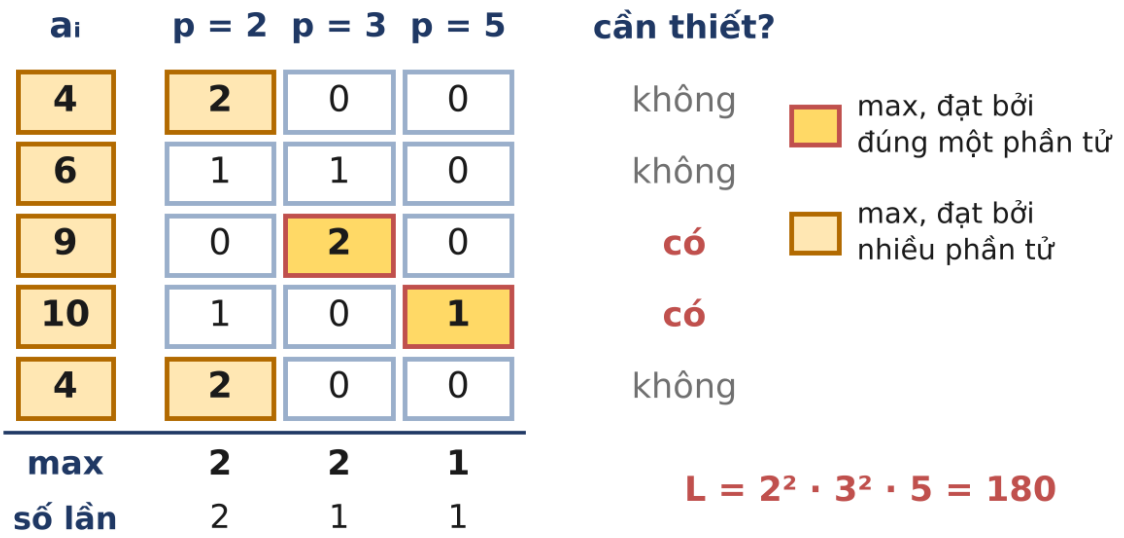
Subtask	Điểm	Ràng buộc thêm
1	20%	$n \le 20$, $a_i \le 50$.
2	30%	$n \le 2000$.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

BCNNDAY.INP	BCNNDAY.OUT	Giải thích
5 4 6 9 10 4	180 2	L = 180 = $2^2 \cdot 3^2 \cdot 5$. Xóa 9 thì còn 60; xóa 10 thì còn 36: hai phần tử cần thiết. Số 4 xuất hiện hai lần: xóa một số 4 thì số kia vẫn giữ 2^2. Xóa 6 cũng không làm L thay đổi.

BCNNDAY.INP	BCNNDAY.OUT	Giải thích
20 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20	232792560 5	BCNN(1, 2, ..., 20) = $2^4 \cdot 3^2 \cdot 5 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 19$. Cần thiết: 16 (duy nhất chứa 2^4), 11, 13, 17, 19. Số 9 không cần thiết vì 18 cũng chứa 3^2.

BCNNDAY.INP	BCNNDAY.OUT	Giải thích
4 9999991 9999973 9999991 1	639300250 1	L = 9 999 991 · 9 999 973 = 99 999 640 000 243, chia $10^9 + 7$ dư 639 300 250. Chỉ 9 999 973 cần thiết; 9 999 991 xuất hiện hai lần, còn 1 thì không bao giờ cần thiết.



*Hình 7.3. Bảng số mũ của ví dụ 1. Mỗi cột tô vàng số mũ lớn nhất; a_i cần thiết khi nó là **phần tử duy nhất** đạt số mũ lớn nhất ở ít nhất một cột.*

CHÚ Ý

Không thể tính L bằng `L = L / gcd(L, a) * a` rồi lấy dư: L thật có thể có hàng trăm nghìn chữ số, mà BCNN không "đi qua" được phép lấy dư (gcd của các số dư chẳng liên quan gì tới gcd của các số thật).

Một số mũ lớn nhất đạt bởi **hai** phần tử bằng nhau (như hai số 4 ở ví dụ 1) thì không ai trong hai phần tử ấy cần thiết.

Số 1 không có thừa số nguyên tố nào nên không bao giờ cần thiết; với n = 1 và $a_1 > 1$ thì a_1 cần thiết (xóa đi, BCNN còn 1).

HƯỚNG SUY NGHĨ

- $L = \Pi\, p^{M(p)}$ với M(p) là số mũ lớn nhất của p trong các a_i. Phân tích mọi a_i bằng sàng spf để tìm M(p), rồi nhân dồn theo mô-đun.
- Xóa a_i làm L nhỏ đi khi và chỉ khi có một p mà a_i đạt M(p) và **không phần tử nào khác** đạt M(p). Vậy cùng với M(p), hãy đếm số phần tử đạt M(p), rồi duyệt dãy lần thứ hai.

BÀI 8 Sàng trên đoạn

ÔN NHANH KIẾN THỨC · Tập 1, Bài 8

- **Chỉ cần các số nguyên tố ≤ √R:** mọi hợp số $x \in [L, R]$ có ước nguyên tố ≤ √R. Với $R \le 10^{12}$, sàng thường tới 10^6 (78 498 số nguyên tố) là đủ.
- **Gạch trong cửa sổ:** mỗi p gạch các bội trong [L, R] bắt đầu từ $\max(p^2, \lceil L / p \rceil \cdot p)$, với $\lceil L / p \rceil \cdot p$ = `(L + p - 1) / p * p`. Số x ứng với ô x − L; khi L = 1 phải gạch riêng số 1.
- **Độ phức tạp** $O(\sqrt{R} \log\log \sqrt{R} + (R - L) \log\log R)$, bộ nhớ $O(\sqrt{R} + R - L)$: với $R = 10^{12}$, $R - L = 10^6$ chỉ khoảng $2{,}9 \cdot 10^6$ phép gạch.
- Quanh N, khoảng cách trung bình giữa hai số nguyên tố liên tiếp xấp xỉ ln N (≈ 27,6 khi $N = 10^{12}$); khoảng cách lớn nhất chỉ vài trăm.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
8.1	**NGUYÊN TỐ TRÊN ĐOẠN**	★☆☆ CƠ BẢN	Sàng nhỏ một lần, dùng lại cho nhiều cửa sổ; tính tổng không tràn số.
8.2	**KHOẢNG CÁCH LỚN NHẤT**	★★☆ TRUNG BÌNH	Duyệt các số nguyên tố liên tiếp trong cửa sổ; xử lí hòa và trường hợp suy biến.
8.3	**SỐ ƯỚC TRÊN ĐOẠN**	★★★ NÂNG CAO	Phân tích cùng lúc mọi số trong cửa sổ; nhận ra phần còn lại là số nguyên tố.

BÀI 8.1 NGUYÊN TỐ TRÊN ĐOẠN ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`NTDOAN.CPP`	`NTDOAN.INP`	`NTDOAN.OUT`	1 giây · 256 MB

Kiến thức: *Sàng trên đoạn [L, R] (Tập 1, Bài 8).*

Cho T đoạn [L, R]. Với mỗi đoạn, hãy đếm số các số nguyên tố trong đoạn và tính tổng của chúng.

Dữ liệu vào

Dòng đầu chứa số nguyên T. Mỗi dòng trong T dòng tiếp theo chứa hai số nguyên L, R.

Kết quả

Gồm T dòng, dòng thứ i ghi hai số: số lượng và tổng các số nguyên tố trong đoạn thứ i (ghi 0 0 nếu đoạn không có số nguyên tố nào).

Ràng buộc

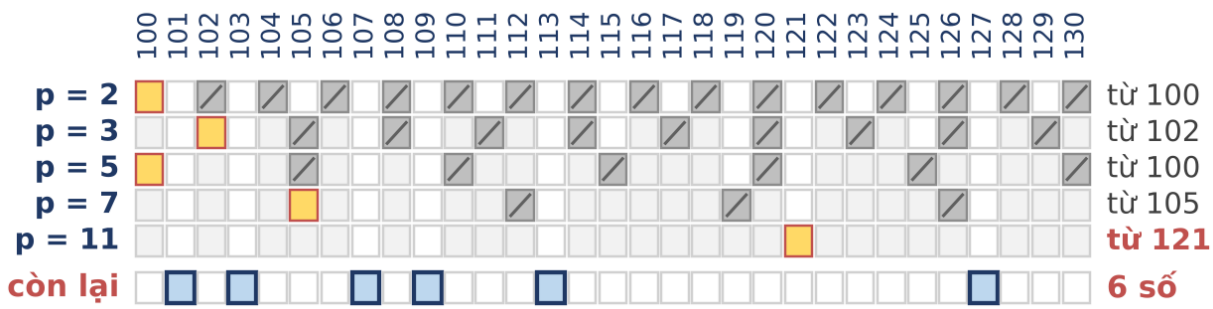
$1 \le T \le 10$; $1 \le L \le R \le 10^{12}$; $R - L \le 10^6$.

Subtask	Điểm	Ràng buộc thêm
1	30%	$R \le 10^7$.
2	30%	$R - L \le 100$.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

NTDOAN.INP	NTDOAN.OUT	Giải thích
3 1 10 100 130 24 28	4 17 6 660 0 0	[1, 10]: 2 + 3 + 5 + 7 = 17. [100, 130] có 6 số nguyên tố 101, 103, 107, 109, 113, 127, tổng 660 (Hình 8.1). [24, 28] không có số nguyên tố nào.

NTDOAN.INP	NTDOAN.OUT	Giải thích
3 999999999900 1000000000000 999966000289 999966000289 1 1	4 3999999999846 0 0 0 0	Bốn số nguyên tố 999 999 999 937, ...959, ...961, ...989; tổng gần $4 \cdot 10^{12}$. 999 966 000 289 = $999\,983^2$: chỉ "thợ gạch" p = 999 983 mới loại được nó. Đoạn [1, 1]: số 1 không nguyên tố.



Hình 8.1. *Cửa sổ [100, 130]: mỗi hàng là một lượt gạch của $p \le \sqrt{130}$; ô vàng là ô bắt đầu $\max(p^2, \lceil L / p \rceil \cdot p)$. Hàng cuối: các số còn lại.*

CHÚ Ý

Đoạn $[10^{12} - 10^6, 10^{12}]$ có 36 400 số nguyên tố với tổng 36 399 981 770 340 822 $\approx 3{,}6 \cdot 10^{16}$: phải dùng `long long`.

Sàng tới 10^6 **một lần** trước khi đọc các truy vấn; mỗi truy vấn chỉ sàng cửa sổ của nó.

Khi $L = 1$, nhớ gạch số 1; khi $L \le p \le R$, đừng gạch chính p: bắt đầu từ $\max(p^2, \lceil L / p \rceil \cdot p)$.

HƯỚNG SUY NGHĨ

- Với mỗi số nguyên tố $p \le 10^6$ mà $p \cdot p \le R$, gạch các bội của p nằm trong [L, R]. Các ô còn lại (trừ số 1) là số nguyên tố.
- Mảng đánh dấu chỉ cần $R - L + 1$ phần tử; số x nằm ở ô $x - L$.

BÀI 8.2	KHOẢNG CÁCH LỚN NHẤT	★★☆ TRUNG BÌNH

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
KHOANGCACH.CPP	KHOANGCACH.INP	KHOANGCACH.OUT	1 giây · 256 MB

Kiến thức: *Sàng trên đoạn, khoảng cách giữa các số nguyên tố liên tiếp (Tập 1, Bài 8, Mở rộng 1).*

Hai số nguyên tố $p < q$ gọi là **liên tiếp** nếu không có số nguyên tố nào nằm giữa chúng; khi đó $q - p$ là khoảng cách giữa chúng.

Cho đoạn $[L, R]$. Xét mọi cặp số nguyên tố liên tiếp $p < q$ mà **cả hai** cùng thuộc $[L, R]$. Hãy tìm cặp có khoảng cách **lớn nhất** và cặp có khoảng cách **nhỏ nhất**. Nếu có nhiều cặp cùng khoảng cách, chọn cặp có p nhỏ nhất.

Dữ liệu vào

Một dòng chứa hai số nguyên L, R.

Kết quả

Nếu đoạn chứa ít hơn hai số nguyên tố, ghi -1. Ngược lại, dòng thứ nhất ghi cặp p q có khoảng cách lớn nhất, dòng thứ hai ghi cặp p q có khoảng cách nhỏ nhất.

Ràng buộc

$1 \le L \le R \le 10^{12}$; $R - L \le 10^{7}$.

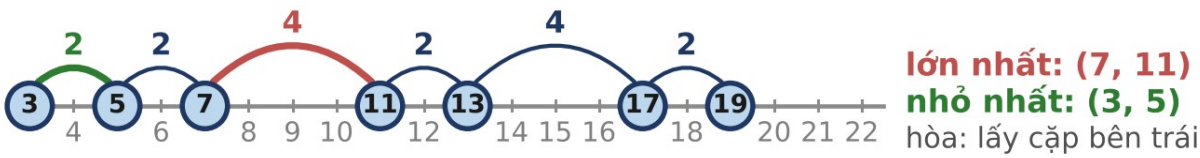
Subtask	Điểm	Ràng buộc thêm
1	30%	$R \le 10^{6}$.
2	30%	$R - L \le 10^{4}$.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

KHOANGCACH.INP	KHOANGCACH.OUT	Giải thích
3 22	7 11 3 5	Các số nguyên tố trong [3, 22] là 3, 5, 7, 11, 13, 17, 19, khoảng cách 2, 2, 4, 2, 4, 2. Khoảng cách 4 xuất hiện hai lần, chọn cặp (7, 11); khoảng cách 2 xuất hiện bốn lần, chọn (3, 5).

KHOANGCACH.INP	KHOANGCACH.OUT	Giải thích
24 28	-1	[24, 28] không có số nguyên tố nào.

KHOANGCACH.INP	KHOANGCACH.OUT	Giải thích
999999548000 999999550000	999999548953 999999549259 999999549551 999999549553	Đoạn dài 2001 số chứa 60 số nguyên tố; khoảng cách lớn nhất là 306. Cặp sinh đôi đầu tiên trong đoạn cho khoảng cách 2.



Hình 8.2. *Ví dụ 1 trên trục số: các cung nối hai số nguyên tố liên tiếp. Cung đỏ là khoảng cách lớn nhất, cung xanh lá là khoảng cách nhỏ nhất (đều chọn cặp đầu tiên).*

CHÚ Ý

Các cặp "liên tiếp" tính trong toàn bộ tập số nguyên tố, nhưng vì p, q đều thuộc [L, R] nên chỉ cần xét các số nguyên tố liên tiếp **sau khi sàng đoạn**. Không tính khoảng cách từ L tới số nguyên tố đầu tiên.

Để chọn cặp có p nhỏ nhất khi hòa, chỉ cập nhật khi khoảng cách **lớn hơn hẳn** (hoặc nhỏ hơn hẳn) kỉ lục hiện tại.

Đoạn [1, 3] có cặp (2, 3) với khoảng cách 1, cặp duy nhất có khoảng cách lẻ.

R − L tới 10^7: mảng đánh dấu kiểu `char` chiếm 10 MB, đừng dùng `long long` hay `int` cho nó.

HƯỚNG SUY NGHĨ

- Sàng đoạn [L, R] như Bài 8, rồi duyệt x từ L tới R, nhớ số nguyên tố gần nhất vừa gặp. Mỗi khi gặp một số nguyên tố mới, ta có một cặp liên tiếp.
- Với một cửa sổ R − L = 10^7, số phép gạch chỉ cỡ (R − L) · ln ln R: vẫn rất nhanh.

BÀI 8.3	SỐ ƯỚC TRÊN ĐOẠN	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
UOCDOAN.CPP	UOCDOAN.INP	UOCDOAN.OUT	1 giây · 256 MB

Kiến thức: *Sàng trên đoạn, phân tích thừa số đồng loạt, công thức số ước (Tập 1, Bài 7–8).*

Kí hiệu d(x) là số ước dương của x. Cho đoạn [L, R], hãy tính tổng d(L) + d(L + 1) + ... + d(R), đồng thời tìm số x ∈ [L, R] có nhiều ước nhất (nếu có nhiều số như vậy, chọn số nhỏ nhất).

Dữ liệu vào

Một dòng chứa hai số nguyên L, R.

Kết quả

Dòng thứ nhất ghi tổng số ước. Dòng thứ hai ghi số x tìm được và d(x), cách nhau một dấu cách.

Ràng buộc

$1 \le L \le R \le 10^{12}$; $R - L \le 10^6$.

Subtask	Điểm	Ràng buộc thêm
1	30%	$R \le 10^6$.
2	30%	$R - L \le 100$.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

UOCDOAN.INP	UOCDOAN.OUT	Giải thích
1 10	27 6 4	d(1..10) = 1, 2, 2, 3, 2, 4, 2, 4, 3, 4, tổng 27. Ba số 6, 8, 10 cùng có 4 ước: chọn 6.

UOCDOAN.INP	UOCDOAN.OUT	Giải thích
999999999990 1000000000000	733 999999999999 256	999 999 999 999 = $3^3 \cdot 7 \cdot 11 \cdot 13 \cdot 37 \cdot 101 \cdot 9901$ có $4 \cdot 2^6 = 256$ ước, nhiều hơn $10^{12} = 2^{12} \cdot 5^{12}$ (169 ước). 999 999 999 991 = 757 · 1321 · 1 000 003: thừa số cuối lớn hơn 10^6.

UOCDOAN.INP	UOCDOAN.OUT	Giải thích
963761198400 963761198400	6720 963761198400 6720	963 761 198 400 = $2^6 \cdot 3^4 \cdot 5^2 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23$ có $7 \cdot 5 \cdot 3 \cdot 2^6 = 6720$ ước.

x	thừa số p ≤ √R	phần còn lại	d(x)
999 999 999 990	$2 \cdot 3^2 \cdot 5 \cdot 21649 \cdot 513239$	1	$3 \cdot 2^4 = 48$
999 999 999 991	$757 \cdot 1321$	**1 000 003**	$\mathbf{2^2 \cdot 2 = 8}$
999 999 999 992	$2^3 \cdot 7 \cdot 79 \cdot 103 \cdot 439 \cdot 4999$	1	$4 \cdot 2^5 = 128$
999 999 999 993	$3 \cdot 19 \cdot 83$	**211 371 803**	$\mathbf{2^3 \cdot 2 = 16}$
999 999 999 994	$2 \cdot 49369$	**10 127 813**	$\mathbf{2^2 \cdot 2 = 8}$
999 999 999 995	$5 \cdot 251 \cdot 1831 \cdot 435179$	1	$2^4 = 16$

Hình 8.3. *Phân tích đồng loạt [999 999 999 990, 999 999 999 995]: mỗi p ≤ √R chỉ "ghé" các bội của nó, chia phần còn lại và nhân d với (e + 1). Ô vàng: phần còn lại > 1 là một số nguyên tố, nhân thêm 2 vào d(x).*

CHÚ Ý

Số $x \le 10^{12}$ có thể có một thừa số nguyên tố lớn hơn 10^6 (như 1 000 003 ở ví dụ 2). Sau khi chia hết cho mọi p ≤ √R, phần còn lại nếu lớn hơn 1 thì **là một số nguyên tố** (vì nó không có ước nguyên tố ≤ √R), góp thêm thừa số 2 vào d(x).

Ở bài này phải duyệt **mọi** bội của p trong đoạn, kể cả p và các bội nhỏ hơn p^2: bắt đầu từ $\lceil L / p \rceil \cdot p$, không lấy max với p^2 như khi sàng.

Mảng phần còn lại có R − L + 1 số `long long` (8 MB). Với $[10^{12} - 10^6, 10^{12}]$, tổng số ước là 28 788 789, vẫn vừa `int`; nhưng hãy cứ cộng dồn bằng `long long` cho yên tâm.

HƯỚNG SUY NGHĨ

- Thử chia từng số tới √x thì quá chậm. Hãy đảo vòng lặp như sàng: với mỗi số nguyên tố p ≤ √R, duyệt các bội x của p trong đoạn, chia con[x − L] cho p tới hết và nhân d[x − L] với (e + 1).
- Mỗi x chỉ bị ghé bởi các ước nguyên tố ≤ √R của nó; tổng số lần ghé xấp xỉ $(R - L) \cdot \Sigma\, 1/p$, chỉ vài triệu.

CHƯƠNG 3. SỐ HỌC ĐỒNG DƯ

Mười hai bài tập của chương này đều bắt đầu bằng cùng một câu: "in kết quả theo mô-đun...". Em sẽ tính lũy thừa với số mũ khổng lồ, chia theo mô-đun, rút gọn số mũ bằng hàm φ và ghép các số dư bằng định lí số dư Trung Hoa. Ý tưởng mỗi bài thường gói trong một công thức, nhưng điều phân biệt lời giải đúng với lời giải "gần đúng" là các chi tiết: mô-đun có nguyên tố không, có ước chung không, tích hai số dư có tràn không, 0^0 bằng bao nhiêu, nghiệm nhỏ nhất có bị âm không.

TRONG CHƯƠNG NÀY

Bài 9. Lũy thừa nhanh

- 9.1 LŨY THỪA — ★☆☆ CƠ BẢN
- 9.2 SỐ MŨ KHỔNG LỒ — ★★☆ TRUNG BÌNH
- 9.3 TỔNG LŨY THỪA — ★★★ NÂNG CAO

Bài 10. Nghịch đảo mô-đun

- 10.1 XÁC SUẤT — ★☆☆ CƠ BẢN
- 10.2 NGHỊCH ĐẢO DÃY — ★★☆ TRUNG BÌNH
- 10.3 PHƯƠNG TRÌNH ĐỒNG DƯ — ★★★ NÂNG CAO

Bài 11. Hàm phi Euler và định lí Euler

- 11.1 PHI — ★☆☆ CƠ BẢN
- 11.2 PHÂN SỐ TỐI GIẢN — ★★☆ TRUNG BÌNH
- 11.3 THÁP LŨY THỪA — ★★★ NÂNG CAO

Bài 12. Định lí số dư Trung Hoa

- 12.1 ĐIỂM BINH — ★☆☆ CƠ BẢN
- 12.2 HAI NGỌN ĐÈN — ★★☆ TRUNG BÌNH
- 12.3 LỊCH CAN CHI — ★★★ NÂNG CAO

LỜI KHUYÊN CHO CHƯƠNG NÀY

- Trước khi dùng định lí Fermat hay định lí Euler, hãy hỏi: mô-đun có nguyên tố không, cơ số và mô-đun có nguyên tố cùng nhau không? Nếu không, tìm cách khác (Horner trên số mũ, Euclid mở rộng, Euler mở rộng).
- Viết sẵn một bộ hàm nhỏ và dùng lại: `power`, `extgcd`, `inverse`, gộp hai phương trình đồng dư. Mỗi hàm tự lấy dư đầu vào và khởi tạo bằng 1 % m.
- Với mô-đun tới 10^{18}, mọi phép nhân hai số dư phải qua `__int128`. Tự tạo test có m = 1, a = 0, b = 0 và các mô-đun có ước chung.

BÀI 9 Lũy thừa nhanh

ÔN NHANH KIẾN THỨC · Tập 1, Bài 9

- **Bình phương liên tiếp:** $a^b = (a^{\lfloor b/2 \rfloor})^2$, nhân thêm a nếu b lẻ. Vòng lặp đi qua các bit của b từ phải sang trái: bit 1 thì $r \leftarrow r \cdot a$, luôn $a \leftarrow a \cdot a$. Chỉ $\lfloor \log_2 b \rfloor + 1$ lượt: 60 lượt với $b \le 10^{18}$.
- **Lấy dư sau mỗi phép nhân** và khởi tạo $r = 1 \bmod m$ (đúng cả khi $m = 1$). Quy ước $a^0 = 1$, kể cả $0^0 = 1$.
- **Chống tràn:** $m \le 3 \cdot 10^9$ thì nhân trong `long long`; m tới 10^{18} thì ép một thừa số sang `__int128` (hoặc nhân Ấn Độ).
- **Mở rộng:** số mũ là xâu dài ($r \leftarrow r^{10} \cdot a^d$ với mỗi chữ số d); tổng cấp số nhân bằng chia đôi $S(2k) = S(k) \cdot (1 + a^k)$, $S(n) = 1 + a \cdot S(n-1)$.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
9.1	**LŨY THỪA**	★☆☆ CƠ BẢN	Cài đặt lũy thừa nhanh chuẩn; lấy dư cơ số trước khi nhân.
9.2	**SỐ MŨ KHỔNG LỒ**	★★☆ TRUNG BÌNH	Áp dụng sơ đồ Horner cho số mũ; không lạm dụng định lí Fermat khi mô-đun là hợp số.
9.3	**TỔNG LŨY THỪA**	★★★ NÂNG CAO	Chia để trị trên số mũ khi không chia được theo mô-đun; chống tràn số $10^{18} \cdot 10^{18}$.

BÀI 9.1 LŨY THỪA ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`LUYTHUAQ.CPP`	`LUYTHUAQ.INP`	`LUYTHUAQ.OUT`	1 giây · 256 MB

Kiến thức: *Lũy thừa nhanh theo mô-đun $10^9 + 7$ (Tập 1, Bài 9).*

Trong phòng thí nghiệm, một loại tế bào cứ sau mỗi giờ lại phân chia: **mỗi** tế bào biến thành đúng a tế bào mới (a = 0 nghĩa là tế bào chết hết, a = 1 nghĩa là không đổi). Bắt đầu từ một tế bào, sau b giờ trong ống nghiệm có đúng a^b tế bào. Sau 0 giờ thì vẫn là 1 tế bào ban đầu, nên ta quy ước $a^0 = 1$ với mọi a, kể cả $0^0 = 1$.

Nhà khoa học có Q câu hỏi, mỗi câu hỏi gồm hai số a và b. Vì số tế bào có thể cực lớn, với mỗi câu hỏi em chỉ cần in **số dư của a^b khi chia cho $10^9 + 7$**.

Dữ liệu vào

Dòng đầu chứa số nguyên Q. Mỗi dòng trong Q dòng tiếp theo chứa hai số nguyên a và b.

Kết quả

Gồm Q dòng, dòng thứ i là $a^b \bmod (10^9 + 7)$ của câu hỏi thứ i.

Ràng buộc

$1 \le Q \le 10^5$; $0 \le a, b \le 10^{18}$.

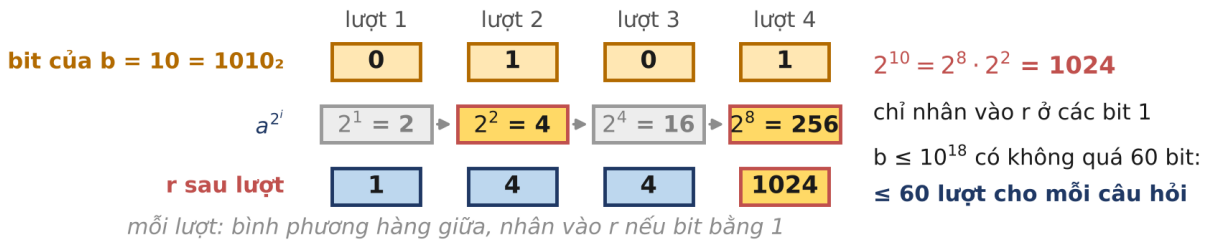
Subtask	Điểm	Ràng buộc thêm
1	30%	$Q \le 10$, $b \le 10^6$.
2	30%	$a < 10^9 + 7$.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

LUYTHUAQ.INP	LUYTHUAQ.OUT	Giải thích
3 2 10 3 1000000000000000000 1000000007 5	1024 246336683 0	$2^{10} = 1024$. 10^{18} chia $10^9 + 6$ dư 36, và 3 không chia hết cho $10^9 + 7$, nên theo định lí Fermat, 3 mũ $10^{18} \equiv 3^{36} \equiv 246\,336\,683$ (Tập 1, Bài 10). $a = 10^9 + 7$ chia hết cho mô-đun: mọi lũy thừa với số mũ dương đều dư **0**.

LUYTHUAQ.INP	LUYTHUAQ.OUT
3 0 0 1000000007 0 1000000000000000000 1000000000000000000	1 1 504853526

Giải thích ví dụ 2. $0^0 = 1$ theo quy ước. Ở câu hỏi thứ hai, $a = 10^9 + 7$ chia hết cho mô-đun nhưng số mũ bằng 0, nên đáp số vẫn là **1**. Ở câu hỏi cuối, cơ số $10^{18} \equiv 49 \pmod{10^9 + 7}$: phải lấy dư **trước** khi bình phương, nếu không $10^{18} \cdot 10^{18}$ tràn ngay. Đáp số là $49^{36} \bmod (10^9 + 7) = 504\,853\,526$.



Hình 9.1. Tính 2^{10} bằng bình phương liên tiếp: $10 = 1010_2$, chỉ nhân vào r các lũy thừa ứng với bit 1 (ô vàng).

CHÚ Ý

Cơ số a tới 10^{18}: nếu không lấy `a %= MOD` trước, ngay phép bình phương đầu tiên a · a đã tràn `long long`.

Sau khi lấy dư, mọi số đều nhỏ hơn $10^9 + 7$ nên tích hai số nhỏ hơn $1{,}01 \cdot 10^{18} < 9{,}2 \cdot 10^{18}$: nhân trong `long long` là đủ, không cần `__int128`.

Ví dụ 2 là ba cái bẫy: $0^0 = 1$; a chia hết cho $10^9 + 7$ nhưng b = 0 vẫn cho 1; và a lớn hơn mô-đun.

Với Q = 10^5 dòng vào ra, nên dùng `scanf`/`printf` (hoặc tắt đồng bộ của `cin`/`cout`).

HƯỚNG SUY NGHĨ

- Cách nhân b lần chỉ đủ cho subtask 1. Hãy viết b ở hệ nhị phân: a^b là tích các lũy thừa a^1, a^2, a^4, a^8, ... ứng với những bit 1 của b.
- Viết một hàm `power(a, b)` dùng chung cho mọi câu hỏi; hàm này sẽ được dùng lại ở gần như mọi bài của chương.

BÀI 9.2	SỐ MŨ KHỔNG LỒ	★★☆ TRUNG BÌNH

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
SOMULON.CPP	SOMULON.INP	SOMULON.OUT	1 giây · 256 MB

Kiến thức: *Lũy thừa với số mũ là xâu, sơ đồ Horner trên số mũ (Tập 1, Bài 9, Mở rộng 3; Bài 1).*

Thầy giáo viết lên bảng ba số a, b, m, trong đó số mũ b dài tới **một trăm nghìn chữ số**, và yêu cầu tính $a^b \bmod m$. Mô-đun m là một số nguyên dương **bất kì**, không nhất thiết là số nguyên tố, và a, m có thể có ước chung. Quy ước $a^0 = 1$ với mọi a.

Bạn Hùng đề xuất: "Lấy b chia cho m − 1 rồi dùng số dư làm số mũ". Em hãy kiểm tra xem cách ấy có đúng không, và viết chương trình tính đúng $a^b \bmod m$ cho T bộ dữ liệu.

Dữ liệu vào

Dòng đầu chứa số nguyên T. Mỗi dòng trong T dòng tiếp theo chứa ba số tự nhiên a, b, m viết trong hệ thập phân (b không có chữ số 0 thừa ở đầu).

Kết quả

Gồm T dòng, dòng thứ i là $a^b \bmod m$ của bộ dữ liệu thứ i.

Ràng buộc

$1 \le T \le 10$; $0 \le a \le 10^{18}$; b có không quá 10^5 chữ số; $1 \le m \le 10^9$.

Subtask	Điểm	Ràng buộc thêm
1	20%	$b \le 10^{18}$.
2	30%	m là số nguyên tố và a không chia hết cho m.
3	50%	Không có ràng buộc gì thêm.

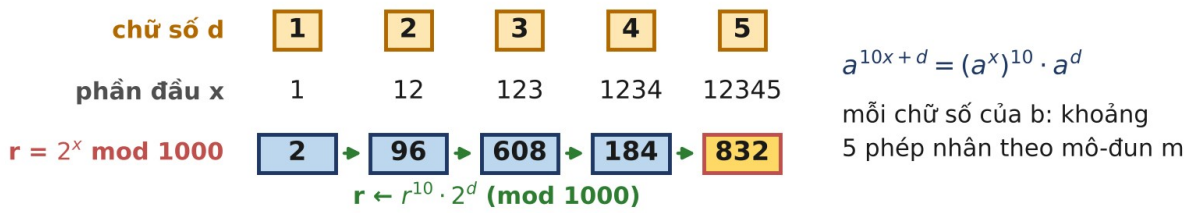
Ví dụ

SOMULON.INP	SOMULON.OUT	Giải thích
3 2 12345 1000 3 100 7 7 0 1	832 4 0	$2^{12345} \bmod 1000 = 832$, xem Hình 9.2. $3^{100} \bmod 7$: theo Fermat $3^6 \equiv 1$, $100 = 6 \cdot 16 + 4$ nên đáp số là $3^4 = 81 \equiv$ **4**. Mọi số chia cho 1 đều dư 0.

SOMULON.INP	SOMULON.OUT
2 2 100 12 10 100000000000000000000000000000000 1000000007	4 540840989

Giải thích ví dụ 2. Bộ thứ nhất: $2^{100} \bmod 12 =$ **4**. Cách của Hùng cho $100 \bmod 11 = 1$ và $2^1 = 2$: sai. Lấy theo $\varphi(12) = 4$ cũng sai: $100 \bmod 4 = 0$ cho $2^0 = 1$, vì 2 và 12 không nguyên tố cùng nhau.

Bộ thứ hai: b = 10^{30} có 31 chữ số. Mô-đun $10^9 + 7$ là số nguyên tố, nên được rút gọn số mũ theo $10^9 + 6$: $10^{30} \bmod (10^9 + 6) = 999\,784\,006$, và $10^{999784006} \bmod (10^9 + 7) = 540\,840\,989$.



Hình 9.2. *Sơ đồ Horner trên số mũ: đọc từng chữ số d của b, cập nhật $r \leftarrow r^{10} \cdot a^d \pmod m$.*

CHÚ Ý

Định lí Fermat (rút gọn số mũ theo m − 1) chỉ đúng khi m là số nguyên tố **và** m không chia hết a. Ví dụ 2 cho thấy với m = 12 thì cả cách lấy theo m − 1 lẫn theo φ(m) đều sai.

b có tới 10^5 chữ số: đọc vào **xâu**, không đọc vào `long long`.

Khởi tạo r = 1 % m (chứ không phải 1), nếu không bộ "7 0 1" sẽ in 1.

Với m ≤ 10^9, tích hai số dư nhỏ hơn 10^{18} nên vừa `long long`; chỉ cần lấy `a %= m` trước khi nhân (a tới 10^{18}).

HƯỚNG SUY NGHĨ

- Nếu x là phần đầu đã đọc của b và d là chữ số tiếp theo thì phần đầu mới là 10x + d. Khi đó $a^{10x+d} = (a^x)^{10} \cdot a^d$.
- Tính sẵn $a^0, a^1, \ldots, a^9$ mod m. Mỗi chữ số chỉ tốn một lần lũy thừa bậc 10 (4 phép nhân) và một phép nhân: tổng cộng chừng $5 \cdot 10^5$ phép nhân cho mỗi bộ.

BÀI 9.3	TỔNG LŨY THỪA	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
TONGLT.CPP	**TONGLT.INP**	**TONGLT.OUT**	**1 giây · 256 MB**

Kiến thức: *Tổng cấp số nhân bằng chia đôi, nhân* `__int128` *(Tập 1, Bài 9, Mở rộng 2).*

Cho ba số nguyên a, n, m. Hãy tính

$$S = 1 + a + a^2 + ... + a^n \pmod m$$

Tổng S có đúng n + 1 số hạng $a^0, a^1, ..., a^n$, trong đó quy ước $a^0 = 1$ (kể cả khi a = 0). Mô-đun m là số nguyên dương **bất kì**, có thể rất lớn và không nhất thiết là số nguyên tố.

Dữ liệu vào

Dòng đầu chứa số nguyên T là số bộ dữ liệu. Mỗi dòng trong T dòng tiếp theo chứa ba số nguyên a, n, m.

Kết quả

Gồm T dòng, mỗi dòng là giá trị S mod m của một bộ dữ liệu.

Ràng buộc

$1 \le T \le 10^4$; $0 \le a \le 10^{18}$; $0 \le n \le 10^{18}$; $1 \le m \le 10^{18}$.

Subtask	Điểm	Ràng buộc thêm
1	20%	$T \le 10$, $n \le 10^6$, $m \le 10^9$.
2	30%	$m \le 10^9$, m là số nguyên tố và a mod m ≠ 0, a mod m ≠ 1.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

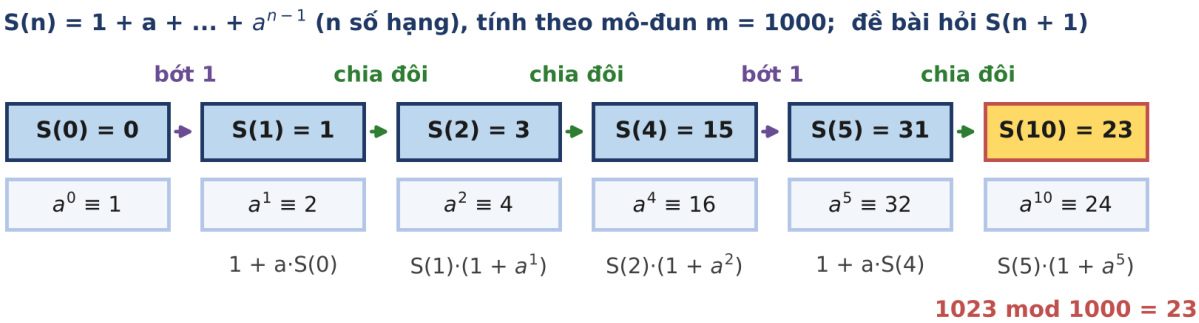
TONGLT.INP	TONGLT.OUT
3 2 9 1000 3 4 100 1 1000000000000000000 1000000000000000000	23 21 1

Giải thích ví dụ 1. $1 + 2 + ... + 2^9 = 1023 \equiv$ **23** (mod 1000); $1 + 3 + 9 + 27 + 81 = 121 \equiv$ **21** (mod 100); với a = 1 thì $S = n + 1 = 10^{18} + 1 \equiv$ **1** (mod 10^{18}).

TONGLT.INP	TONGLT.OUT
3 0 5 7 5 3 4 999999999999999999 1000000000000000000 1000000000000000000	1 0 1

Giải thích ví dụ 2. Bộ thứ nhất: $S = 0^0 + 0^1 + \ldots + 0^5 = 1$. Bộ thứ hai: $1 + 5 + 25 + 125 = 156 = 4 \cdot 39$, nên $S \equiv \mathbf{0} \pmod 4$; ở đây $a \equiv 1 \pmod 4$ nên công thức $(a^{n+1} - 1) / (a - 1)$ đòi chia cho một số chia hết cho mô-đun.

Bộ thứ ba: $a = 10^{18} - 1 \equiv -1 \pmod{10^{18}}$, nên các số hạng lần lượt là 1, −1, 1, −1, ..., và vì có $10^{18} + 1$ số hạng (một số lẻ) nên tổng bằng **1**. Mọi tích trung gian ở đây cỡ $10^{18} \cdot 10^{18} = 10^{36}$.



Hình 9.3. *Tính $1 + 2 + \ldots + 2^9 \bmod 1000 = S(10)$: số lẻ thì bớt 1, số chẵn thì chia đôi, đồng thời giữ a^n để dùng cho bước chia đôi.*

CHÚ Ý

Công thức $(a^{n+1} - 1) / (a - 1)$ cần **chia** cho $a - 1$ theo mô-đun m, chỉ làm được khi $\gcd(a - 1, m) = 1$. Với m bất kì (và nhất là khi $a \equiv 1$) nó không dùng được.

m tới 10^{18} nên mọi phép nhân hai số dư phải ép một thừa số sang `__int128`: `(__int128)x * y % m`.

Đề hỏi tổng **n + 1** số hạng (từ a^0 tới a^n). Nhầm thành n số hạng là sai cả ba ví dụ.

Cẩn thận `1 + P` có thể bằng m (khi P = m − 1): hãy lấy dư `(1 + P) % m` trước khi nhân.

HƯỚNG SUY NGHĨ

- Đặt $S(k) = 1 + a + \ldots + a^{k-1}$ (k số hạng). Nửa sau của S(2k) chính là nửa đầu nhân với a^k, nên $S(2k) = S(k) \cdot (1 + a^k)$.
- Khi k lẻ: $S(k) = 1 + a \cdot S(k - 1)$. Viết một hàm trả về đồng thời S(k) và a^k; mỗi lần gọi số số hạng giảm ít nhất một nửa sau hai bước, nên chỉ có $O(\log n)$ lời gọi.
- Muốn kiểm tra lời giải, em có thể so với vòng lặp cộng dồn trực tiếp khi n nhỏ.

BÀI 10 Nghịch đảo mô-đun

ÔN NHANH KIẾN THỨC · Tập 1, Bài 10

- **Nghịch đảo:** $x = b^{-1} \bmod m$ nếu $b \cdot x \equiv 1 \pmod m$; tồn tại và duy nhất khi và chỉ khi $\gcd(b, m) = 1$. "Chia cho b" theo mô-đun là nhân với b^{-1}.
- **Fermat nhỏ:** p nguyên tố, $p \nmid b$ thì $b^{p-1} \equiv 1$, do đó $b^{-1} \equiv b^{p-2} \pmod p$, tính bằng lũy thừa nhanh trong $O(\log p)$.
- **Mô-đun bất kì:** nghịch đảo là hệ số x trong $b \cdot x + m \cdot y = 1$ (Euclid mở rộng); đưa x âm về $[0, m)$. **Nghịch đảo của 1..n:** $inv[i] = (p - \lfloor p/i \rfloor) \cdot inv[p \bmod i] \bmod p$, $O(n)$.
- **Phương trình $a \cdot x \equiv c \pmod m$:** có nghiệm $\Leftrightarrow$ $g = \gcd(a, m)$ chia hết c; khi đó nghiệm là $x \equiv x_0 \pmod{m/g}$, có đúng g nghiệm trong $[0, m)$.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
10.1	**XÁC SUẤT**	★☆☆ CƠ BẢN	Biểu diễn phân số theo mô-đun $10^9 + 7$; trừ theo mô-đun không ra số âm.
10.2	**NGHỊCH ĐẢO DÃY**	★★☆ TRUNG BÌNH	Công thức truy hồi cho nghịch đảo; tiết kiệm thời gian và bộ nhớ với $n = 10^7$.
10.3	**PHƯƠNG TRÌNH ĐỒNG DƯ**	★★★ NÂNG CAO	Giải $a \cdot x \equiv c \pmod m$ tổng quát; đếm nghiệm trên đoạn với số âm và số cỡ 10^{18}.

BÀI 10.1 XÁC SUẤT ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`XACSUAT.CPP`	`XACSUAT.INP`	`XACSUAT.OUT`	1 giây · 256 MB

Kiến thức: *Phép chia theo mô-đun nguyên tố, nghịch đảo bằng định lí Fermat (Tập 1, Bài 10).*

Có n chiếc túi. Túi thứ i chứa x_i viên bi đỏ và y_i viên bi xanh. Từ **mỗi** túi, em rút ngẫu nhiên đúng một viên bi (mọi viên trong túi có khả năng được rút như nhau, các túi rút độc lập với nhau).

Hãy tính hai xác suất: P_1 là xác suất **mọi** viên bi rút được đều đỏ, và P_2 là xác suất có **ít nhất một** viên bi đỏ.

Mỗi xác suất là một phân số P/Q tối giản; có thể chứng minh rằng với ràng buộc của bài, Q không chia hết cho $10^9 + 7$. Hãy in mỗi xác suất dưới dạng $P \cdot Q^{-1} \bmod (10^9 + 7)$, tức là số nguyên R duy nhất trong $[0, 10^9 + 7)$ thỏa $Q \cdot R \equiv P \pmod{10^9 + 7}$.

Dữ liệu vào

Dòng đầu chứa số nguyên n. Dòng thứ i trong n dòng tiếp theo chứa hai số nguyên x_i và y_i.

Kết quả

Dòng thứ nhất là P_1, dòng thứ hai là P_2, đều in theo mô-đun $10^9 + 7$ như trên.

Ràng buộc

$1 \le n \le 10^5$; $x_i, y_i \ge 0$; $1 \le x_i + y_i \le 10^9$.

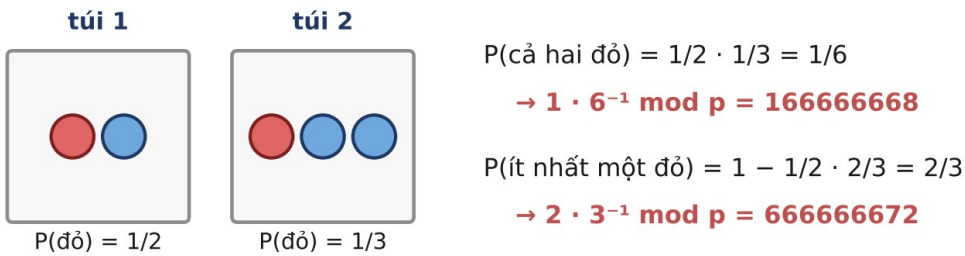
Subtask	Điểm	Ràng buộc thêm
1	30%	$n \le 10$ và $x_i + y_i \le 10$.
2	30%	$n = 1$.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

XACSUAT.INP	XACSUAT.OUT	Giải thích
2 1 1 1 2	166666668 666666672	$P_1 = 1/2 \cdot 1/3 = 1/6$; $6 \cdot 166\,666\,668 = 1\,000\,000\,008 \equiv 1$. Hai viên đều xanh với xác suất $1/2 \cdot 2/3 = 1/3$, nên $P_2 = 2/3 \equiv 666\,666\,672$.

XACSUAT.INP	XACSUAT.OUT	Giải thích
2 0 5 3 0	0 1	Túi 1 không có bi đỏ nên $P_1 =$ **0**. Túi 2 chỉ có bi đỏ nên chắc chắn có bi đỏ: $P_2 =$ **1**.

XACSUAT.INP	XACSUAT.OUT	Giải thích
3 1 999999999 1 999999999 1 999999999	793002921 17492711	$P_1 = 1/10^{27} \equiv 793\,002\,921$. $P_2 = 1 - (1 - 10^{-9})^3 \equiv 17\,492\,711$. Dù P_2 là số thực rất nhỏ, số in ra có thể là số nào cũng được trong $[0, 10^9 + 7)$.



Hình 10.1. *Ví dụ 1: hai túi, xác suất "cùng đỏ" là tích các xác suất từng túi; xác suất "ít nhất một đỏ" là phần bù của "tất cả đều xanh".*

CHÚ Ý

Không được tính xác suất bằng `double` rồi đổi sang mô-đun: phân số theo mô-đun không phải số thực.

$P_2 = 1 - (\text{tích các } y_i) / (\text{tích các } x_i + y_i)$. Phép trừ theo mô-đun có thể ra số âm trong C++: cộng thêm $10^9 + 7$ rồi lấy dư lần nữa.

$x_i + y_i \le 10^9 < 10^9 + 7$ nên mẫu số không bao giờ chia hết cho $10^9 + 7$: nghịch đảo luôn tồn tại. Không cần rút gọn phân số trước khi đổi sang mô-đun.

Nhân dần và lấy dư sau mỗi bước: tích của 10^5 số cỡ 10^9 không vừa kiểu nào.

HƯỚNG SUY NGHĨ

- Hai túi độc lập nên xác suất "cả hai đỏ" bằng tích hai xác suất. Có thể tính riêng tử số $\Pi\, x_i$ và mẫu số $\Pi\, (x_i + y_i)$ theo mô-đun.
- Cả bài chỉ cần **một** lần tìm nghịch đảo (của mẫu số chung), bằng $b^{p-2} \bmod p$.

BÀI 10.2	NGHỊCH ĐẢO DÃY	★★☆ TRUNG BÌNH	
Chương trình **NGHICHDAO.CPP**	Dữ liệu vào **NGHICHDAO.INP**	Kết quả **NGHICHDAO.OUT**	Giới hạn **1 giây · 256 MB**

Kiến thức: *Nghịch đảo của mọi số 1..n theo mô-đun nguyên tố trong O(n) (Tập 1, Bài 10, Mở rộng 1).*

Cho số nguyên tố p và số nguyên n < p. Với mỗi i = 1, 2, ..., n, gọi inv(i) là nghịch đảo của i theo mô-đun p, tức là số nguyên duy nhất trong khoảng [1, p − 1] thỏa $i \cdot \text{inv}(i) \equiv 1 \pmod p$.

Vì in ra n số là quá nhiều, em chỉ cần in hai "mã kiểm tra":

- H = (inv(1) + inv(2) + ... + inv(n)) mod p;
- X = inv(1) **xor** inv(2) **xor** ... **xor** inv(n), trong đó xor là phép "hoặc loại trừ" theo bit (toán tử ^ trong C++), thực hiện trên chính các giá trị inv(i) (không lấy dư).

Dữ liệu vào

Một dòng chứa hai số nguyên n và p.

Kết quả

Dòng thứ nhất là H, dòng thứ hai là X.

Ràng buộc

$1 \le n < p \le 2 \cdot 10^9$; p là số nguyên tố; $n \le 10^7$.

Subtask	Điểm	Ràng buộc thêm
1	20%	$n \le 1000$, $p \le 10^6$.
2	30%	$n \le 10^6$.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

NGHICHDAO.INP	NGHICHDAO.OUT	Giải thích
`10 11`	`0` `11`	Nghịch đảo của 1..10 theo mô-đun 11 là 1, 6, 4, 3, 9, 2, 8, 7, 5, 10 (Hình 10.2): một hoán vị của 1..10, nên H = 55 mod 11 = **0**. Xor của chúng bằng 11.

NGHICHDAO.INP	NGHICHDAO.OUT	Giải thích
`6 1000000007`	`650000007` `516764016`	Các nghịch đảo: 1, 500 000 004, 333 333 336, 250 000 002, 400 000 003, 166 666 668. H chính là 1 + 1/2 + ... + 1/6 = 49/20 theo mô-đun $10^9 + 7$, bằng 650 000 007.

NGHICHDAO.INP	NGHICHDAO.OUT	Giải thích
1 2	1 1	Trường hợp nhỏ nhất: inv(1) = 1.



Hình 10.2. *Bảng truy hồi với p = 11: inv[i] = (p − ⌊p/i⌋) · inv[p mod i] mod p, trong đó p mod i < i nên inv[p mod i] đã có sẵn.*

CHÚ Ý

Gọi lũy thừa nhanh i^{p-2} cho từng i tốn khoảng 60 phép nhân cho mỗi số: với $n = 10^7$ là $6 \cdot 10^8$ phép nhân và phép chia lấy dư, quá giới hạn 1 giây.

$(p - \lfloor p/i \rfloor) \cdot \text{inv}[p \bmod i]$ có thể tới $4 \cdot 10^{18}$: vừa `long long` nhưng không vừa `int`.

Mảng 10^7 phần tử `long long` tốn 80 MB; vì $p < 2^{31}$ nên có thể dùng `unsigned int` (40 MB).

X lấy xor trên giá trị **thật** của inv(i) trong [1, p − 1]; còn H mới lấy dư theo p.

HƯỚNG SUY NGHĨ

- Chia p cho i: $p = q \cdot i + r$ với $0 < r < i$. Lấy đồng dư theo p: $q \cdot i + r \equiv 0$. Nhân hai vế với $i^{-1} \cdot r^{-1}$ xem được gì?
- Công thức cho inv(i) chỉ dùng inv(r) với r < i: tính theo thứ tự i tăng dần, O(n) tổng cộng.

BÀI 10.3	PHƯƠNG TRÌNH ĐỒNG DƯ	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
PTDONGDU.CPP	PTDONGDU.INP	PTDONGDU.OUT	1 giây · 256 MB

Kiến thức: *Phương trình đồng dư bậc nhất, gcd và Euclid mở rộng (Tập 1, Bài 10, Mở rộng 3; Bài 4).*

Cho các số nguyên a, c, m (m > 0) và một đoạn [L, R]. Một số nguyên x gọi là **nghiệm** của phương trình đồng dư

$$\mathbf{a \cdot x \equiv c \pmod{m}}$$

nếu a · x − c chia hết cho m. Chú ý a, c và các đầu mút L, R có thể âm.

Với mỗi câu hỏi, hãy đếm số nghiệm x nằm trong đoạn [L, R] (tức $L \le x \le R$), và nếu có nghiệm thì cho biết nghiệm **nhỏ nhất** trong đoạn.

Dữ liệu vào

Dòng đầu chứa số nguyên Q. Mỗi dòng trong Q dòng tiếp theo chứa năm số nguyên a, c, m, L, R.

Kết quả

Gồm Q dòng. Với mỗi câu hỏi: nếu không có nghiệm nào trong [L, R] thì in số 0; ngược lại in hai số cách nhau một dấu cách: số nghiệm trong [L, R] và nghiệm nhỏ nhất trong [L, R].

Ràng buộc

$1 \le Q \le 10^5$; $|a|, |c| \le 10^{18}$; $1 \le m \le 10^{18}$; $-10^{18} \le L \le R \le 10^{18}$.

Subtask	Điểm	Ràng buộc thêm
1	20%	$Q \le 10$, $m \le 10^6$, $-10^6 \le L \le R \le 10^6$.
2	30%	$\gcd(a, m) = 1$, $m \le 10^9$.
3	50%	Không có ràng buộc gì thêm.

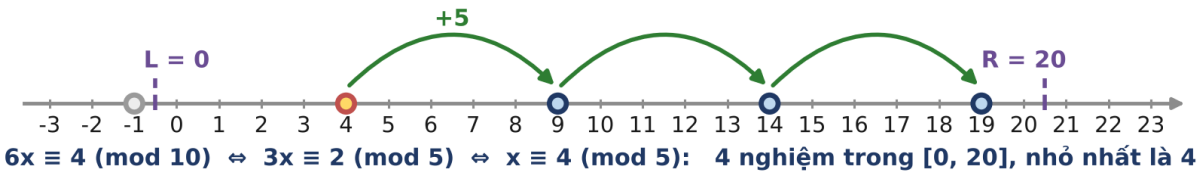
Ví dụ

PTDONGDU.INP	PTDONGDU.OUT	Giải thích
3 6 4 10 0 20 6 3 10 0 100 3 2 5 -10 10	4 4 0 4 -6	6x ≡ 4 (mod 10): g = 2 | 4, chia đôi được 3x ≡ 2 (mod 5), x ≡ 4 (mod 5). Trong [0, 20] có 4, 9, 14, 19. 6x ≡ 3 (mod 10): 2 ∤ 3, vô nghiệm. x ≡ 4 (mod 5) trên [−10, 10]: −6, −1, 4, 9.

PTDONGDU.INP	PTDONGDU.OUT
3 0 0 7 -5 5 0 3 7 -5 5 -3 1 1000000000000000000 -1000000000000000000 0	11 -5 0 1 -666666666666666667

Giải thích ví dụ 2. Hai câu hỏi đầu có a = 0: phương trình thành 0 ≡ c (mod 7). Với c = 0 **mọi** số nguyên đều là nghiệm, cả 11 số trong [−5, 5]; với c = 3 thì không có nghiệm nào.

Câu hỏi thứ ba: $-3x \equiv 1 \pmod{10^{18}}$ tương đương $3x \equiv -1$. Vì $3 \cdot 666\,666\,666\,666\,666\,667 = 2 \cdot 10^{18} + 1$, nghịch đảo của 3 là 666 666 666 666 666 667, nên $x \equiv -666\,666\,666\,666\,666\,667 \pmod{10^{18}}$. Trong $[-10^{18}, 0]$ chỉ có đúng một nghiệm là −666 666 666 666 666 667. Tích $a \cdot x$ ở đây cỡ 10^{36}.



Hình 10.3. Ví dụ 1, câu hỏi 1: các nghiệm cách đều nhau m/g = 5 đơn vị; nghiệm nhỏ nhất trong đoạn được tô vàng, nghiệm ngoài đoạn tô xám.

CHÚ Ý

Đưa a và c về [0, m) trước khi làm gì khác: `a % m` trong C++ có thể âm.

a = 0 (hoặc a chia hết cho m): gcd(0, m) = m, phương trình có nghiệm khi và chỉ khi m | c, và khi đó **mọi** x đều là nghiệm. Số nghiệm có thể tới $2 \cdot 10^{18} + 1$, vẫn vừa `long long`.

Khi m tới 10^{18}, tích $(c/g) \cdot (a/g)^{-1}$ phải nhân bằng `__int128` trước khi lấy dư.

Tìm nghiệm nhỏ nhất ≥ L bằng $L + ((x_0 - L) \bmod m')$ với phép lấy dư không âm; $x_0 - L$ có thể tới $2 \cdot 10^{18}$, vẫn vừa `long long`. Đừng đi từng bước m' từ x_0.

HƯỚNG SUY NGHĨ

- Đặt g = gcd(a, m). Nếu g ∤ c thì vô nghiệm. Ngược lại chia cả a, c, m cho g được $a' \cdot x \equiv c' \pmod{m'}$ với gcd(a', m') = 1.
- Khi đó tập nghiệm là một lớp đồng dư $x \equiv x_0 \pmod{m'}$, với $x_0 = c' \cdot a'^{-1} \bmod m'$ (nghịch đảo bằng Euclid mở rộng, vì m' không nhất thiết là số nguyên tố).
- Đếm số phần tử của một cấp số cộng công sai m' trong đoạn [L, R]: tìm phần tử đầu tiên ≥ L rồi dùng một phép chia.

BÀI 11 Hàm phi Euler và định lí Euler

ÔN NHANH KIẾN THỨC · Tập 1, Bài 11

- **φ(n)** là số các k ∈ [1, n] có gcd(k, n) = 1. φ(p) = p − 1, $\varphi(p^k) = p^k - p^{k-1}$, φ(1) = 1.
- **Công thức tích:** $\varphi(n) = n \cdot \Pi(1 - 1/p)$ qua các ước nguyên tố khác nhau p của n; tính trong O(√n) bằng res ← res − res / p, nhớ ước nguyên tố lớn còn lại. **Sàng φ** cho 1..N trong O(N log log N).
- **Định lí Euler:** gcd(a, m) = 1 ⇒ $a^{\varphi(m)} \equiv 1 \pmod m$, nên số mũ được lấy dư theo φ(m).
- **Euler mở rộng:** với **mọi** a và mọi E ≥ φ(m): $a^E \equiv a^{(E \bmod \varphi(m)) + \varphi(m)} \pmod m$. Dãy m, φ(m), φ(φ(m)), ... về 1 sau O(log m) bước.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
11.1	**PHI**	★☆☆ CƠ BẢN	Nhận ra hàm φ trong một câu chuyện; cài đặt O(√n) không sai ước lớn còn lại.
11.2	**PHÂN SỐ TỐI GIẢN**	★★☆ TRUNG BÌNH	Sàng một hàm số học tới 10^7; trả lời truy vấn đoạn bằng tổng tiền tố.
11.3	**THÁP LŨY THỪA**	★★★ NÂNG CAO	Rút gọn số mũ với mô-đun bất kì; biết khi nào được rút gọn bằng cách chặn trên giá trị thật của tháp.

BÀI 11.1 PHI ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
PHI.CPP	**PHI.INP**	**PHI.OUT**	**1 giây · 256 MB**

Kiến thức: *Hàm phi Euler, công thức tích, thử chia tới √n (Tập 1, Bài 11).*

Mặt đồng hồ của một chiếc máy lạ có n vạch, đánh số 0, 1, ..., n − 1 theo chiều kim đồng hồ. Kim của máy bắt đầu ở vạch 0, và mỗi giây nhảy đúng k vạch theo chiều kim đồng hồ: từ vạch v nhảy tới vạch (v + k) mod n.

Với một số k, kim sớm muộn đi qua **mọi** vạch của mặt đồng hồ; với số k khác, kim chỉ quanh quẩn ở vài vạch. Hỏi có bao nhiêu số k trong 1, 2, ..., n để kim đi qua đủ n vạch? Hãy trả lời Q câu hỏi như vậy.

Dữ liệu vào

Dòng đầu chứa số nguyên Q. Mỗi dòng trong Q dòng tiếp theo chứa một số nguyên n.

Kết quả

Gồm Q dòng, dòng thứ i là số các giá trị k thỏa mãn của câu hỏi thứ i.

Ràng buộc

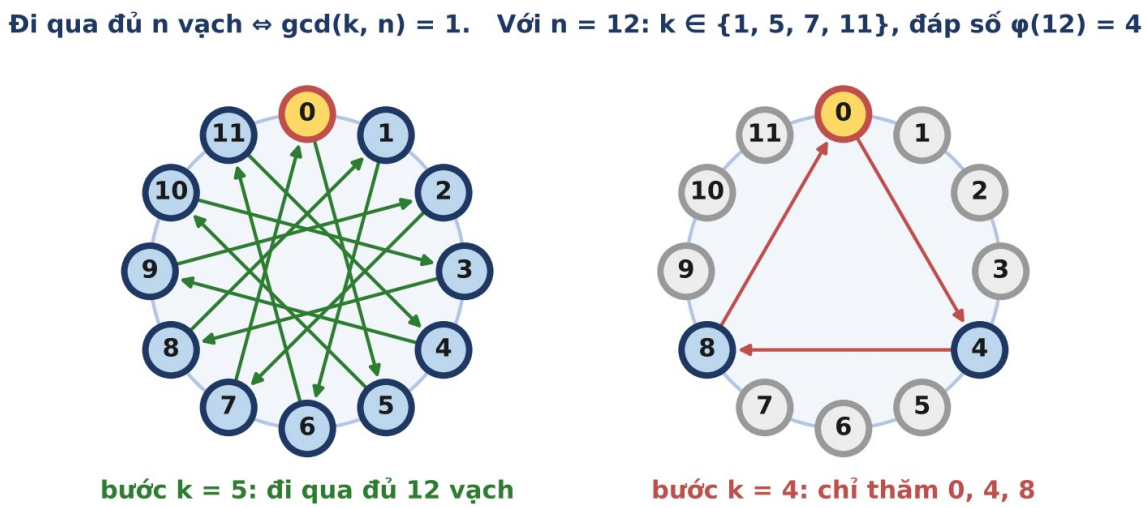
$1 \le Q \le 100$; $1 \le n \le 10^{12}$.

Subtask	Điểm	Ràng buộc thêm
1	30%	$n \le 10^4$.
2	30%	$Q = 1$.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

PHI.INP	PHI.OUT	Giải thích
3 12 7 1	4 6 1	n = 12: chỉ k = 1, 5, 7, 11 thỏa mãn (Hình 11.1). n = 7 nguyên tố: mọi k từ 1 tới 6 đều được; k = 7 thì kim đứng yên. n = 1: mặt đồng hồ chỉ có vạch 0, kim đã ở đó: k = 1 thỏa mãn.

PHI.INP	PHI.OUT	Giải thích
2 1000000000000 999999999989	400000000000 999999999988	$10^{12} = 2^{12} \cdot 5^{12}$, đáp số $10^{12} \cdot (1/2) \cdot (4/5) = 4 \cdot 10^{11}$. 999 999 999 989 là số nguyên tố: phải thử chia tới 10^6 mới biết.



Hình 11.1. Mặt đồng hồ 12 vạch. Bước k = 5 đi qua đủ 12 vạch; bước k = 4 chỉ thăm 0, 4, 8 vì gcd(4, 12) = 4.

CHÚ Ý

Đáp số chính là φ(n), nhưng em cần tự thuyết phục mình vì sao: kim đi qua đủ n vạch khi và chỉ khi tồn tại t với $t \cdot k \equiv 1 \pmod n$.

Sau vòng thử chia tới √n, nếu phần còn lại của n lớn hơn 1 thì đó là một ước nguyên tố lớn: đừng quên nhân thêm (1 − 1/n).

Đừng nhân với (1 − 1/p) bằng số thực; dùng `res -= res / p`. Biến chạy p cũng phải là `long long` vì $p \cdot p$ có thể vượt 2^{31}.

Với Q = 100 số nguyên tố cỡ 10^{12}, chương trình thử chia khoảng $100 \cdot 10^6$ lần. Chỉ thử 2 rồi các số lẻ giúp giảm một nửa thời gian.

HƯỚNG SUY NGHĨ

- Kim nhảy k vạch mỗi lần thăm đúng các vạch là bội của gcd(k, n). Vậy khi nào nó thăm được vạch 1?
- Số các k trong 1..n có gcd(k, n) = 1 có một cái tên: hàm φ(n). Tính nó bằng công thức tích sau khi phân tích n ra thừa số nguyên tố.

BÀI 11.2	PHÂN SỐ TỐI GIẢN	★★☆ TRUNG BÌNH

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
PHANSO.CPP	**PHANSO.INP**	**PHANSO.OUT**	**1 giây · 256 MB**

Kiến thức: *Sàng φ, tổng tiền tố, đếm phân số tối giản (Tập 1, Bài 11, Mở rộng 2).*

Phân số a/b (a, b nguyên dương) gọi là **tối giản** nếu gcd(a, b) = 1. Mỗi số hữu tỉ dương được viết dưới dạng phân số tối giản theo đúng một cách.

Cô giáo có Q câu hỏi. Câu hỏi thứ i gồm hai số L, R: hãy đếm số phân số tối giản a/b thỏa mãn 0 < a/b < 1 và **mẫu số** b nằm trong đoạn [L, R].

Dữ liệu vào

Dòng đầu chứa số nguyên Q. Mỗi dòng trong Q dòng tiếp theo chứa hai số nguyên L và R.

Kết quả

Gồm Q dòng, dòng thứ i là đáp số của câu hỏi thứ i.

Ràng buộc

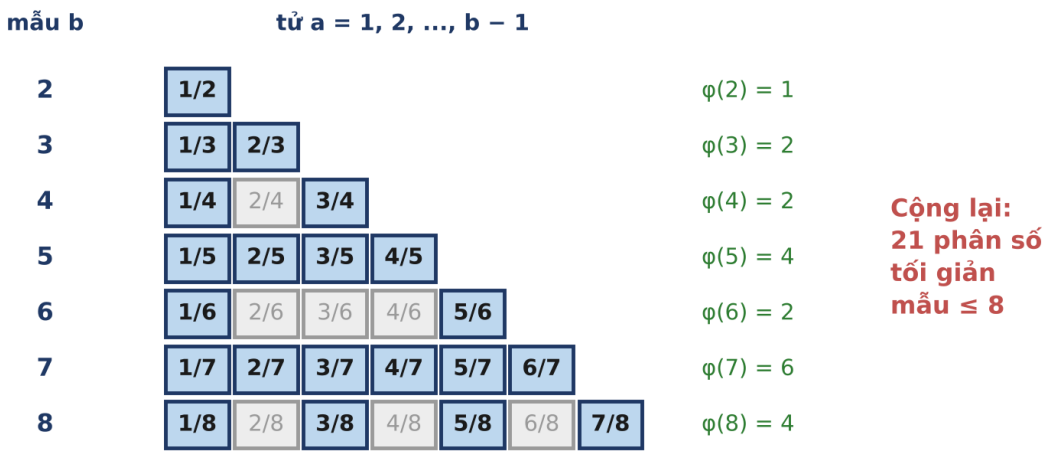
$1 \le Q \le 10^5$; $1 \le L \le R \le 10^7$.

Subtask	Điểm	Ràng buộc thêm
1	20%	$Q \le 10$, $R \le 1000$.
2	30%	$R \le 10^5$.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

PHANSO.INP	PHANSO.OUT	Giải thích
3 1 8 2 2 5 5	21 1 4	Mẫu từ 1 tới 8: có 21 phân số (Hình 11.2). Mẫu 1 không cho phân số nào trong (0, 1). Mẫu 2: chỉ có 1/2. Mẫu 5: 1/5, 2/5, 3/5, 4/5.

PHANSO.INP	PHANSO.OUT	Giải thích
2 1 1 1 10000000	0 30396356427241	b = 1 thì a/b ≥ 1: không có phân số nào. Mẫu từ 1 tới 10^7: hơn $3 \cdot 10^{13}$ phân số, vượt xa kiểu `int`.



Hình 11.2. *Phân số a/b với 1 ≤ a < b ≤ 8: ô xanh là phân số tối giản; hàng mẫu b có đúng φ(b) ô xanh.*

CHÚ Ý

Với b ≥ 2, số phân số tối giản mẫu b trong (0, 1) là φ(b). Riêng b = 1 cho **0** chứ không phải φ(1) = 1.

Gọi hàm φ thử chia cho từng b tới 10^7 tốn khoảng $10^7 \cdot \sqrt{(10^7)}$ phép tính: quá chậm. Hãy sàng.

Tổng tiền tố có thể tới khoảng $3 \cdot 10^{13}$: dùng `long long`. Mảng φ kiểu `int` (40 MB) cộng mảng tổng tiền tố `long long` (80 MB) vẫn vừa 256 MB.

Chỉ cần sàng tới R lớn nhất trong các câu hỏi; đọc hết câu hỏi trước rồi mới sàng.

HƯỚNG SUY NGHĨ

- Cố định mẫu b ≥ 2: có bao nhiêu tử a trong 1..b − 1 để a/b tối giản?
- Tính φ(1), ..., φ(N) bằng sàng: với mỗi số nguyên tố p (nhận ra vì phi[p] vẫn bằng p), gán phi[j] ← phi[j] − phi[j] / p cho mọi bội j của p.
- Đáp số của câu hỏi là S(R) − S(L − 1) với S(b) là tổng φ từ 2 tới b.

BÀI 11.3	THÁP LŨY THỪA	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
THAPLT.CPP	**THAPLT.INP**	**THAPLT.OUT**	**1 giây · 256 MB**

Kiến thức: *Định lí Euler mở rộng, đệ quy theo dãy m, φ(m), φ(φ(m)), ... (Tập 1, Bài 11, Mở rộng 3).*

Cho dãy k số tự nhiên $a_1, a_2, ..., a_k$ và số nguyên dương m. **Tháp lũy thừa** của dãy được tính từ trên xuống (từ phải sang trái):

$$\mathbf{a_1 \text{ mũ } (a_2 \text{ mũ } (... \text{ mũ } (a_{k-1} \text{ mũ } a_k)))}$$

Chẳng hạn tháp của dãy 2, 3, 2 là 2 mũ $(3^2) = 2^9 = 512$, chứ không phải $(2^3)^2 = 64$. Tháp của dãy chỉ có một số a_1 là chính a_1. Quy ước $0^0 = 1$.

Hãy tính số dư khi chia tháp lũy thừa cho m. Chú ý m là số **bất kì**, các a_i và m có thể có ước chung.

Dữ liệu vào

Dòng đầu chứa số nguyên T là số bộ dữ liệu. Mỗi bộ gồm hai dòng: dòng đầu chứa k và m, dòng sau chứa k số $a_1, a_2, ..., a_k$.

Kết quả

Gồm T dòng, mỗi dòng là số dư khi chia tháp lũy thừa của một bộ dữ liệu cho m.

Ràng buộc

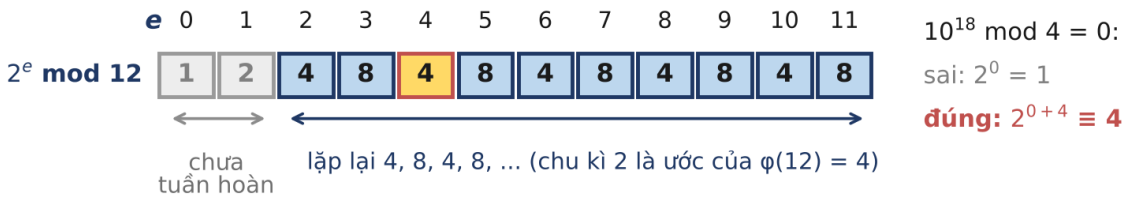
$1 \le T \le 10$; $1 \le k \le 10^5$; $0 \le a_i \le 10^9$; $1 \le m \le 10^9$.

Subtask	Điểm	Ràng buộc thêm
1	20%	$k \le 3$ và giá trị đúng của tháp không vượt quá 10^{18}.
2	20%	$k = 3$, m là số nguyên tố, a_1 không chia hết cho m, $a_2 \ge 1$.
3	30%	$k = 3$, $a_2 \ge 1$.
4	30%	Không có ràng buộc gì thêm.

Ví dụ

THAPLT.INP	THAPLT.OUT	Giải thích
3 3 1000 2 3 2 3 12 2 10 18 4 7 2 0 0 5	512 4 2	2 mũ $(3^2) = 2^9 =$ **512**. 2 mũ (10^{18}) mod 12 = **4**. Rút gọn số mũ theo φ(12) = 4 được 10^{18} mod 4 = 0 và $2^0 = 1$: **sai**, vì gcd(2, 12) ≠ 1 (Hình 11.3). $0^5 = 0$, $0^0 = 1$, nên tháp là $2^1 =$ **2**.

THAPLT.INP	THAPLT.OUT	Giải thích
3 4 1000 3 3 3 3 5 1000000007 2 2 2 2 2 1 1 5	387 973586826 0	3 mũ (3 mũ 3) = 3^{27} = 7 625 597 484 987, và tháp là 3 mũ số đó; theo mô-đun 1000 bằng **387**. 2 mũ (2 mũ (2 mũ (2 mũ 2))) = $2^{65536} \equiv 973\,586\,826$. Mọi số chia cho 1 đều dư 0.



Hình 11.3. *Dãy $2^e \bmod 12$: hai số hạng đầu chưa tuần hoàn, sau đó lặp lại với chu kì 2. Rút gọn số mũ chỉ đúng khi số mũ đủ lớn và phải cộng thêm φ(12).*

CHÚ Ý

Định lí Euler thường (lấy số mũ mod φ(m)) chỉ đúng khi gcd(a, m) = 1. Với mô-đun bất kì phải dùng dạng mở rộng: $a^E \equiv a^{(E \bmod \varphi(m)) + \varphi(m)} \pmod m$, và **chỉ khi E ≥ φ(m)**.

Vậy ngoài E mod φ(m), em còn cần biết số mũ thật E có ≥ φ(m) hay không. Hãy tính "giá trị chặn" min(E, C) của mọi tháp con với một hằng số $C > 10^9$: phép này nhanh vì với cơ số ≥ 2, số mũ 64 đã vượt C.

Các số 0 và 1 trong dãy làm thay đổi tất cả: $0^0 = 1$, $0^x = 0$ khi $x \ge 1$, $1^x = 1$. Ví dụ 1 có tháp 2, 0, 0, 5.

Dãy m, φ(m), φ(φ(m)), ... về 1 chỉ sau chừng 30 bước; khi mô-đun bằng 1 thì kết quả là 0, nên chỉ vài chục số đầu của dãy cần đệ quy.

HƯỚNG SUY NGHĨ

- Gọi f(i, m) là tháp của a_i, ..., a_k theo mô-đun m. Để tính f(i, m) em cần số mũ E = tháp của a_{i+1}, ..., a_k: hoặc giá trị thật của E (nếu E nhỏ hơn φ(m)), hoặc f(i + 1, φ(m)).
- Tính φ(m) bằng thử chia tới √m; mỗi bộ dữ liệu chỉ cần φ của khoảng 30 mô-đun.
- Kiểm tra lời giải trên các tháp nhỏ bằng số nguyên lớn của Python: tính đúng giá trị tháp rồi mới lấy dư.

BÀI 12 Định lí số dư Trung Hoa

ÔN NHANH KIẾN THỨC · Tập 1, Bài 12

- **Định lí:** m_1, m_2 nguyên tố cùng nhau thì hệ $x \equiv a_1 \pmod{m_1}$, $x \equiv a_2 \pmod{m_2}$ có đúng một nghiệm trong $[0, m_1m_2)$. Tổng quát, với $g = \gcd(m_1, m_2)$: có nghiệm $\Leftrightarrow a_1 \equiv a_2 \pmod{g}$, nghiệm duy nhất theo mô-đun $\operatorname{lcm}(m_1, m_2)$.
- **Gộp từng cặp:** từ $x \equiv r \pmod{M}$ và $x \equiv a \pmod{m}$, giải $M \cdot t \equiv a - r \pmod{m}$: $g \nmid (a - r)$ thì vô nghiệm, ngược lại $t \equiv (a - r)/g \cdot (M/g)^{-1} \pmod{m/g}$; nghiệm mới $r + M \cdot t$ theo mô-đun $M \cdot (m/g)$. Bắt đầu từ $(r, M) = (0, 1)$.
- **Tránh tràn:** nghịch đảo lấy bằng Euclid mở rộng (m/g không nhất thiết nguyên tố); $r + M \cdot t < \operatorname{lcm}$, nhân bằng `__int128` khi lcm có thể vượt 10^{18}.
- **Can Chi:** 10 Can và 12 Chi chỉ tạo ra $\operatorname{lcm}(10, 12) = 60$ tên năm, vì Can và Chi phải cùng tính chẵn lẻ.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
12.1	**ĐIỂM BINH**	★☆☆ CƠ BẢN	Cài đặt gộp từng cặp phương trình bằng Euclid mở rộng; tìm nghiệm nhỏ nhất không nhỏ hơn một ngưỡng.
12.2	**HAI NGỌN ĐÈN**	★★☆ TRUNG BÌNH	Gộp hai phương trình tổng quát; xử lí điều kiện "đã bắt đầu" ngoài đồng dư.
12.3	**LỊCH CAN CHI**	★★★ NÂNG CAO	Gộp nhiều phương trình khi lcm vượt xa 10^{18}; tính lcm theo mô-đun qua phân tích thừa số.

BÀI 12.1 ĐIỂM BINH ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`DIEMBINH.CPP`	`DIEMBINH.INP`	`DIEMBINH.OUT`	1 giây · 256 MB

Kiến thức: *Định lí số dư Trung Hoa với các mô-đun đôi một nguyên tố cùng nhau, gộp từng cặp (Tập 1, Bài 12).*

Danh tướng Hàn Tín điểm quân theo cách riêng: ông cho quân xếp lần lượt thành các hàng m_1, m_2, ..., m_k người, mỗi lần chỉ ghi lại số người lẻ ra ở hàng cuối: xếp hàng m_i thì thừa a_i người ($0 \le a_i < m_i$). Các số m_i **đôi một nguyên tố cùng nhau**.

Ông còn biết đội quân có **không ít hơn** L người. Hỏi đội quân có ít nhất bao nhiêu người? Nói cách khác, hãy tìm số nguyên x nhỏ nhất thỏa $x \geq L$ và $x \bmod m_i = a_i$ với mọi i. Theo định lí số dư Trung Hoa, số như vậy luôn tồn tại.

Dữ liệu vào

Dòng đầu chứa hai số nguyên k và L. Dòng thứ i trong k dòng tiếp theo chứa hai số nguyên m và a_i.

Kết quả

Một số nguyên là số x tìm được.

Ràng buộc

$1 \leq k \leq 10$; $0 \leq L \leq 10^{18}$; $1 \leq m_i \leq 10^9$; $0 \leq a_i < m_i$; các m_i đôi một nguyên tố cùng nhau và $m_1 \cdot m_2 \cdot ... \cdot m_k \leq 10^{18}$.

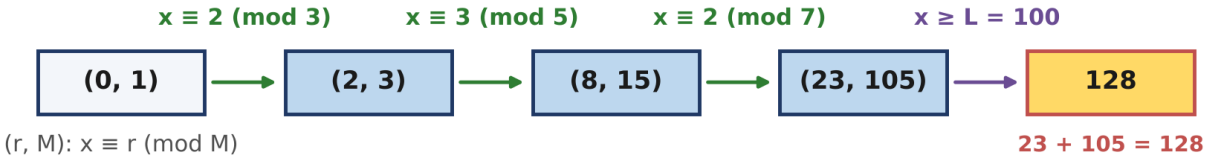
Subtask	Điểm	Ràng buộc thêm
1	30%	$m_1 \cdot m_2 \cdot ... \cdot m_k \leq 10^6$ và $L \leq 10^6$.
2	30%	$L = 0$.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

DIEMBINH.INP	DIEMBINH.OUT	Giải thích
3 0 3 2 5 3 7 2	23	Bài toán của Hàn Tín: $23 = 3 \cdot 7 + 2 = 5 \cdot 4 + 3 = 7 \cdot 3 + 2$.

DIEMBINH.INP	DIEMBINH.OUT	Giải thích
4 100 3 2 5 3 7 2 1 0	128	Các nghiệm là 23, 128, 233, ... (cách nhau $3 \cdot 5 \cdot 7 = 105$). Số nhỏ nhất ≥ 100 là **128**. Phương trình với m = 1 luôn đúng.

DIEMBINH.INP	DIEMBINH.OUT	Giải thích
2 0 1000000000 1 999999999 0	999999998000000001	999 999 998 000 000 001 chia 10^9 dư 1 và chia hết cho 999 999 999; tích hai mô-đun gần 10^{18}.



Hình 12.1. *Ví dụ 2: gộp lần lượt ba phương trình vào cặp (r, M), rồi nhảy theo bước M = 105 tới số đầu tiên không nhỏ hơn L = 100.*

CHÚ Ý

Các m_i nguyên tố cùng nhau nên hệ luôn có nghiệm, nhưng nghịch đảo của M theo mô-đun m_i vẫn phải tính bằng Euclid mở rộng (m_i không nhất thiết nguyên tố).

Cách nhảy từng bước m_1 có thể cần tới 10^{18} bước. Cách thử từng x từ L lên còn tệ hơn.

Nghiệm nhỏ nhất ≥ L có thể tới $L + M - 1 < 2 \cdot 10^{18}$: vẫn vừa `long long`. Đừng quên trường hợp nghiệm nhỏ nhất r đã ≥ L.

Hiệu $a_i - r$ có thể âm; dùng phép lấy dư không âm.

HƯỚNG SUY NGHĨ

- Mọi x thỏa các phương trình đã gộp có dạng $r + M \cdot t$. Thay vào phương trình mới để tìm t.
- Khi đã có nghiệm r nhỏ nhất ($0 \le r < M$), các nghiệm khác là r + M, r + 2M, ...: tính thẳng số bước cần nhảy để vượt L.

BÀI 12.2	HAI NGỌN ĐÈN	★★☆ TRUNG BÌNH

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
HAIDEN.CPP	HAIDEN.INP	HAIDEN.OUT	1 giây · 256 MB

Kiến thức: *Hệ hai phương trình đồng dư với mô-đun có ước chung (Tập 1, Bài 12).*

Trên ngọn hải đăng có hai ngọn đèn. Đèn thứ i (i = 1, 2) nháy sáng lần đầu tiên vào giây thứ a_i sau đó cứ đúng p_i giây lại nháy một lần: nó nháy vào các giây a_i, $a_i + p_i$, $a_i + 2p_i$, ... và **không** nháy vào giây nào khác (trước giây a_i đèn chưa bật).

Người gác muốn biết thời điểm **đầu tiên** cả hai đèn cùng nháy. Hãy trả lời Q câu hỏi như vậy; nếu hai đèn không bao giờ nháy cùng lúc, in −1.

Dữ liệu vào

Dòng đầu chứa số nguyên Q. Mỗi dòng trong Q dòng tiếp theo chứa bốn số nguyên a_1, p_1, a_2, p_2.

Kết quả

Gồm Q dòng, mỗi dòng là thời điểm đầu tiên cả hai đèn cùng nháy, hoặc −1.

Ràng buộc

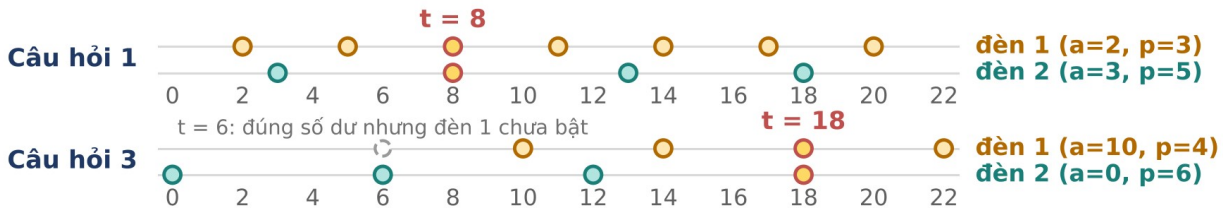
$1 \le Q \le 10^5$; $0 \le a_1, a_2 \le 10^{18}$; $1 \le p_1, p_2 \le 10^9$.

Subtask	Điểm	Ràng buộc thêm
1	30%	$a_1, a_2, p_1, p_2 \le 1000$.
2	30%	$\gcd(p_1, p_2) = 1$ và $a_1 < p_1$, $a_2 < p_2$.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

HAIDEN.INP	HAIDEN.OUT	Giải thích
3 2 3 3 5 1 4 2 6 10 4 0 6	8 -1 18	Đèn 1 nháy ở 2, 5, **8**, 11, ...; đèn 2 ở 3, **8**, 13, ... Đèn 1 luôn nháy ở giây lẻ, đèn 2 luôn ở giây chẵn: **−1**. $t \equiv 2 \pmod 4$ và $t \equiv 0 \pmod 6$ cho t = 6, 18, 30, ...; nhưng ở giây 6 đèn 1 chưa bật, nên đáp số là **18**.

HAIDEN.INP	HAIDEN.OUT	Giải thích
2 0 1000000000 999999999 999999999 1000000000000000000 2 7 1	999999999000000000 1000000000000000000	Mọi lần nháy chung phải là bội của lcm(10^9, 999 999 999) = 999 999 999 000 000 000. Đèn 2 nháy mỗi giây; đèn 1 bật ở giây 10^{18}.



Hình 12.2. *Câu hỏi 1 và 3 của ví dụ 1. Ở câu hỏi 3, giây 6 thỏa cả hai đồng dư nhưng đèn 1 chưa bật (vòng nét đứt).*

CHÚ Ý

Thời điểm cần tìm phải thỏa **ba** điều kiện: $t \equiv a_1 \pmod{p_1}$, $t \equiv a_2 \pmod{p_2}$ và $t \geq \max(a_1, a_2)$. Chỉ giải hệ đồng dư thì sai ở câu hỏi 3 của ví dụ 1.

p_1, p_2 có thể có ước chung: hệ vô nghiệm khi a_1 và a_2 không đồng dư theo $\gcd(p_1, p_2)$.

$\text{lcm}(p_1, p_2) \leq 10^{18}$ và đáp số nhỏ hơn $\max(a_1, a_2) + \text{lcm} \leq 2 \cdot 10^{18}$: vừa `long long`. Nhưng a_i đã tới 10^{18}, nên đừng cộng thêm gì "cho chắc".

Mô phỏng từng lần nháy chỉ đủ cho subtask 1.

HƯỚNG SUY NGHĨ

- Bỏ qua điều kiện "đã bật": gộp hai phương trình $x \equiv a_1 \pmod{p_1}$, $x \equiv a_2 \pmod{p_2}$ thành $x \equiv r \pmod{\text{lcm}}$ hoặc kết luận vô nghiệm.
- Sau đó tìm số nhỏ nhất $\geq \max(a_1, a_2)$ trong lớp đồng dư $x \equiv r \pmod{\text{lcm}}$.

BÀI 12.3	LỊCH CAN CHI	★★★ NÂNG CAO	
Chương trình **CANCHI.CPP**	Dữ liệu vào **CANCHI.INP**	Kết quả **CANCHI.OUT**	Giới hạn **1 giây · 256 MB**

Kiến thức: *Hệ đồng dư tổng quát, điều kiện gcd, lcm theo mô-đun và chống tràn số (Tập 1, Bài 12; Bài 7).*

Người Việt gọi tên năm bằng hai vòng: Can (10 tên) và Chi (12 tên). Đánh số các tên từ 0 và lấy năm 1984 (Giáp Tý) làm năm thứ 0, thì năm thứ x có tên Can số x mod 10 và Chi số x mod 12; chẳng hạn năm 2026 là năm thứ 42, tên (2, 6) là Bính Ngọ.

Ở vương quốc Đồng Dư, lịch phức tạp hơn: tên mỗi năm gồm k vòng, vòng thứ i có m_i tên đánh số 0, 1, ..., $m_i - 1$. Năm thứ x ($x \geq 0$) mang tên ($x \bmod m_1$, $x \bmod m_2$, ..., $x \bmod m_k$). Các m_i có thể có ước chung, nên có những tên không bao giờ xuất hiện.

Hãy cho biết:

1) Có bao nhiêu tên khác nhau thực sự xuất hiện (tính trên mọi năm $x \geq 0$)? In số này theo mô-đun $10^9 + 7$.

2) Với mỗi câu hỏi gồm một tên ($c_1, c_2, ..., c_k$) và đoạn năm [L, R]: có bao nhiêu năm x với $L \leq x \leq R$ mang tên đó, và năm sớm nhất trong số đó là năm nào?

Dữ liệu vào

Dòng đầu chứa hai số nguyên k và Q. Dòng thứ hai chứa k số nguyên $m_1, m_2, ..., m_k$. Mỗi dòng trong Q dòng tiếp theo chứa k + 2 số nguyên $c_1, c_2, ..., c_k$, L, R.

Kết quả

Dòng đầu là số tên khác nhau theo mô-đun $10^9 + 7$. Tiếp theo là Q dòng: nếu không có năm nào trong [L, R] mang tên được hỏi thì in 0; ngược lại in hai số là số năm như vậy và năm sớm nhất.

Ràng buộc

$1 \leq k \leq 20$; $1 \leq Q \leq 10^5$; $1 \leq m_i \leq 10^6$; $0 \leq c_i < m_i$; $0 \leq L \leq R \leq 10^{18}$.

Subtask	Điểm	Ràng buộc thêm
1	20%	$k = 2$, $m_i \leq 100$, $Q \leq 100$, $R \leq 10^5$.
2	30%	$m_1 \cdot m_2 \cdot ... \cdot m_k \leq 10^{18}$.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

CANCHI.INP	CANCHI.OUT	Giải thích
2 3 10 12 2 6 0 1000 0 1 0 1000 0 0 17 100	60 16 42 0 1 60	Lịch Can Chi có lcm(10, 12) = **60** tên. Bính Ngọ (2, 6): các năm 42, 102, ..., 942, tổng cộng 16 năm trong [0, 1000]. Giáp Sửu (0, 1) cần x chẵn và x lẻ: không bao giờ có. Giáp Tý (0, 0) trong [17, 100]: chỉ có năm 60.

CANCHI.INP	CANCHI.OUT	Giải thích
3 3 4 6 9 1 3 3 0 100 1 2 0 0 100 3 5 8 0 1000000000000000000	36 3 21 0 27777777777777777 35	lcm(4, 6, 9) = 36 tên. Tên (1, 3, 3): x = 21, 57, 93 trong [0, 100]. Tên (1, 2, 0): x lẻ và chẵn: vô nghiệm. Tên (3, 5, 8): năm 35, rồi cứ 36 năm một lần.

	Tý 0	Sửu 1	Dần 2	Mão 3	Thìn 4	Tỵ 5	Ngọ 6	Mùi 7	Thân 8	Dậu 9	Tuất 10	Hợi 11
Giáp 0	0		50		40		30		20		10	
Ất 1		1		51		41		31		21		11
Bính 2	12		2		52		**42**		32		22	
Đinh 3		13		3		53		43		33		23
Mậu 4	24		14		4		54		44		34	
Kỷ 5		25		15		5		55		45		35
Canh 6	36		26		16		6		56		46	
Tân 7		37		27		17		7		57		47
Nhâm 8	48		38		28		18		8		58	
Quý 9		49		39		29		19		9		59

Ô ghi số x: năm thứ x (tính từ năm 1984 Giáp Tý)

Bính Ngọ = (2, 6): x = 42, 42 + 60 = 102, ... (1984 + 42 = 2026)

Ô xám: tên không bao giờ xuất hiện, vì Can và Chi phải cùng chẵn hoặc cùng lẻ (gcd(10, 12) = 2)

Chỉ 60 = lcm(10, 12) trong 120 ô có tên năm

Hình 12.3. *Bảng 10 × 12 tên Can Chi. Ô ghi x là năm thứ x trong một vòng 60 năm; ô xám là tên không bao giờ xuất hiện.*

CHÚ Ý

Số tên xuất hiện là $\text{lcm}(m_1, ..., m_k)$, có thể tới khoảng 10^{120}. Không thể tính lcm bằng gcd rồi lấy dư; hãy phân tích từng m_i ra thừa số nguyên tố và lấy số mũ lớn nhất của mỗi số nguyên tố.

Khi gộp các phương trình, mô-đun M = lcm của các m_i đã gộp có thể vượt 10^{18}. Một khi M > R, trong [0, R] có **nhiều nhất một** năm mang tên được hỏi: đó là số dư r hiện tại. Từ đó chỉ cần kiểm tra r với các phương trình còn lại, không cần gộp tiếp.

Với $M \le 10^{18}$ và $m_i \le 10^6$, phép gộp tạo ra số tới 10^{24}: dùng `__int128` cho r và M.

Số năm trong [L, R] có thể tới $10^{18} + 1$ (khi mọi $m_i = 1$).

HƯỚNG SUY NGHĨ

- Mỗi câu hỏi là một hệ k phương trình $x \equiv c_i \pmod{m_i}$: gộp từng cặp như Bài 12, gặp g $\nmid$ (c − r) thì in 0.
- Nếu gộp xong được $x \equiv r \pmod{M}$ với $M \le 10^{18}$, đếm số phần tử của cấp số cộng trong [L, R] giống bài 10.3.
- Nghĩ xem: nếu M đã lớn hơn R thì lớp đồng dư $x \equiv r \pmod{M}$ có bao nhiêu phần tử trong [0, R]?

CHƯƠNG 4. TỔ HỢP VÀ ĐẾM

Mười hai bài tập của chương này rèn luyện bốn công cụ đếm của Tập 1: hệ số nhị thức theo mô-đun, nguyên lí bù trừ, lũy thừa ma trận cho dãy truy hồi và kĩ thuật chia khối ⌊n / i⌋. Đáp số luôn khổng lồ nên gần như bài nào cũng in kết quả theo mô-đun. Cái khó nằm ở chỗ biến câu hỏi "có bao nhiêu" thành một công thức hay một phép nhân ma trận, rồi tính nó mà không tràn số. Mỗi bài đều có subtask cho lời giải vét cạn: hãy viết nó trước và dùng nó để kiểm tra lời giải nhanh.

TRONG CHƯƠNG NÀY

Bài 13. Tổ hợp và hệ số nhị thức

- 13.1 CHIA KẸO — ★☆☆ CƠ BẢN
- 13.2 ĐƯỜNG ĐI TRÊN LƯỚI — ★★☆ TRUNG BÌNH
- 13.3 TỔ HỢP KHỔNG LỒ — ★★★ NÂNG CAO

Bài 14. Nguyên lí bù trừ

- 14.1 BỘI SỐ — ★☆☆ CƠ BẢN
- 14.2 NGUYÊN TỐ CÙNG NHAU — ★★☆ TRUNG BÌNH
- 14.3 TẶNG QUÀ — ★★★ NÂNG CAO

Bài 15. Dãy truy hồi và lũy thừa ma trận

- 15.1 LÁT GẠCH — ★☆☆ CƠ BẢN
- 15.2 TRIBONACCI — ★★☆ TRUNG BÌNH
- 15.3 ĐƯỜNG ĐI ĐỘ DÀI K — ★★★ NÂNG CAO

Bài 16. Kĩ thuật chia khối ⌊n / i⌋

- 16.1 TỔNG ƯỚC — ★☆☆ CƠ BẢN
- 16.2 TỔNG DƯ — ★★☆ TRUNG BÌNH
- 16.3 HAI THƯƠNG — ★★★ NÂNG CAO

LỜI KHUYÊN CHO CHƯƠNG NÀY

- Trước khi tin một công thức, hãy đếm tay bằng cách liệt kê với n thật nhỏ (n = 0, 1, 2, 3). Công thức không khớp những con số ấy thì chắc chắn sai.
- Theo mô-đun: phép chia là phép nhân với nghịch đảo, phép trừ có thể ra số âm, còn tích của ba số thì phải lấy dư sau từng phép nhân.
- Ghi ra giá trị lớn nhất của mọi chỉ số tra bảng (n + k − 1, m + n, ...) và mọi đại lượng trung gian (k · t, lcm, tổng có trọng số) để chọn đúng kích thước mảng và kiểu dữ liệu.

BÀI 13 Tổ hợp và hệ số nhị thức

ÔN NHANH KIẾN THỨC · Tập 1, Bài 13

- **Tổ hợp:** $C(n, k) = n! / (k! \cdot (n - k)!)$ là số tập con k phần tử của tập n phần tử; $C(n, k) = 0$ khi $k < 0$ hoặc $k > n$. Hằng đẳng thức Pascal $C(n, k) = C(n - 1, k - 1) + C(n - 1, k)$ chỉ dùng phép cộng nên đúng với mọi mô-đun.
- **p nguyên tố, p > n:** $C(n, k) \equiv fact[n] \cdot ifact[k] \cdot ifact[n - k] \pmod p$, với $ifact[N] = fact[N]^{p-2}$ và $ifact[i - 1] = ifact[i] \cdot i$. Chuẩn bị $O(N + \log p)$, mỗi truy vấn $O(1)$.
- **Chia kẹo Euler:** số nghiệm nguyên không âm của $x_1 + ... + x_k = n$ là $C(n + k - 1, k - 1)$; số nghiệm nguyên dương là $C(n - 1, k - 1)$.
- **n rất lớn:** k nhỏ thì nhân dần $n(n - 1)...(n - k + 1) / k!$; p nguyên tố nhỏ thì dùng định lí Lucas $C(n, k) \equiv \Pi\, C(n_i, k_i)$ với n_i, k_i là các chữ số trong cơ số p.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
13.1	**CHIA KẸO**	★☆☆ CƠ BẢN	Đưa bài đếm về một số C(a, b); chuẩn bị bảng giai thừa đủ lớn.
13.2	**ĐƯỜNG ĐI TRÊN LƯỚI**	★★☆ TRUNG BÌNH	Tách đường đi qua một điểm thành hai đoạn; bù trừ khi có hai điểm cấm.
13.3	**TỔ HỢP KHỔNG LỒ**	★★★ NÂNG CAO	Tính C(n, k) theo mô-đun không nguyên tố: tách theo từng ước nguyên tố rồi ghép lại.

BÀI 13.1 CHIA KẸO ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`CHIAKEO.CPP`	`CHIAKEO.INP`	`CHIAKEO.OUT`	1 giây · 256 MB

Kiến thức: *Chia kẹo Euler, giai thừa và nghịch đảo giai thừa (Tập 1, Bài 13).*

Cô giáo có n cái kẹo **giống hệt nhau** và muốn chia **hết** số kẹo cho k bạn trong nhóm. Để công bằng, cô yêu cầu **mỗi bạn nhận ít nhất t cái**. Hai cách chia được coi là khác nhau nếu có một bạn nhận được số kẹo khác nhau trong hai cách.

Nói cách khác, cần đếm số bộ số nguyên $(x_1, x_2, ..., x_k)$ thỏa mãn $x_1 + x_2 + ... + x_k = n$ và $x_i \geq t$ với mọi i.

Cô có Q câu hỏi như vậy, mỗi câu với bộ n, k, t riêng. Vì số cách chia có thể rất lớn, hãy in số dư của nó khi chia cho $10^9 + 7$.

Dữ liệu vào

Dòng đầu chứa số nguyên Q. Mỗi dòng trong Q dòng tiếp theo chứa ba số nguyên n, k, t.

Kết quả

Gồm Q dòng, dòng thứ i là số cách chia của câu hỏi thứ i, lấy dư cho $10^9 + 7$ (in 0 nếu không có cách nào).

Ràng buộc

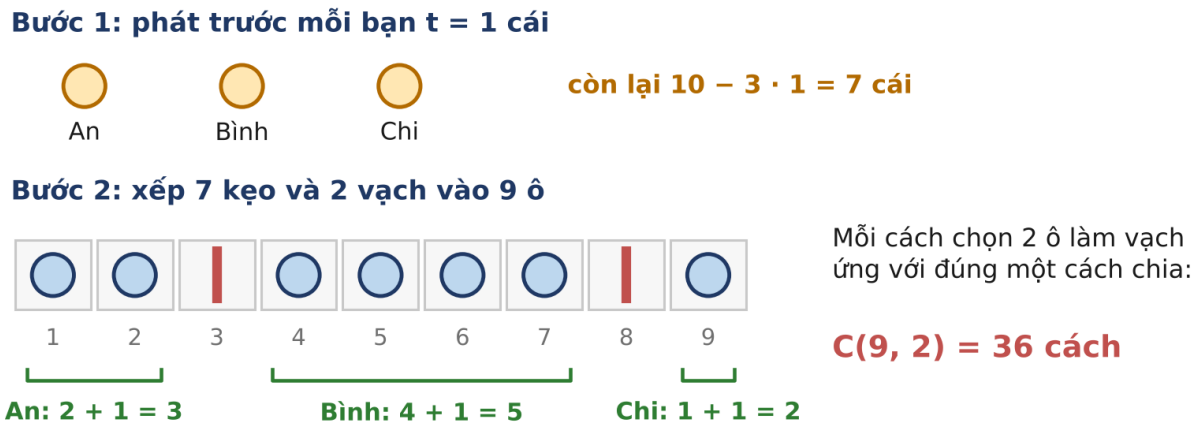
$1 \le Q \le 10^5$; $0 \le n \le 10^6$; $1 \le k \le 10^6$; $0 \le t \le 10^6$.

Subtask	Điểm	Ràng buộc thêm
1	20%	$n \le 10$, $k \le 5$.
2	30%	$n, k \le 2000$.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

CHIAKEO.INP	CHIAKEO.OUT	Giải thích
3 10 3 0 10 3 1 7 3 2	66 36 3	10 kẹo, 3 bạn, không yêu cầu gì (t = 0): đặt 2 vạch ngăn giữa 10 viên kẹo, có C(12, 2) = **66** cách. Mỗi bạn ít nhất 1 cái: phát trước mỗi bạn 1 cái, còn 7 cái chia tùy ý, C(9, 2) = **36** cách (Hình 13.1). Mỗi bạn ít nhất 2 cái: còn đúng 1 cái, trao cho một trong ba bạn: (3, 2, 2), (2, 3, 2), (2, 2, 3).

CHIAKEO.INP	CHIAKEO.OUT	Giải thích
4 5 7 1 0 4 0 1000000 1000000 1 1000000 1000000 1000000	0 1 1 0	5 kẹo không đủ để mỗi bạn trong 7 bạn có ít nhất một cái: **0**. Không có kẹo (n = 0): chỉ một cách, ai cũng nhận 0 cái. 10^6 kẹo cho 10^6 bạn, mỗi bạn ít nhất 1: mỗi bạn đúng một cái. $k \cdot t = 10^{12} > n$ nên in **0**. Nếu tính $k \cdot t$ bằng `int`, tích bị tràn thành $-727\,379\,968 < n$ và chương trình sẽ đi tính một tổ hợp vô nghĩa.



Hình 13.1. *Ví dụ 1, câu hỏi thứ hai: sau khi phát trước, mỗi cách chia 7 cái kẹo còn lại là một cách đặt 2 vạch vào 9 ô.*

CHÚ Ý

k · t có thể tới 10^{12}: hãy so sánh k · t > n bằng `long long`.

Chỉ số lớn nhất cần tra bảng là n − k · t + k − 1, có thể gần **$2 \cdot 10^6$** (khi t = 0, n = k = 10^6). Bảng giai thừa chỉ tới 10^6 là chưa đủ.

Lấy dư sau **mỗi** phép nhân trong fact[a] · ifact[b] · ifact[a − b]: tích ba số cỡ 10^9 tràn `long long`.

HƯỚNG SUY NGHĨ

- Nếu mỗi bạn phải có ít nhất t cái, hãy phát trước cho mỗi bạn t cái. Bài toán còn lại có ràng buộc gì?
- Với 10^5 câu hỏi, đừng tính mỗi C(a, b) bằng một vòng lặp: tính sẵn giai thừa và nghịch đảo giai thừa một lần, rồi trả lời mỗi câu trong O(1).

BÀI 13.2 ĐƯỜNG ĐI TRÊN LƯỚI

★★☆ TRUNG BÌNH

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
DUONGLUOI.CPP	DUONGLUOI.INP	DUONGLUOI.OUT	1 giây · 256 MB

Kiến thức: *Đếm đường đi trên lưới bằng hệ số nhị thức, quy tắc nhân, bù trừ hai tập (Tập 1, Bài 13, 14).*

Thành phố Lưới có các con đường chạy song song với hai trục tọa độ, cắt nhau tại các **giao lộ** có tọa độ nguyên. Robot giao hàng xuất phát ở giao lộ O(0, 0) và cần tới giao lộ T(m, n). Mỗi bước, robot đi từ giao lộ (x, y) sang **(x + 1, y)** (sang phải) hoặc **(x, y + 1)** (lên trên).

Có K giao lộ đang sửa chữa (K ≤ 2), robot **không được đi qua** các giao lộ này. Hãy đếm số đường đi khác nhau của robot từ O tới T, lấy dư cho $10^9 + 7$. Hai đường đi khác nhau nếu dãy giao lộ đi qua của chúng khác nhau.

Dữ liệu vào

Dòng đầu chứa ba số nguyên m, n, K. Mỗi dòng trong K dòng tiếp theo chứa hai số nguyên x, y ($0 \le x \le m$, $0 \le y \le n$) là tọa độ một giao lộ đang sửa chữa. Hai giao lộ sửa chữa (nếu có) khác nhau; chúng có thể trùng O hoặc T.

Kết quả

Một số nguyên là số đường đi, lấy dư cho $10^9 + 7$.

Ràng buộc

$0 \le m, n \le 10^6$; $0 \le K \le 2$.

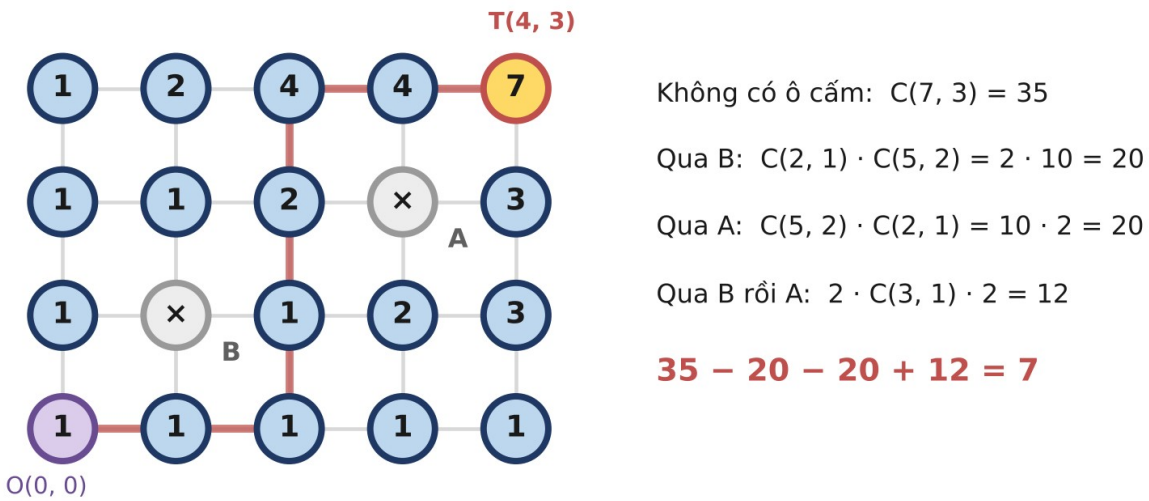
Subtask	Điểm	Ràng buộc thêm
1	20%	m, n ≤ 1000.
2	30%	K ≤ 1.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

DUONGLUOI.INP	DUONGLUOI.OUT	Giải thích
3 2 1 1 1	4	Không có ô cấm thì có C(5, 2) = 10 đường đi. Số đường đi qua (1, 1) là C(2, 1) · C(3, 1) = 2 · 3 = 6: đi từ O tới (1, 1), rồi từ (1, 1) tới (3, 2). Đáp số 10 − 6 = **4**.

DUONGLUOI.INP	DUONGLUOI.OUT	Giải thích
4 3 2 3 2 1 1	7	Hình 13.2: có 35 đường đi tất cả, 20 đường qua B(1, 1) và 20 đường qua A(3, 2). Có 2 · 3 · 2 = 12 đường qua cả B lẫn A đã bị trừ hai lần, phải cộng lại: 35 − 20 − 20 + 12 = **7**.

DUONGLUOI.INP	DUONGLUOI.OUT	Giải thích
2 2 2 1 0 0 1	0	Bước đầu tiên luôn tới (1, 0) hoặc (0, 1), nên đáp số là **0**. Không đường nào qua cả hai ô này (không ô nào nằm "phía dưới bên trái" ô kia): 6 − 3 − 3 + 0 = 0.



Hình 13.2. *Ví dụ 2: số trong mỗi giao lộ là số đường đi từ O tới đó tránh các ô cấm (đúng như quy hoạch động của subtask 1); đường đỏ là một trong 7 đường đi.*

CHÚ Ý

Số đường đi từ (x_1, y_1) tới (x_2, y_2) là $C((x_2 - x_1) + (y_2 - y_1), x_2 - x_1)$ nếu $x_2 \geq x_1$ và $y_2 \geq y_1$, còn lại thì bằng **0**. Quên trường hợp bằng 0 là lỗi hay gặp khi hai ô cấm "không so sánh được" như ở ví dụ 3.

Chỉ số lớn nhất cần tra bảng giai thừa là $m + n \leq 2 \cdot 10^6$.

Phép trừ theo mô-đun có thể cho số âm: in `((ans % MOD) + MOD) % MOD`.

Ô cấm trùng O hoặc T thì đáp số là 0; công thức bù trừ tự cho kết quả này, không cần xử lí riêng.

HƯỚNG SUY NGHĨ

- Một đường đi từ O tới T gồm đúng m bước sang phải và n bước lên trên. Nó được xác định hoàn toàn khi biết những bước nào là bước sang phải.
- Một đường đi qua ô A là ghép của một đường từ O tới A với một đường từ A tới T: dùng quy tắc nhân.
- Với hai ô cấm A, B, hãy dùng nguyên lí bù trừ cho hai tập "đường đi qua A" và "đường đi qua B". Một đường đi qua cả hai ô thì phải qua ô nào trước?

BÀI 13.3 TỔ HỢP KHỔNG LỒ

★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
TOHOPLON.CPP	TOHOPLON.INP	TOHOPLON.OUT	1 giây · 256 MB

Kiến thức: *Định lí Lucas (Tập 1, Bài 13), định lí số dư Trung Hoa (Tập 1, Bài 12).*

Cho số nguyên dương m và Q truy vấn, mỗi truy vấn gồm hai số nguyên n, k. Với mỗi truy vấn, hãy tính C(n, k) mod m, trong đó C(n, k) là số cách chọn k phần tử từ một tập n phần tử (quy ước C(n, k) = 0 khi k > n).

Số n có thể tới 10^{18}, nên C(n, k) có thể có hàng tỉ tỉ chữ số. May mắn là mô-đun m có dạng đặc biệt: m là **tích của các số nguyên tố đôi một khác nhau**, mỗi số không vượt quá 10^5. Chẳng hạn m có thể là 7, 30 = 2 · 3 · 5 hay 9 998 000 099 = 99 989 · 99 991, nhưng không thể là 12 = 2^2 · 3. Riêng m = 1 (tích của không số nào) thì mọi đáp số đều bằng 0.

Dữ liệu vào

Dòng đầu chứa hai số nguyên Q và m. Mỗi dòng trong Q dòng tiếp theo chứa hai số nguyên n, k.

Kết quả

Gồm Q dòng, dòng thứ i là C(n, k) mod m của truy vấn thứ i.

Ràng buộc

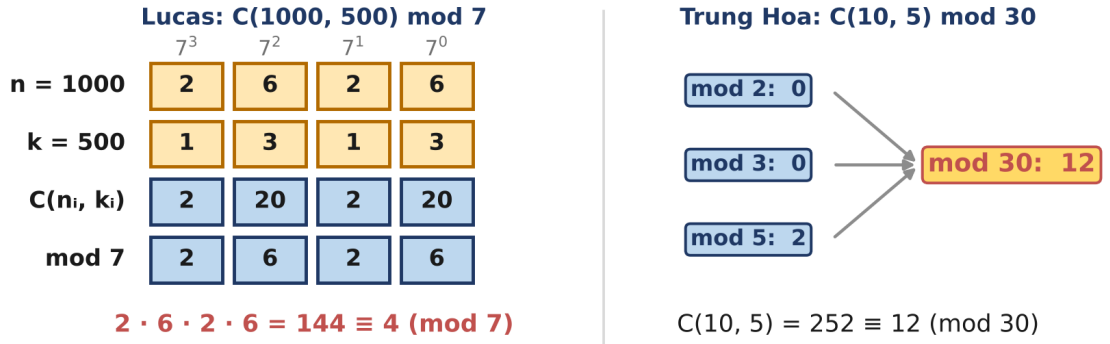
$1 \le Q \le 10^5$; $1 \le m \le 10^{10}$, m là tích các số nguyên tố đôi một khác nhau, mỗi số không vượt quá 10^5; $0 \le n, k \le 10^{18}$.

Subtask	Điểm	Ràng buộc thêm
1	20%	n ≤ 2000.
2	30%	m là số nguyên tố.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

TOHOPLON.INP	TOHOPLON.OUT	Giải thích
3 7 1000 500 10 3 5 7	4 1 0	1000 = $(2626)_7$ và 500 = $(1313)_7$. Theo Lucas, C(1000, 500) ≡ C(2, 1) · C(6, 3) · C(2, 1) · C(6, 3) = 2 · 20 · 2 · 20 = 1600 ≡ **4** (mod 7). C(10, 3) = 120 = 7 · 17 + 1. k > n nên C(5, 7) = 0.

TOHOPLON.INP	TOHOPLON.OUT	Giải thích
3 30 6 3 10 5 1000000000000000000 1	20 12 10	m = 30 = 2 · 3 · 5. C(6, 3) = 20. C(10, 5) = 252 chia cho 2, 3, 5 lần lượt dư 0, 0, 2. Số duy nhất trong [0, 30) có ba số dư ấy là **12**; quả thật 252 = 30 · 8 + 12. C(10^{18}, 1) = 10^{18} ≡ 10 (mod 30).



Hình 13.3. *Hai bước của lời giải: định lí Lucas theo từng ước nguyên tố p của m (trái), rồi ghép các số dư bằng định lí số dư Trung Hoa (phải).*

CHÚ Ý

m không nguyên tố nên k! có thể **không** có nghịch đảo theo mô-đun m; cũng không thể lập bảng giai thừa tới n = 10^{18}. Công thức fact[n] · ifact[k] · ifact[n − k] không dùng trực tiếp được.

Trong định lí Lucas, chỉ cần một chữ số $k_i > n_i$ là cả tích bằng 0 (điều này tự xảy ra khi k > n). Giai thừa theo mô-đun p chỉ cần tới p − 1.

Các hệ số ghép Trung Hoa nhỏ hơn m ≤ 10^{10}; nhân với số dư (< 10^5) vẫn vừa `long long`, nhưng tích hai số cỡ m thì không.

HƯỚNG SUY NGHĨ

- Subtask 2 là định lí Lucas thuần túy: chuẩn bị giai thừa và nghịch đảo giai thừa theo mô-đun p cho 0, 1, ..., p − 1, rồi tách n, k theo cơ số p.
- Nếu m = $p_1 \cdot p_2 \cdot \ldots \cdot p_r$ và em biết C(n, k) mod p_i với mọi i, thì C(n, k) mod m có được xác định duy nhất không? Bài 12 của Tập 1 trả lời câu hỏi này.
- m ≤ 10^{10} có không quá 10 ước nguyên tố (tích 11 số nguyên tố đầu tiên đã là 200 560 490 130). Các hệ số ghép chỉ cần tính một lần cho mọi truy vấn.

BÀI 14 Nguyên lí bù trừ

ÔN NHANH KIẾN THỨC · Tập 1, Bài 14

- Số bội của a trong [1, N] là $\lfloor N / a \rfloor$, trong đoạn [L, R] là $\lfloor R / a \rfloor - \lfloor (L - 1) / a \rfloor$. Số chia hết cho cả a và b là bội của **lcm(a, b)**, không phải của $a \cdot b$.
- **Nguyên lí bù trừ:** $|A_1 \cup ... \cup A_k|$ là tổng theo mọi tập con khác rỗng S của số phần tử của giao các A_i ($i \in S$), mang dấu + khi |S| lẻ, dấu − khi |S| chẵn. Duyệt tập con bằng mặt nạ bit trong $O(2^k)$.
- **Chặn tràn lcm:** $(x / \gcd(x, a)) \cdot a > N$ khi và chỉ khi $x / \gcd(x, a) > \lfloor N / a \rfloor$; khi đó ghi N + 1, mọi số hạng chứa nó đều bằng 0.
- Đếm $x \le N$ nguyên tố cùng nhau với n: bù trừ trên các ước nguyên tố **phân biệt** của n. Hoán vị không điểm cố định: D(0) = 1, D(1) = 0, D(n) = (n − 1)(D(n − 1) + D(n − 2)).

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
14.1	**BỘI SỐ**	★☆☆ CƠ BẢN	Đổi điều kiện "hoặc", "nhưng không" thành công thức bù trừ; chống tràn số.
14.2	**NGUYÊN TỐ CÙNG NHAU**	★★☆ TRUNG BÌNH	Phân tích nhiều số tới 10^{12} đủ nhanh nhờ danh sách số nguyên tố sàng sẵn.
14.3	**TẶNG QUÀ**	★★★ NÂNG CAO	Phối hợp quy tắc nhân, truy hồi D(n) và tổ hợp với n vượt mô-đun.

BÀI 14.1 BỘI SỐ ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`BOISO.CPP`	`BOISO.INP`	`BOISO.OUT`	1 giây · 256 MB

Kiến thức: *Đếm bội trong một đoạn, nguyên lí bù trừ với ba tập, chặn tràn lcm (Tập 1, Bài 3, 14).*

Cho Q câu hỏi. Mỗi câu hỏi gồm năm số nguyên dương L, R, a, b, c. Hãy đếm số các số nguyên x trong đoạn [L, R] thỏa mãn **đồng thời** hai điều kiện:

- x chia hết cho a **hoặc** x chia hết cho b (hoặc cả hai);
- x **không** chia hết cho c.

Dữ liệu vào

Dòng đầu chứa số nguyên Q. Mỗi dòng trong Q dòng tiếp theo chứa năm số nguyên L, R, a, b, c.

Kết quả

Gồm Q dòng, dòng thứ i là số lượng số x của câu hỏi thứ i.

Ràng buộc

$1 \le Q \le 10^5$; $1 \le L \le R \le 10^{18}$; $1 \le a, b, c \le 10^{18}$.

Subtask	Điểm	Ràng buộc thêm
1	30%	$Q \le 10$, $R - L \le 10^5$.
2	30%	$a, b, c \le 10^6$.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

BOISO.INP	BOISO.OUT	Giải thích
2 1 30 2 3 5 10 20 4 6 12	16 3	Hình 14.1: trong [1, 30] có 15 + 10 − 5 = 20 số chia hết cho 2 hoặc 3; bốn số 10, 15, 20, 30 trong đó chia hết cho 5 nên bị loại, còn **16** số. Trong [10, 20], các số chia hết cho 4 hoặc 6 là 12, 16, 18, 20; bỏ 12 vì chia hết cho 12, còn **3** số.

BOISO.INP	BOISO.OUT
2 1 1000000000000000000 1000000007 1000000009 998244353 1 1000000000000000000 4 6 1000000000000000000	1999999982 333333333333333332

Ví dụ 2. a, b, c là ba số nguyên tố khác nhau cỡ 10^9. Có 999 999 993 bội của a và 999 999 991 bội của b; lcm(a, b) ≈ 1,000000016 · 10^{18} > R nên không có bội chung. Mỗi số lcm(a, c) = 998 244 359 987 710 471 và lcm(b, c) = 998 244 361 984 199 177 có đúng một bội trong đoạn. Đáp số 999 999 993 + 999 999 991 − 1 − 1 = **1 999 999 982**. Nhân thẳng a · b hay a · c trong `long long` đều tràn số.

Ở câu hỏi thứ hai, c = 10^{18} chỉ loại đúng số 10^{18} (nó chia hết cho 4): 250 000 000 000 000 000 + 166 666 666 666 666 666 − 83 333 333 333 333 333 − 1 = **333 333 333 333 333 332**.



$$\lfloor 30/2 \rfloor + \lfloor 30/3 \rfloor - \lfloor 30/6 \rfloor - \lfloor 30/10 \rfloor - \lfloor 30/15 \rfloor + \lfloor 30/30 \rfloor = 15 + 10 - 5 - 3 - 2 + 1 = 16$$

Hình 14.1. Ví dụ 1, câu hỏi thứ nhất: ô vàng được đếm, ô xám bị loại vì chia hết cho 5.

CHÚ Ý

Đếm trên đoạn [L, R] bằng hiệu f(R) − f(L − 1), với f(N) là số lượng trên [1, N]. Công thức "⌊(R − L + 1) / a⌋" là **sai**: đoạn [2, 4] có hai bội của 2, nhưng ⌊3 / 2⌋ = 1.

Tích a · b có thể tới 10^{36}, và ngay cả lcm(a, b) cũng có thể vượt xa 10^{18}. Kiểm tra trước khi nhân như Tập 1, Bài 14: lcm vượt R thì số bội của nó bằng 0.

Phần "không chia hết cho c" **không** phải là trừ đi ⌊N / c⌋: chỉ trừ những số **vừa** chia hết cho a hoặc b, **vừa** chia hết cho c.

HƯỚNG SUY NGHĨ

- Gọi A, B, C là tập các bội của a, b, c trong [1, N]. Tập cần đếm là (A ∪ B) bỏ đi C, có |A ∪ B| − |(A ∪ B) ∩ C| phần tử.
- (A ∪ B) ∩ C = (A ∩ C) ∪ (B ∩ C): lại là hợp của hai tập. Giao của hai tập này là gì?

BÀI 14.2	NGUYÊN TỐ CÙNG NHAU	★★☆ TRUNG BÌNH

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
NTCN.CPP	NTCN.INP	NTCN.OUT	1 giây · 256 MB

Kiến thức: *Bù trừ trên các ước nguyên tố, phân tích thừa số nguyên tố, sàng Eratosthenes (Tập 1, Bài 6, 7, 14).*

Hai số nguyên dương được gọi là **nguyên tố cùng nhau** nếu ước chung lớn nhất của chúng bằng 1. Chẳng hạn 8 và 15 nguyên tố cùng nhau, còn 8 và 12 thì không.

Cho Q câu hỏi, mỗi câu hỏi gồm hai số nguyên dương n và N. Hãy đếm số các số nguyên x với $1 \le x \le N$ mà x và n nguyên tố cùng nhau.

Dữ liệu vào

Dòng đầu chứa số nguyên Q. Mỗi dòng trong Q dòng tiếp theo chứa hai số nguyên n, N.

Kết quả

Gồm Q dòng, dòng thứ i là đáp số của câu hỏi thứ i.

Ràng buộc

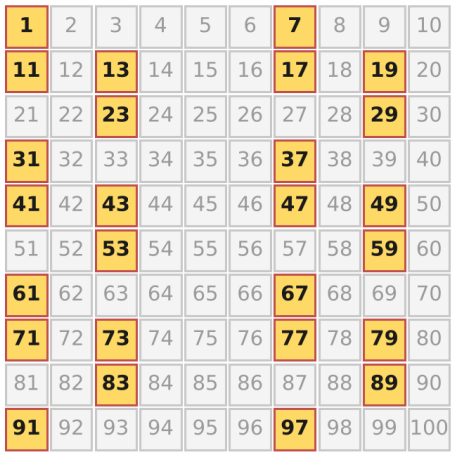
$1 \le Q \le 1000$; $1 \le n, N \le 10^{12}$.

Subtask	Điểm	Ràng buộc thêm
1	30%	$Q \le 10, N \le 10^6$.
2	30%	$Q \le 10$.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

NTCN.INP	NTCN.OUT	Giải thích
3 30 100 12 12 1 5	26 4 5	Hình 14.2: trong [1, 100] có 74 số chia hết cho 2, 3 hoặc 5, nên có 100 − 74 = **26** số nguyên tố cùng nhau với 30. Khi N = n, đáp số chính là φ(n): φ(12) = 4 (các số 1, 5, 7, 11). Mọi số đều nguyên tố cùng nhau với 1.

NTCN.INP	NTCN.OUT	Giải thích
2 999999999989 1000000000000 200560490130 1000000000000	999999999999 152852151382	999 999 999 989 là số nguyên tố; trong $[1, 10^{12}]$ chỉ có chính nó là không nguyên tố cùng nhau với nó. n = 200 560 490 130 = 2 · 3 · 5 · ... · 31 là tích 11 số nguyên tố đầu tiên: phải bù trừ trên $2^{11} - 1 = 2047$ tập con.



1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

Tập con	Dấu	⌊100 / d⌋
{2}	+	50
{3}	+	33
{5}	+	20
{2, 3}	−	16
{2, 5}	−	10
{3, 5}	−	6
{2, 3, 5}	+	3

Có ước chung với 30:

50 + 33 + 20 − 16 − 10 − 6 + 3 = 74

Đáp số: 100 − 74 = 26

Hình 14.2. *Ví dụ 1, câu hỏi thứ nhất: 26 ô vàng là các số nguyên tố cùng nhau với 30; bảng bên phải là phép bù trừ trên các ước nguyên tố 2, 3, 5 (d là tích các số trong tập con).*

CHÚ Ý

Chỉ bù trừ trên các ước nguyên tố **phân biệt**: với $n = 12 = 2^2 \cdot 3$ thì dùng {2, 3}. Thêm "4" vào danh sách sẽ đếm sai.

Phân tích $n \le 10^{12}$ cần thử chia tới $\sqrt{n} = 10^6$. Thử mọi số từ 2 tới 10^6 cho 1000 câu hỏi là 10^9 phép chia 64 bit: quá chậm. Chỉ thử các **số nguyên tố** (có 78 498 số nguyên tố không vượt quá 10^6, sàng sẵn một lần).

Sau khi thử hết các số nguyên tố p với $p^2 \le n$, phần còn lại của n nếu lớn hơn 1 thì là một số nguyên tố, có thể lớn tới 10^{12}. Đừng quên nó.

HƯỚNG SUY NGHĨ

- x không nguyên tố cùng nhau với n khi và chỉ khi x chia hết cho ít nhất một ước nguyên tố của n. Đây đúng là bài BUTRU của Tập 1.
- Một số $n \le 10^{12}$ có nhiều nhất bao nhiêu ước nguyên tố phân biệt? (Tích 12 số nguyên tố đầu tiên là 7 420 738 134 810.) Vậy có bao nhiêu tập con cần duyệt?

BÀI 14.3	TẶNG QUÀ	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
TANGQUA.CPP	**TANGQUA.INP**	**TANGQUA.OUT**	**1 giây · 256 MB**

Kiến thức: *Hoán vị không điểm cố định (Tập 1, Bài 14), tính C(n, k) bằng nhân dần khi n rất lớn (Tập 1, Bài 13).*

Lễ hội cuối năm của một mạng xã hội có n người tham gia, mỗi người gửi một món quà vào kho chung. Sau đó hệ thống trả cho mỗi người **đúng một** món quà, và mỗi món quà được trả cho đúng một người. Một người là **may mắn** nếu nhận lại đúng món quà của chính mình.

Có bao nhiêu cách trả quà mà có **đúng m** người may mắn? Hai cách trả quà khác nhau nếu có một người nhận được hai món quà khác nhau trong hai cách. Hãy in số cách lấy dư cho $10^9 + 7$.

Số người tham gia rất lớn, nhưng ban tổ chức biết rằng số người **không** may mắn, $n - m$, không quá mười triệu.

Dữ liệu vào

Một dòng chứa hai số nguyên n và m.

Kết quả

Một số nguyên là số cách trả quà, lấy dư cho $10^9 + 7$.

Ràng buộc

$1 \le n \le 10^{18}$; $0 \le m \le n$; $n - m \le 10^7$.

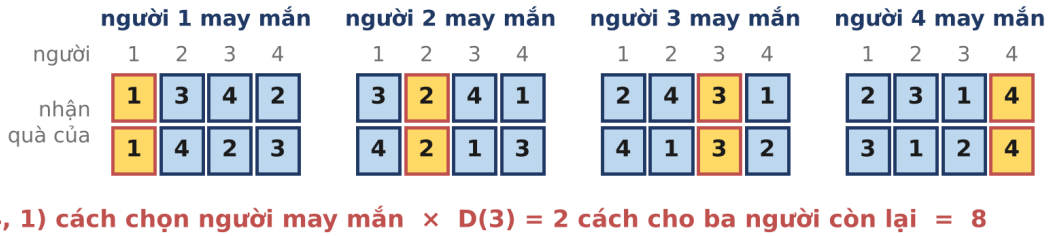
Subtask	Điểm	Ràng buộc thêm
1	20%	$n \le 8$.
2	30%	$n \le 10^6$.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

TANGQUA.INP	TANGQUA.OUT	Giải thích
`4 1`	`8`	Hình 14.3: chọn người may mắn có C(4, 1) = 4 cách; ba người còn lại nhận quà của nhau mà không ai nhận lại quà của mình, có D(3) = 2 cách. Tổng 4 · 2 = **8**.

TANGQUA.INP	TANGQUA.OUT	Giải thích
`5 4`	`0`	Nếu 4 trong 5 người đã nhận lại quà của mình thì món quà còn lại cũng phải về tay người thứ năm. Không thể có **đúng** 4 người may mắn (D(1) = 0).

TANGQUA.INP	TANGQUA.OUT	Giải thích
1000000000000000000 999999999999999990	513687792	C(10^{18}, 10) ≡ 217 822 480 cách chọn 10 người không may mắn, D(10) = 1 334 961 cách trả quà cho họ; tích ≡ **513 687 792**.



Hình 14.3. *Ví dụ 1: tám cách trả quà cho 4 người với đúng một người may mắn (ô vàng). Cột thứ j ứng với người j, số trong ô là chủ của món quà người j nhận.*

CHÚ Ý

Không thể lập bảng giai thừa tới n = 10^{18}. Nhưng C(n, m) = C(n, k) với k = n − m ≤ 10^7, và C(n, k) = n(n − 1)...(n − k + 1) / k! chỉ có k thừa số (Tập 1, Bài 13, Mở rộng 2).

n có thể lớn hơn p = 10^9 + 7: lấy dư **từng thừa số** (n − i) mod p trước khi nhân. Nếu có một thừa số chia hết cho p thì tích bằng 0, và đó là đáp số đúng; công thức vẫn hợp lệ vì k < p nên k! có nghịch đảo.

D(1) = 0: không bao giờ có đúng n − 1 người may mắn. Còn m = n thì có đúng một cách (D(0) = 1).

HƯỚNG SUY NGHĨ

- Chia việc trả quà làm hai bước: chọn những người may mắn, rồi trả quà cho những người còn lại sao cho không ai nhận lại quà của mình.
- Số cách của bước hai chỉ phụ thuộc vào k = n − m và tuân theo một hệ thức truy hồi đơn giản. Bước một thì sao, khi n lớn mà k nhỏ?

BÀI 15 Dãy truy hồi và lũy thừa ma trận

ÔN NHANH KIẾN THỨC · Tập 1, Bài 15

- Dãy truy hồi tuyến tính bậc k: gói k số hạng liên tiếp thành một **vectơ trạng thái**; một bước của dãy là nhân với ma trận đồng hành T cỡ k × k, đi n bước là nhân với T^n.
- Lũy thừa nhanh ma trận tốn $O(k^3 \log n)$. Với Fibonacci, Q = [[1, 1], [1, 0]] và Q^n = $[[F_{n+1}, F_n], [F_n, F_{n-1}]]$.
- Nếu A là ma trận kề thì $A^n[u][v]$ là **số đường đi độ dài đúng n** từ u tới v (được đi lại đỉnh và cạnh cũ).
- Phần tử < m ≤ $10^9 + 7$ thì mỗi tích < $1{,}1 \cdot 10^{18}$: lấy dư sau mỗi phép cộng tích. Ma trận đơn vị khởi tạo bằng `1 % m`.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
15.1	**LÁT GẠCH**	★☆☆ CƠ BẢN	Lập hệ thức truy hồi bằng cách nhìn vào phần cuối của hình.
15.2	**TRIBONACCI**	★★☆ TRUNG BÌNH	Ngắt dãy truy hồi tại các vị trí đặc biệt: lũy thừa ma trận theo từng đoạn.
15.3	**ĐƯỜNG ĐI ĐỘ DÀI K**	★★★ NÂNG CAO	Tính sẵn các lũy thừa A^1, A^2, A^4, ...; nhân vectơ với ma trận thay cho ma trận với ma trận.

BÀI 15.1 LÁT GẠCH ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
LATGACH.CPP	**LATGACH.INP**	**LATGACH.OUT**	**1 giây · 256 MB**

Kiến thức: *Dãy Fibonacci, lũy thừa nhanh ma trận 2 × 2 (Tập 1, Bài 15).*

Hành lang thư viện là một hình chữ nhật rộng 2 mét, dài n mét. Bác thợ lát hành lang bằng các viên gạch cỡ **1 × 2** mét; mỗi viên được đặt **dọc** hoặc **ngang**. Gạch phải phủ kín hành lang, không viên nào chồng lên viên nào hay thò ra ngoài.

Hai cách lát khác nhau nếu có một viên gạch nằm ở vị trí khác nhau. Với mỗi độ dài n, hãy cho biết số cách lát, lấy dư cho $10^9 + 7$.

Dữ liệu vào

Dòng đầu chứa số nguyên T là số hành lang. Mỗi dòng trong T dòng tiếp theo chứa một số nguyên n.

Kết quả

Gồm T dòng, dòng thứ i là số cách lát hành lang thứ i, lấy dư cho $10^9 + 7$.

Ràng buộc

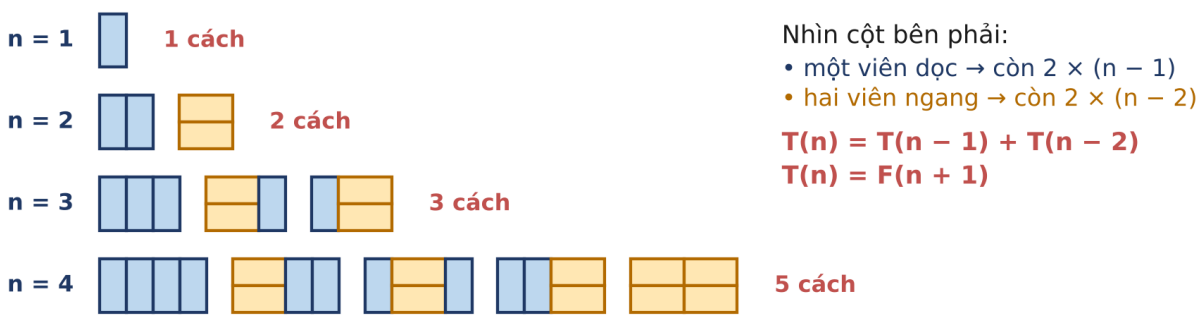
$1 \le T \le 1000$; $1 \le n \le 10^{18}$.

Subtask	Điểm	Ràng buộc thêm
1	20%	$n \le 20$.
2	30%	$n \le 10^6$.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

LATGACH.INP	LATGACH.OUT	Giải thích
3 1 2 4	1 2 5	Hành lang 2 × 1: một viên dọc. Hành lang 2 × 2: hai viên dọc hoặc hai viên ngang. Hành lang 2 × 4: **5** cách (Hình 15.1).

LATGACH.INP	LATGACH.OUT	Giải thích
2 10 1000000000000000000	89 680057396	Số cách lát hành lang 2 × 10 là F_{11} = **89**. Với n = 10^{18}, vòng lặp n bước không thể chạy xong; lũy thừa ma trận chỉ cần khoảng 60 lượt.



Hình 15.1. *Mọi cách lát hành lang 2 × n với n = 1, 2, 3, 4: lần lượt 1, 2, 3, 5 cách.*

CHÚ Ý

Số cách lát hành lang $2 \times n$ là F_{n+1}, không phải F_n: hãy kiểm tra với $n = 1$ và $n = 2$.

Với $T = 1000$, mỗi hành lang tự lũy thừa ma trận (khoảng 120 phép nhân ma trận 2×2): tổng cộng chỉ khoảng 10^6 phép nhân số.

Các phần tử ma trận nhỏ hơn $10^9 + 7$ nên tổng hai tích nhỏ hơn $2{,}1 \cdot 10^{18}$, vẫn vừa `long long`.

HƯỚNG SUY NGHĨ

- Nhìn vào cột cuối cùng của hành lang: nó được phủ bởi một viên dọc, hoặc bởi hai viên ngang nằm chồng lên nhau. Vì sao không thể có đúng một viên ngang ở cột cuối?
- Gọi $T(n)$ là số cách lát. Viết $T(n)$ theo $T(n-1)$ và $T(n-2)$, rồi so sánh với dãy Fibonacci.

BÀI 15.2	TRIBONACCI	★★☆ TRUNG BÌNH

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
TRIBO.CPP	TRIBO.INP	TRIBO.OUT	1 giây · 256 MB

Kiến thức: *Truy hồi bậc 3, ma trận đồng hành, lũy thừa nhanh ma trận (Tập 1, Bài 15).*

Cầu thang lên tháp chuông có n bậc, đánh số từ 1 (thấp nhất) tới n (đỉnh tháp); mặt đất được coi là bậc 0. Mỗi bước, Tí bước lên **1, 2 hoặc 3** bậc.

Đáng tiếc có K bậc đã hỏng: Tí **không được đặt chân** lên các bậc này, nhưng vẫn được bước qua chúng. Tí có bao nhiêu cách đi từ bậc 0 lên đúng bậc n? Hai cách khác nhau nếu dãy các bậc Tí đặt chân khác nhau. Hãy in số cách lấy dư cho $10^9 + 7$.

Dữ liệu vào

Dòng đầu chứa hai số nguyên n và K. Nếu K > 0, dòng thứ hai chứa K số nguyên đôi một khác nhau $b_1, b_2, ..., b_K$ là số thứ tự các bậc hỏng (không nhất thiết theo thứ tự tăng dần).

Kết quả

Một số nguyên là số cách đi, lấy dư cho $10^9 + 7$.

Ràng buộc

$1 \le n \le 10^{18}$; $0 \le K \le 10^4$; $1 \le b_i \le n$.

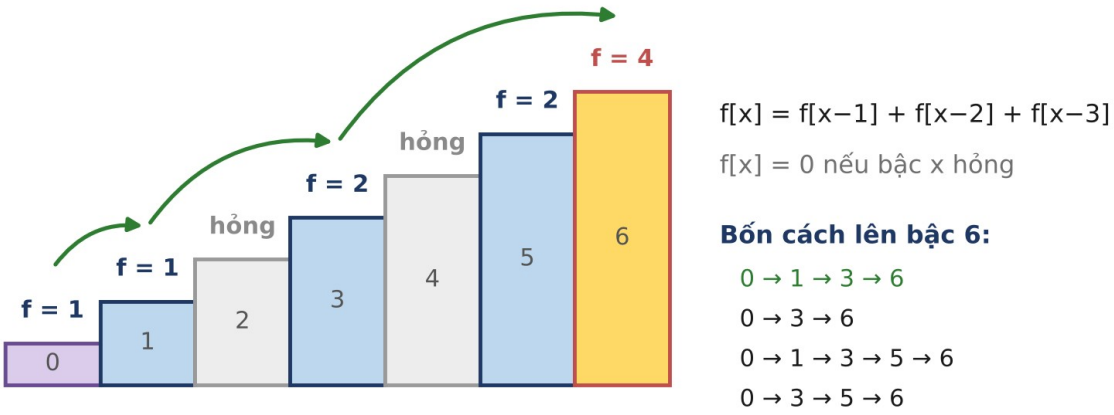
Subtask	Điểm	Ràng buộc thêm
1	30%	$n \le 10^6$.
2	30%	K = 0.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

TRIBO.INP	TRIBO.OUT	Giải thích
5 0	13	Không có bậc hỏng: số cách lên các bậc 0, 1, ..., 5 là 1, 1, 2, 4, 7, **13** (dãy Tribonacci).

TRIBO.INP	TRIBO.OUT	Giải thích
6 2 4 2	4	Bậc 2 và 4 hỏng (cho không theo thứ tự). Bốn cách: 0 → 1 → 3 → 6, 0 → 3 → 6, 0 → 1 → 3 → 5 → 6 và 0 → 3 → 5 → 6 (Hình 15.2).

TRIBO.INP	TRIBO.OUT	Giải thích
1000000000000000000 3 7 5 6	0	Ba bậc liên tiếp 5, 6, 7 đều hỏng, mà mỗi bước chỉ vượt qua được nhiều nhất 2 bậc: không thể lên tới đỉnh, đáp số **0**.



Hình 15.2. *Ví dụ 2: f là số cách lên tới từng bậc. Bậc hỏng có f = 0, nhưng các bậc phía trên vẫn bước qua được nó.*

CHÚ Ý

Các bậc hỏng không cho theo thứ tự: hãy sắp xếp trước.

Nếu chính bậc n bị hỏng thì đáp số là 0.

Với $K = 10^4$, mỗi đoạn giữa hai bậc hỏng là một lần lũy thừa ma trận 3×3 (khoảng 120 phép nhân ma trận, mỗi phép 27 phép nhân số): tổng khoảng $3 \cdot 10^7$ phép nhân, vẫn rất nhanh.

Khởi đầu $f[0] = 1$ (đứng ở mặt đất là một cách), và coi $f[-1] = f[-2] = 0$.

HƯỚNG SUY NGHĨ

- Khi không có bậc hỏng, $f[x] = f[x-1] + f[x-2] + f[x-3]$. Trạng thái $(f[x], f[x-1], f[x-2])$ đi lên một bậc là nhân với ma trận 3×3 nào?
- Giữa hai bậc hỏng liên tiếp, dãy f tuân theo đúng hệ thức Tribonacci. Tại một bậc hỏng b, chỉ cần sửa **một** thành phần của vectơ trạng thái.

BÀI 15.3	ĐƯỜNG ĐI ĐỘ DÀI K	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
DUONGK.CPP	DUONGK.INP	DUONGK.OUT	1 giây · 256 MB

Kiến thức: *Lũy thừa ma trận kề, đếm đường đi độ dài K (Tập 1, Bài 15).*

Một mạng lưới có n trạm, đánh số từ 1 tới n, và m **tuyến một chiều**: tuyến u → v cho phép đi từ trạm u sang trạm v. Có thể có tuyến từ một trạm về chính nó (u = v); không có hai tuyến nào trùng nhau.

Một **hành trình độ dài K** từ s tới t là một dãy K tuyến nối tiếp nhau, bắt đầu ở s và kết thúc ở t; hành trình được phép đi qua một trạm hay một tuyến nhiều lần. Từ mỗi trạm s có đúng một hành trình độ dài 0, kết thúc ở chính s.

Cho Q câu hỏi, mỗi câu hỏi gồm s, t và K: hãy đếm số hành trình độ dài đúng K từ s tới t, lấy dư cho $10^9 + 7$.

Dữ liệu vào

Dòng đầu chứa ba số nguyên n, m, Q. Mỗi dòng trong m dòng tiếp theo chứa hai số nguyên u, v mô tả tuyến u → v. Mỗi dòng trong Q dòng cuối chứa ba số nguyên s, t, K.

Kết quả

Gồm Q dòng, dòng thứ i là số hành trình của câu hỏi thứ i, lấy dư cho $10^9 + 7$.

Ràng buộc

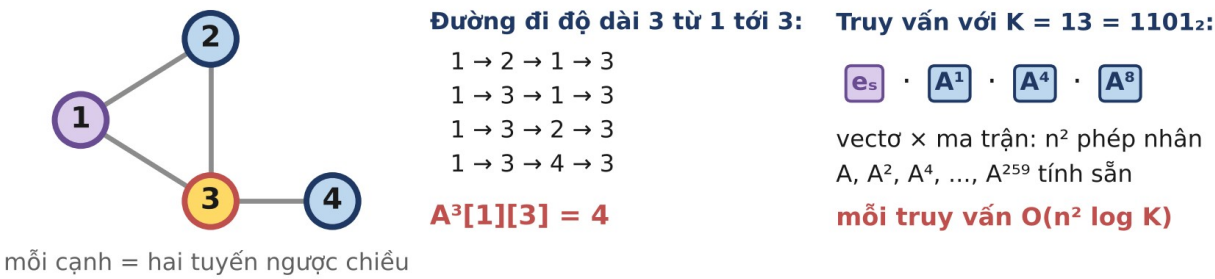
$1 \le n \le 50$; $0 \le m \le n^2$; $1 \le Q \le 1000$; $1 \le u, v, s, t \le n$; $0 \le K \le 10^{18}$.

Subtask	Điểm	Ràng buộc thêm
1	20%	$Q \le 10$, $K \le 1000$.
2	30%	$Q \le 10$.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

DUONGK.INP	DUONGK.OUT	Giải thích
4 8 3 1 2 2 1 1 3 3 1 2 3 3 2 3 4 4 3 1 3 3 1 1 2 4 4 0	4 2 1	Đồ thị ở Hình 15.3, mỗi cạnh ứng với hai tuyến ngược chiều. Có bốn hành trình độ dài 3 từ 1 tới 3. Hai hành trình độ dài 2 từ 1 về 1: 1 → 2 → 1 và 1 → 3 → 1. K = 0: chỉ có hành trình rỗng.

DUONGK.INP	DUONGK.OUT	Giải thích
2 3 2 1 1 1 2 2 1 1 1 10 1 2 1000000000000000000	89 209783453	Ma trận kề là [[1, 1], [1, 0]], chính là ma trận Q của dãy Fibonacci: số hành trình độ dài K từ 1 về 1 là F_{K+1}, với K = 10 là F_{11} = **89**. Số hành trình độ dài 10^{18} từ 1 tới 2 là số Fibonacci thứ 10^{18}, chia $10^9 + 7$ dư 209 783 453.



Hình 15.3. *Trái: đồ thị của ví dụ 1 và bốn hành trình độ dài 3 từ 1 tới 3. Phải: trả lời một câu hỏi bằng cách nhân vectơ hàng với các lũy thừa tính sẵn.*

CHÚ Ý

Nếu mỗi câu hỏi tự lũy thừa ma trận 50 × 50 thì tốn khoảng $120 \cdot 50^3 = 1{,}5 \cdot 10^7$ phép nhân; 1000 câu hỏi là $1{,}5 \cdot 10^{10}$ phép nhân: quá chậm.

Cộng dồn 50 tích, mỗi tích tới khoảng 10^{18}, sẽ tràn `long long`. Hãy lấy dư sau mỗi phép cộng, hoặc dùng `unsigned long long` và trừ đi MOD^2 mỗi khi tổng vượt MOD^2.

K = 0: đáp số là 1 nếu s = t, ngược lại là 0. Tuyến u → u là một hành trình độ dài 1 từ u về u, không phải độ dài 0.

HƯỚNG SUY NGHĨ

- Số hành trình độ dài K từ s tới t là phần tử ở hàng s, cột t của A^K, với A là ma trận kề. Nhưng ta chỉ cần **một hàng** của A^K.
- Tính sẵn A^1, A^2, A^4, ... một lần cho mọi câu hỏi. Nhân một **vectơ** với ma trận rẻ hơn nhân hai ma trận bao nhiêu lần?

BÀI 16 Kĩ thuật chia khối ⌊n / i⌋

ÔN NHANH KIẾN THỨC · Tập 1, Bài 16

- Dãy ⌊n / i⌋ (i = 1..n) có không quá 2√n giá trị khác nhau; khối chứa i kết thúc tại j = ⌊n / ⌊n / i⌋⌋. Nhảy i ← j + 1 cho thuật toán O(√n).
- **Đếm hai lần:** $\Sigma_{x \le n}$ d(x) = $\Sigma_{i \le n}$ ⌊n / i⌋ và σ(1) + ... + σ(n) = Σ i · ⌊n / i⌋; khối [i, j] cùng thương q góp q · (i + j)(j − i + 1) / 2.
- n mod i = n − i · ⌊n / i⌋, nên Σ n mod i quy về Σ i · ⌊n / i⌋.
- **Hai thương cùng lúc:** ⌊n / i⌋ và ⌊m / i⌋ cùng không đổi trên [i, j] với j = min(⌊n / ⌊n / i⌋⌋, ⌊m / ⌊m / i⌋⌋). Tổng có trọng số tới 10^{24}: dùng `__int128` hoặc lấy dư.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
16.1	**TỔNG ƯỚC**	★☆☆ CƠ BẢN	Cộng theo khối với trọng số là cấp số cộng; chống tràn bằng `__int128`.
16.2	**TỔNG DƯ**	★★☆ TRUNG BÌNH	Chia khối trên đoạn [1, min(n, k)]; xử lí riêng phần i > n.
16.3	**HAI THƯƠNG**	★★★ NÂNG CAO	Khai triển thành các tổng Σ 1, Σ i, Σ i^2 trên mỗi khối chung của hai thương.

BÀI 16.1 TỔNG ƯỚC

★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`TONGSIGMA.CPP`	`TONGSIGMA.INP`	`TONGSIGMA.OUT`	1 giây · 256 MB

Kiến thức: *Đếm hai lần, chia khối ⌊n / i⌋ (Tập 1, Bài 16, Mở rộng 2).*

Kí hiệu σ(x) là **tổng các ước dương** của x. Chẳng hạn σ(12) = 1 + 2 + 3 + 4 + 6 + 12 = 28 và σ(7) = 1 + 7 = 8.

Cho số nguyên dương n, hãy tính σ(1) + σ(2) + ... + σ(n), lấy dư cho $10^9 + 7$.

Dữ liệu vào

Một số nguyên n.

Kết quả

Một số nguyên là tổng cần tính, lấy dư cho $10^9 + 7$.

Ràng buộc

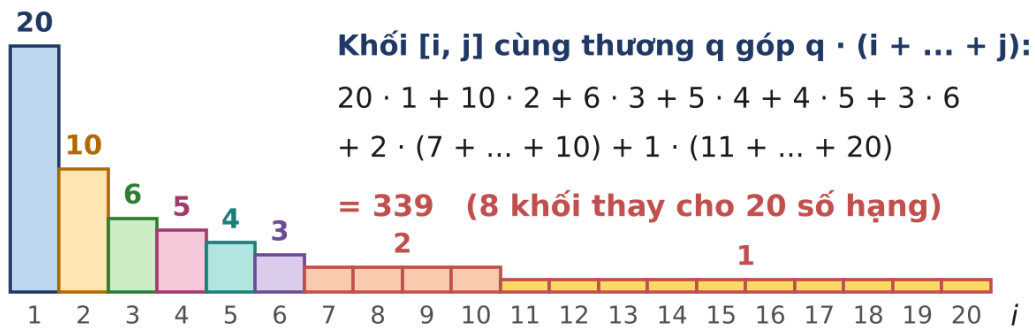
$1 \le n \le 10^{12}$.

Subtask	Điểm	Ràng buộc thêm
1	20%	$n \le 10^4$.
2	30%	$n \le 10^7$.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

TONGSIGMA.INP	TONGSIGMA.OUT	Giải thích
`20`	`339`	Mỗi số i là ước của ⌊20 / i⌋ số trong [1, 20], mỗi lần góp giá trị i. Gom thành 8 khối như Hình 16.1, tổng là **339**.

TONGSIGMA.INP	TONGSIGMA.OUT	Giải thích
`1000000000000`	`146462362`	Tổng thật ≈ $8{,}2 \cdot 10^{23}$, vượt xa `long long`; ta chỉ in số dư.



Hình 16.1. *Bậc thang ⌊20 / i⌋: mỗi khối cùng màu có cùng thương q và góp q nhân với tổng các chỉ số i trong khối.*

CHÚ Ý

Tổng i + ... + j = (i + j)(j − i + 1) / 2 có thể tới khoảng $5 \cdot 10^{23}$: tính bằng `__int128` và chia 2 **trước** khi lấy dư. Lấy dư i, j trước rồi mới chia 2 là sai, vì tích có thể thành số lẻ. Các biến i, j, q tới 10^{12} phải là `long long`.

HƯỚNG SUY NGHĨ

- Đổi thứ tự đếm: thay vì hỏi "x có những ước nào", hãy hỏi "i là ước của những số x nào trong [1, n]", và mỗi lần như vậy i góp bao nhiêu vào tổng.
- Trong một khối [i, j] cùng thương q, phần đóng góp là q · (i + (i + 1) + ... + j).

BÀI 16.2	TỔNG DƯ	★★☆ TRUNG BÌNH

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
TONGDU.CPP	TONGDU.INP	TONGDU.OUT	1 giây · 256 MB

Kiến thức: *Số dư qua thương n mod i = n − i · ⌊n / i⌋, chia khối có chặn trên (Tập 1, Bài 1, 16).*

Cho hai số nguyên n và k. Hãy tính

$$S = (n \bmod 1) + (n \bmod 2) + ... + (n \bmod k)$$

trong đó n mod i là số dư của n khi chia cho i. Vì S có thể rất lớn, hãy in S mod $(10^9 + 7)$.

Dữ liệu vào

Một dòng chứa hai số nguyên n và k.

Kết quả

Một số nguyên là S mod $(10^9 + 7)$.

Ràng buộc

$0 \le n \le 10^{12}$; $1 \le k \le 10^{12}$.

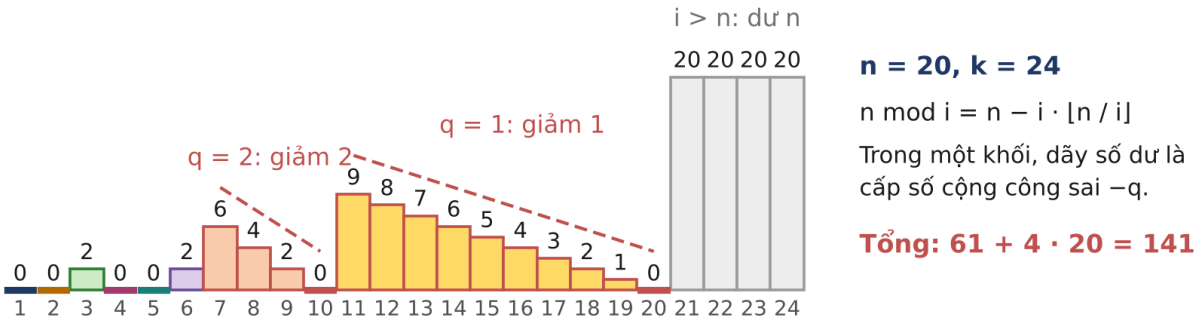
Subtask	Điểm	Ràng buộc thêm
1	30%	$k \le 10^7$.
2	30%	k = n.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

TONGDU.INP	TONGDU.OUT	Giải thích
20 20	61	Giống Tập 1, Bài 16, Mở rộng 3: 20^2 − (σ(1) + ... + σ(20)) = 400 − 339 = **61**.

TONGDU.INP	TONGDU.OUT	Giải thích
5 10	29	Số dư của 5 khi chia cho 1, 2, ..., 10 là 0, 1, 2, 1, 0, 5, 5, 5, 5, 5; tổng **29**. Với i > n thì n mod i = n.

TONGDU.INP	TONGDU.OUT	Giải thích
1000000000000 1000000000000	902537645	Tổng thật xấp xỉ $1{,}78 \cdot 10^{23}$.



Hình 16.2. *n mod i với n = 20, k = 24. Trong mỗi khối cùng thương q, các số dư giảm đều q đơn vị; với i > n, số dư luôn bằng n.*

CHÚ Ý

k có thể lớn hơn n. Với i > n thì ⌊n / i⌋ = 0 và n mod i = n. Nếu duyệt khối tới tận k, công thức j = ⌊n / q⌋ gặp q = 0 và **chia cho 0**.

n = 0: mọi số dư đều bằng 0.

n · k tới 10^{24}: lấy dư n và k trước khi nhân. Phép trừ theo mô-đun cần cộng thêm MOD để không ra số âm.

HƯỚNG SUY NGHĨ

- Viết n mod i = n − i · ⌊n / i⌋. Tổng các số hạng n tính được ngay; phần còn lại giống bài TỔNG ƯỚC.
- Khối không được vượt quá k: điểm cuối khối là min(⌊n / q⌋, k).

BÀI 16.3	HAI THƯƠNG	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
HAITHUONG.CPP	HAITHUONG.INP	HAITHUONG.OUT	1 giây · 256 MB

Kiến thức: *Chia khối theo hai thương đồng thời, tổng $1 + 2 + \ldots + r$ và $1^2 + 2^2 + \ldots + r^2$ theo mô-đun (Tập 1, Bài 16, Mở rộng 4).*

Bài TỔNG DƯ cộng các số dư của **một** số. Bài này làm việc với **hai** số cùng lúc: cho hai số nguyên dương n và m, hãy tính

$$S = \sum_{i=1..\min(n,m)} (n \bmod i) \cdot (m \bmod i)$$

và in S mod $(10^9 + 7)$.

Dữ liệu vào

Một dòng chứa hai số nguyên n và m.

Kết quả

Một số nguyên là S mod $(10^9 + 7)$.

Ràng buộc

$1 \le n, m \le 10^{12}$.

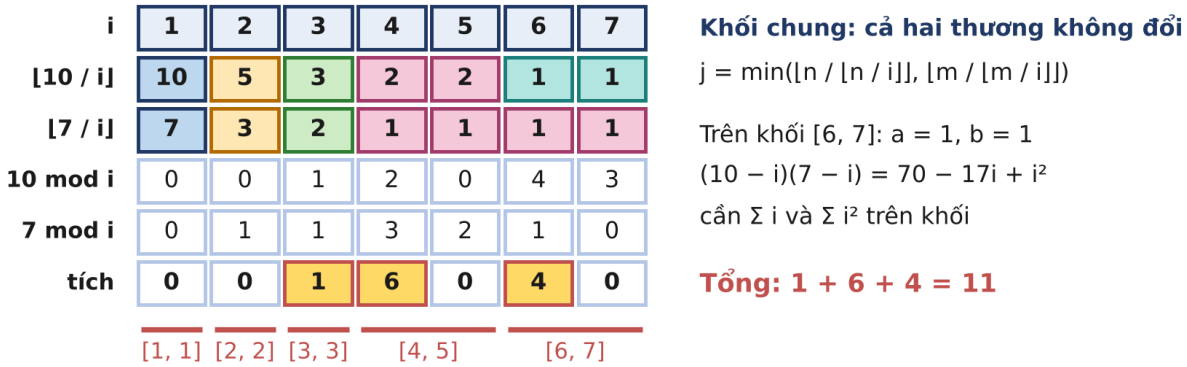
Subtask	Điểm	Ràng buộc thêm
1	30%	$n, m \le 10^7$.
2	30%	n = m.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

HAITHUONG.INP	HAITHUONG.OUT	Giải thích
`10 7`	`11`	Hình 16.3: chỉ có ba tích khác 0 là 1 · 1 (i = 3), 2 · 3 (i = 4) và 4 · 1 (i = 6); S = 1 + 6 + 4 = **11**.

HAITHUONG.INP	HAITHUONG.OUT	Giải thích
`1 5`	`0`	min(n, m) = 1, và mọi số chia cho 1 đều dư 0.

HAITHUONG.INP	HAITHUONG.OUT	Giải thích
`1000000000000 1000000000000`	`498452477`	$n = m = 10^{12}$: S là tổng bình phương các số dư; có khoảng $4 \cdot 10^6$ khối.



Hình 16.3. *$n = 10$, $m = 7$. Mỗi thương được tô theo khối của riêng nó; khối chung (gạch đỏ) là giao của hai cách chia, trên đó cả $\lfloor n / i \rfloor$ lẫn $\lfloor m / i \rfloor$ đều không đổi.*

CHÚ Ý

Số khối chung không quá số khối của n cộng số khối của m, tức không quá $2\sqrt{n} + 2\sqrt{m} \le 4 \cdot 10^6$.

$1^2 + 2^2 + \ldots + r^2 = r(r+1)(2r+1)/6$; với $r = 10^{12}$ tích này cỡ $2 \cdot 10^{36}$, vẫn vừa `__int128` (tới khoảng $1{,}7 \cdot 10^{38}$). Chia 6 trước rồi mới lấy dư, hoặc nhân với nghịch đảo của 6.

Các đại lượng n, m, $\lfloor n / i \rfloor$, $\lfloor m / i \rfloor$ đều tới 10^{12}: lấy dư từng đại lượng trước khi nhân chúng với nhau.

i chỉ chạy tới min(n, m): nếu chạy tới max(n, m), khối của số nhỏ hơn gặp thương bằng 0 và phép chia cho 0.

HƯỚNG SUY NGHĨ

- Trên một đoạn mà $a = \lfloor n / i \rfloor$ và $b = \lfloor m / i \rfloor$ cùng không đổi, $(n \bmod i)(m \bmod i) = (n - a \cdot i)(m - b \cdot i)$ là một đa thức bậc hai theo i.
- Khai triển đa thức ấy ra, em cần ba tổng trên mỗi khối: số phần tử, tổng các i và tổng các i^2.

CHƯƠNG 5. KĨ THUẬT SỐ HỌC TRONG LẬP TRÌNH

*Chương cuối rèn luyện những "mẹo nghề" khi đem số học vào máy tính: tự xây số nguyên lớn khi kết quả dài hàng vạn chữ số, đổi cơ số và đếm bit theo cột, biến bài toán đếm đoạn con thành đếm cặp tiền tố cùng số dư, và dùng ƯCLN cùng định lí Pick để đếm hàng tỉ điểm nguyên mà không duyệt điểm nào. Kẻ thù lớn nhất của chương là **tràn số**: nhiều đáp số vượt 2^{64}, và nhiều phép tính trung gian vượt* `long long` *dù kết quả cuối cùng thì không.*

TRONG CHƯƠNG NÀY

Bài 17. Số nguyên lớn
- 17.1 TỔNG LỚN — ★☆☆ CƠ BẢN
- 17.2 LŨY THỪA ĐẦY ĐỦ — ★★☆ TRUNG BÌNH
- 17.3 TÍCH LỚN — ★★★ NÂNG CAO

Bài 18. Hệ cơ số và biểu diễn nhị phân
- 18.1 ĐỔI CƠ SỐ — ★☆☆ CƠ BẢN
- 18.2 BIT TRONG ĐOẠN — ★★☆ TRUNG BÌNH
- 18.3 SỐ CÓ K BIT 1 — ★★★ NÂNG CAO

Bài 19. Tổng tiền tố và số dư
- 19.1 TỔNG BẰNG KHÔNG — ★☆☆ CƠ BẢN
- 19.2 ĐOẠN DÀI NHẤT — ★★☆ TRUNG BÌNH
- 19.3 HÌNH CHỮ NHẬT CHIA HẾT — ★★★ NÂNG CAO

Bài 20. Điểm nguyên và hình học số nguyên
- 20.1 TRỒNG CÂY — ★☆☆ CƠ BẢN
- 20.2 ĐA GIÁC LƯỚI — ★★☆ TRUNG BÌNH
- 20.3 TAM GIÁC NGUYÊN — ★★★ NÂNG CAO

LỜI KHUYÊN CHO CHƯƠNG NÀY

- Trước khi code, ước lượng độ lớn của mọi đại lượng trung gian (tích, tổng tiền tố, 2S, số nhớ...) để chọn đúng `long long`, `__int128` hay số lớn.
- Với số lớn, viết riêng từng hàm nhỏ (so sánh, cộng, trừ, nhân, in) và thử từng hàm với các test góc: số 0, số âm, hai số bằng nhau, kết quả ngắn hơn số hạng.
- Đối chiếu lời giải nhanh với một chương trình vét cạn trên hàng trăm test nhỏ ngẫu nhiên; với bài hình học, vẽ hình trên giấy kẻ ô.

BÀI 17 Số nguyên lớn

ÔN NHANH KIẾN THỨC · Tập 1, Bài 17

- **Trần của các kiểu số:** `long long` tới khoảng $9{,}2 \cdot 10^{18}$ (19 chữ số), `unsigned long long` tới $1{,}8 \cdot 10^{19}$, `__int128` tới $1{,}7 \cdot 10^{38}$. Vượt trần thì máy lặng lẽ cắt bỏ các bit thừa, không báo lỗi.
- **Số lớn = mảng chữ số đảo ngược:** a[i] là hệ số của B^i. Cộng, trừ, nhân đi từ hàng thấp lên; chia cho số nhỏ đi từ hàng cao xuống (sơ đồ Horner). So sánh: số dài hơn thì lớn hơn, dài bằng nhau thì so như hai xâu.
- **Nhân số lớn với số nhỏ x:** t = a[i] · x + c, giữ t mod B, nhớ $\lfloor t / B \rfloor$; số nhớ luôn nhỏ hơn x. **Nhân hai số lớn:** cộng dồn mọi tích vào c[i + j] rồi dồn nhớ một lượt, $O(L_1 \cdot L_2)$.
- **Cơ số 10^9** nhanh gần 9 lần cơ số 10; khi in, ô cao nhất in thường, các ô sau in đủ 9 chữ số bằng `%09lld`.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
17.1	**TỔNG LỚN**	★☆☆ CƠ BẢN	Cộng, trừ số lớn; xử lí dấu, số 0 thừa và kết quả bằng 0.
17.2	**LŨY THỪA ĐẦY ĐỦ**	★★☆ TRUNG BÌNH	Nhân số lớn với số nhỏ trong cơ số 10^9; in đúng định dạng.
17.3	**TÍCH LỚN**	★★★ NÂNG CAO	Nhân hai số lớn $O(L_1 \cdot L_2)$; chọn cơ số vừa nhanh vừa không tràn.

BÀI 17.1 TỔNG LỚN ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`TONGLON.CPP`	`TONGLON.INP`	`TONGLON.OUT`	1 giây · 256 MB

Kiến thức: *Cộng, trừ và so sánh hai số lớn (Tập 1, Bài 17, Mở rộng 1).*

Trong một trò chơi trực tuyến, điểm của người chơi tăng giảm rất nhanh và có thể trở thành những con số khổng lồ, dài tới hàng trăm nghìn chữ số. Điểm cũng có thể **âm** khi người chơi đang nợ điểm.

Hiện tại An có a điểm. Sau một ván đấu, An nhận thêm b điểm (b < 0 nghĩa là An bị trừ |b| điểm). Hãy cho biết số điểm của An sau ván đấu, tức giá trị của a + b.

Dữ liệu vào

Dòng đầu chứa số nguyên a, dòng thứ hai chứa số nguyên b. Mỗi số được viết trong hệ thập phân, số âm có dấu `-` đứng ngay trước chữ số đầu tiên; không có chữ số 0 thừa ở đầu, và số 0 được viết là `0` (không có `-0`).

Kết quả

Giá trị của a + b, viết theo cùng quy cách: không có chữ số 0 thừa ở đầu, không in `-0`.

Ràng buộc

Mỗi số a, b có không quá 10^5 chữ số (không kể dấu).

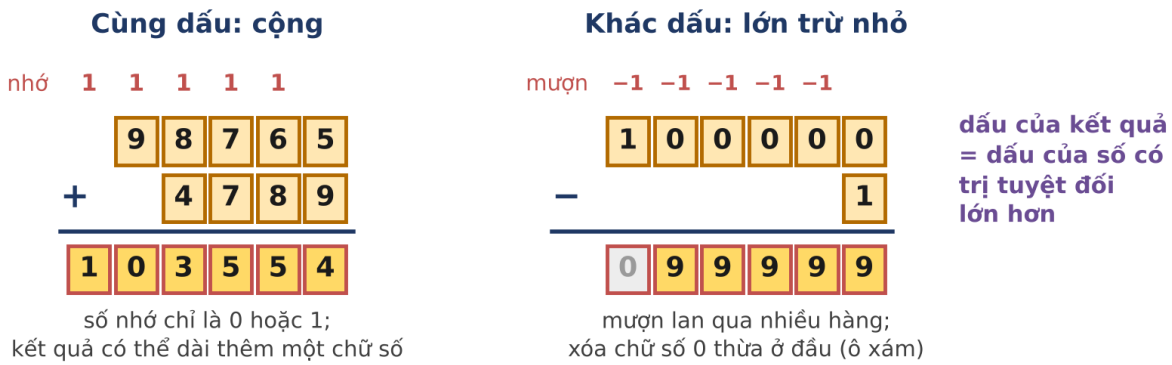
Subtask	Điểm	Ràng buộc thêm
1	20%	$\|a\|, \|b\| \le 10^{18}$.
2	40%	$a \ge 0, b \ge 0$.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

TONGLON.INP	TONGLON.OUT	Giải thích
98765 4789	103554	Cộng từ hàng đơn vị: 5 + 9 = 14 (viết 4, nhớ 1), 6 + 8 + 1 = 15, 7 + 7 + 1 = 15, 8 + 4 + 1 = 13, 9 + 0 + 1 = 10, còn nhớ 1: **103554**, dài hơn cả hai số hạng.

TONGLON.INP	TONGLON.OUT	Giải thích
100000000000000000000 -1	99999999999999999999	$10^{20} + (-1) = 10^{20} - 1$. Hai số khác dấu nên lấy số có trị tuyệt đối lớn hơn trừ số kia. Hàng đơn vị 0 − 1 phải mượn, và việc mượn lan qua cả 20 hàng: kết quả là 20 chữ số 9, ngắn hơn số bị trừ.

TONGLON.INP	TONGLON.OUT	Giải thích
-2026 2026	0	Hai số đối nhau: tổng là **0**, phải in `0` chứ không phải `-0` hay `0000`.



Hình 17.1. *Trái: hai số cùng dấu thì cộng hai trị tuyệt đối, số nhớ chỉ là 0 hoặc 1. Phải: hai số khác dấu thì lấy số lớn trừ số nhỏ; số mượn có thể lan qua nhiều hàng và để lại các chữ số 0 thừa ở đầu.*

CHÚ Ý

Với $|a|, |b| \le 10^{18}$, tổng có thể tới $2 \cdot 10^{18}$: vẫn vừa `long long`. Ngoài subtask 1 thì phải đọc a, b vào **xâu**.

So sánh trị tuyệt đối của hai số: số nào nhiều chữ số hơn thì lớn hơn; chỉ khi dài bằng nhau mới so sánh như hai xâu. So sánh từ điển hai xâu khác độ dài sẽ sai: `"9" > "10"`.

Sau phép trừ, 100000 − 99999 cho ra `00001`: phải xóa các chữ số 0 thừa nhưng giữ lại ít nhất một chữ số. Khi a = −b, kết quả là `0` và không mang dấu trừ.

HƯỚNG SUY NGHĨ

- Tách dấu ra khỏi trị tuyệt đối. Nếu a, b cùng dấu: cộng hai trị tuyệt đối, giữ dấu chung. Nếu khác dấu: trị tuyệt đối của kết quả là hiệu, còn dấu là dấu của số có trị tuyệt đối lớn hơn.
- Viết ba hàm nhỏ trên xâu chữ số: so sánh, cộng, trừ (số lớn trừ số nhỏ). Đi từ cuối xâu (hàng đơn vị) về đầu xâu, cuối cùng đảo ngược kết quả.

BÀI 17.2	LŨY THỪA ĐẦY ĐỦ	★★☆ TRUNG BÌNH	
Chương trình **LUYTHUALON.CPP**	Dữ liệu vào **LUYTHUALON.INP**	Kết quả **LUYTHUALON.OUT**	Giới hạn **1 giây · 256 MB**

Kiến thức: *Nhân số lớn với số nhỏ, cơ số 10^9 (Tập 1, Bài 17).*

Máy tính bỏ túi của Bình chỉ hiện được $3^{40} \approx 1{,}215766546 \cdot 10^{19}$: mười chữ số đầu, các chữ số còn lại đã bị làm tròn mất. Bình muốn thấy **tất cả** các chữ số.

Cho hai số tự nhiên a và b, hãy in ra giá trị chính xác của a^b (quy ước $0^0 = 1$).

Dữ liệu vào

Một dòng chứa hai số nguyên a và b.

Kết quả

Giá trị chính xác của a^b trong hệ thập phân, không có chữ số 0 thừa ở đầu.

Ràng buộc

$0 \le a \le 10^9$; $0 \le b \le 10^4$. Kết quả có thể có tới 90 001 chữ số.

Subtask	Điểm	Ràng buộc thêm
1	20%	$a^b \le 10^{18}$.
2	30%	$a \le 1000$, $b \le 3000$.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

LUYTHUALON.INP	LUYTHUALON.OUT	Giải thích
`3 40`	`12157665459056928801`	3^{40} có 20 chữ số, vượt `long long`. Trong cơ số 10^9 nó được lưu thành ba ô [56928801, 157665459, 12]; ô thấp nhất chỉ có 8 chữ số nên khi in phải thêm một chữ số 0 đằng trước.

LUYTHUALON.INP	LUYTHUALON.OUT	Giải thích
`1000000000 2`	`1000000000000000000`	$(10^9)^2 = 10^{18}$ được lưu thành [0, 0, 1]. Nếu in mỗi ô bằng `%lld`, em sẽ được `100` thay vì số 1 với 18 chữ số 0 phía sau.

LUYTHUALON.INP	LUYTHUALON.OUT	Giải thích
`0 0`	`1`	Quy ước $0^0 = 1$.

Nhân d = 123456789² = [750190521, 15241578] với x = 123456789 (cơ số 10^9):

i	d[i] cũ	t = d[i] · x + c	d[i] mới	c mới
0	750190521	92 616 112 860 897 069	**860897069**	**92616112**
1	15241578	1 881 676 371 789 154	**371789154**	**1881676**
2 (mới)	—	c còn lại	**1881676**	**0**

In 3^{40} = [56928801, 157665459, 12]: ô cao nhất in thường, các ô sau in đủ 9 chữ số

12	157665459	056928801	→ 12157665459056928801
`%lld`	`%09lld`	`%09lld`	(in "%lld" cả ba ô: mất chữ số 0, sai)

Hình 17.2. *Trên: một lần nhân với số nhỏ x trong cơ số 10^9; số nhớ c luôn nhỏ hơn x, còn t có thể vượt 10^{18} nên phải dùng* `long long`*. Dưới: in kết quả, mọi ô sau ô cao nhất phải đủ 9 chữ số.*

CHÚ Ý

Với a = 10^9, b = 10^4, kết quả có 90 001 chữ số: không kiểu số nào chứa nổi, kể cả `__int128` (39 chữ số).

Trong cơ số 10^9 với a tới 10^9, t = d[i] · a + c có thể vượt 10^{18} (nhưng nhỏ hơn $10^{18} + 10^9$): phải tính bằng `long long`, không được dùng `int` cho d[i] rồi nhân.

Cơ số 10 cần khoảng $4{,}5 \cdot 10^8$ phép nhân với a = 10^9, b = 10^4: quá sát giới hạn thời gian. Cơ số 10^9 chỉ cần khoảng $5 \cdot 10^7$.

Đừng quên a = 0: kết quả là `0` khi b > 0 và `1` khi b = 0.

HƯỚNG SUY NGHĨ

- Bắt đầu từ số lớn 1 (mảng một ô), nhân b lần với số nhỏ a, mỗi lần đúng như phép nhân số lớn với số nhỏ trong bài GIAITHUA của Tập 1.
- Vì số nhớ luôn nhỏ hơn a ≤ 10^9, sau mỗi lần nhân chỉ cần thêm nhiều nhất **một** ô mới.

BÀI 17.3	TÍCH LỚN	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`TICHLON.CPP`	`TICHLON.INP`	`TICHLON.OUT`	**1 giây · 256 MB**

Kiến thức: *Nhân hai số lớn bằng cộng dồn c[i + j], chọn cơ số chống tràn (Tập 1, Bài 17, Mở rộng 2).*

Thầy giáo đố cả lớp: "Ai viết được chương trình nhân hai số **năm mươi nghìn chữ số** trong vòng một giây?" Hai số có thể âm.

Cho hai số nguyên a và b, hãy tính tích a · b.

Dữ liệu vào

Dòng đầu chứa số nguyên a, dòng thứ hai chứa số nguyên b, viết theo quy cách như bài TONGLON: số âm có dấu `-` đứng trước, không có chữ số 0 thừa ở đầu, không có `-0`.

Kết quả

Giá trị của a · b, không có chữ số 0 thừa ở đầu, không in `-0`.

Ràng buộc

Mỗi số a, b có không quá $5 \cdot 10^4$ chữ số (không kể dấu).

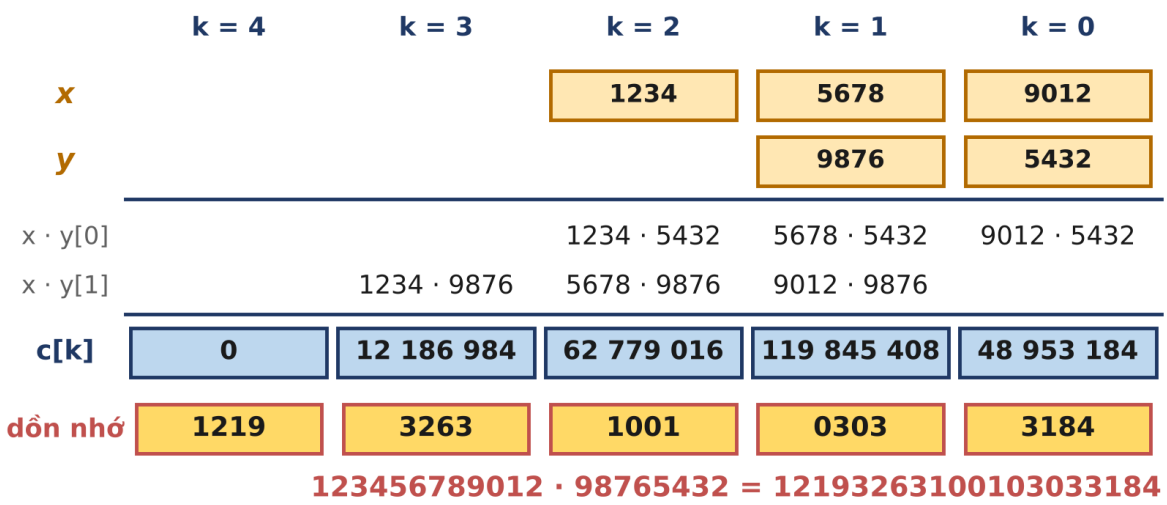
Subtask	Điểm	Ràng buộc thêm
1	20%	$\lvert a\rvert, \lvert b\rvert \le 10^{18}$.
2	30%	a, b có không quá 5000 chữ số.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

TICHLON.INP	TICHLON.OUT	Giải thích
`123` `45`	`5535`	Nhân "trên giấy" trong cơ số 10: c[0] = 3 · 5 = 15, c[1] = 2 · 5 + 3 · 4 = 22, c[2] = 1 · 5 + 2 · 4 = 13, c[3] = 1 · 4 = 4. Dồn nhớ một lượt: **5535**.

TICHLON.INP	TICHLON.OUT	Giải thích
`-99999999` `99999999`	`-9999999800000001`	Trong cơ số 10^4, 99999999 = [9999, 9999]. c[0] = 9999^2 = 99 980 001, c[1] = 199 960 002, c[2] = 99 980 001. Dồn nhớ được [0001, 0000, 9998, 9999]. Hai số khác dấu nên tích âm.

TICHLON.INP	TICHLON.OUT	Giải thích
`0` `-12345`	`0`	Tích bằng 0: in `0`, không in `-0`.



Hình 17.3. Nhân hai số trong cơ số 10^4: tích x[i] · y[j] được cộng dồn vào cột k = i + j; chỉ sau khi cộng xong mới dồn nhớ một lượt từ phải sang trái.

CHÚ Ý

Cơ số 10: mỗi số có $5 \cdot 10^4$ ô, tức $2{,}5 \cdot 10^9$ phép nhân: quá chậm. Cơ số 10^4: mỗi số còn 12 500 ô, tức khoảng $1{,}56 \cdot 10^8$ phép nhân.

Vì sao không dùng cơ số 10^9? Mỗi tích hai ô gần 10^{18}, cộng dồn hàng nghìn tích vào c[k] sẽ **tràn** `long long`. Với cơ số 10^4, $c[k] \le 12\,500 \cdot 9999^2 \approx 1{,}25 \cdot 10^{12}$: an toàn.

Khi in, các ô sau ô cao nhất phải đủ 4 chữ số (`%04lld`). Dấu trừ chỉ in khi đúng một số âm **và** tích khác 0.

HƯỚNG SUY NGHĨ

- Dấu của tích tính riêng, phần còn lại là nhân hai trị tuyệt đối.
- Cắt mỗi xâu thành các khối 4 chữ số tính từ **cuối** xâu; khối ở đầu xâu có thể ngắn hơn 4 chữ số.
- Cộng dồn mọi tích x[i] · y[j] vào c[i + j] trước, rồi dồn nhớ một lần duy nhất ở cuối. Vòng lặp trong càng đơn giản thì trình biên dịch tối ưu càng tốt.

BÀI 18 Hệ cơ số và biểu diễn nhị phân

ÔN NHANH KIẾN THỨC · Tập 1, Bài 18

- **Cơ số b:** $N = (d_k \dots d_1 d_0)_b$. Chữ số cuối $d_0 = N \bmod b$, bỏ chữ số cuối bằng $\lfloor N / b \rfloor$: chia liên tiếp rồi **đọc ngược**. Chiều ngược lại dùng sơ đồ Horner $N \leftarrow N \cdot b + d_i$.
- **Phép toán bit:** bit k của n là `(n >> k) & 1`; `n & (n - 1)` xóa bit 1 thấp nhất; dùng `1LL << k` cho số 64 bit; `__builtin_popcountll` đếm số bit 1.
- **Đếm theo cột:** cột bit k của dãy 0, 1, 2, ... tuần hoàn chu kì 2^{k+1} (2^k số 0 rồi 2^k số 1). Tổng bit 1 của 0..n bằng $\Sigma_k [q \cdot 2^k + \max(0, r - 2^k)]$ với q, r là thương và dư của n + 1 cho 2^{k+1}: O(log n).
- Tổng số bit 1 của 1..10^{18} là 29 761 222 783 429 247 000 > 2^{64}: cộng dồn bằng `unsigned __int128` và tự viết hàm in.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
18.1	**ĐỔI CƠ SỐ**	★☆☆ CƠ BẢN	Đổi qua lại giữa hai cơ số bất kì; xử lí chữ cái, số 0 và chữ số 0 thừa.
18.2	**BIT TRONG ĐOẠN**	★★☆ TRUNG BÌNH	Trả lời truy vấn đoạn bằng hiệu hai "tiền tố"; xử lí đáp số vượt 2^{64}.
18.3	**SỐ CÓ K BIT 1**	★★★ NÂNG CAO	Đếm số có tính chất về chữ số không vượt quá n; tìm số thứ m theo thứ tự.

BÀI 18.1 ĐỔI CƠ SỐ

★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`DOICOSO.CPP`	`DOICOSO.INP`	`DOICOSO.OUT`	1 giây · 256 MB

Kiến thức: *Sơ đồ Horner, chia liên tiếp để đổi cơ số (Tập 1, Bài 18).*

Trong cơ số b ($2 \le b \le 36$), người ta dùng các chữ số 0, 1, ..., 9 rồi tới các chữ cái in hoa A = 10, B = 11, ..., Z = 35. Chẳng hạn $(7EA)_{16} = 7 \cdot 16^2 + 14 \cdot 16 + 10 = 2026$.

Em nhận được Q yêu cầu. Mỗi yêu cầu cho một số tự nhiên S viết trong cơ số a, hãy viết lại số đó trong cơ số b.

Dữ liệu vào

Dòng đầu chứa số nguyên Q. Mỗi dòng trong Q dòng tiếp theo chứa hai số nguyên a, b và một xâu S gồm các kí tự 0–9, A–Z, là một số viết trong cơ số a (mọi chữ số của S đều nhỏ hơn a). Xâu S **có thể có các chữ số 0 ở đầu**.

Kết quả

Gồm Q dòng, dòng thứ i là số của yêu cầu thứ i viết trong cơ số b, dùng chữ in hoa, không có chữ số 0 thừa ở đầu (số 0 được viết là `0`).

Ràng buộc

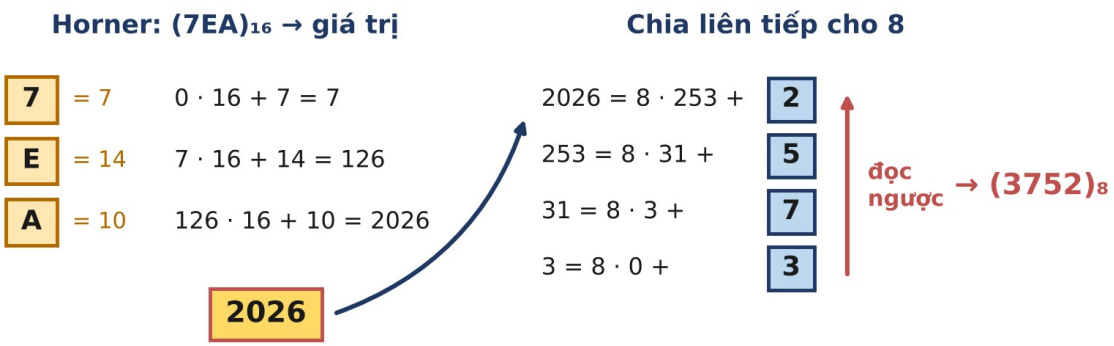
$1 \le Q \le 10^5$; $2 \le a, b \le 36$; S dài không quá 100 kí tự và có giá trị không vượt quá 10^{18}.

Subtask	Điểm	Ràng buộc thêm
1	30%	b = 10.
2	30%	a = 10 và S không có chữ số 0 ở đầu.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

DOICOSO.INP	DOICOSO.OUT	Giải thích
3 16 8 7EA 10 2 2026 36 10 ZZ	3752 11111101010 1295	$(7EA)_{16} = 2026 = 3 \cdot 512 + 7 \cdot 64 + 5 \cdot 8 + 2 = (3752)_8$. $2026 = 1024 + 512 + 256 + 128 + 64 + 32 + 8 + 2$. $(ZZ)_{36} = 35 \cdot 36 + 35 = 1295$.

DOICOSO.INP	DOICOSO.OUT	Giải thích
3 2 16 0000 7 7 00123 10 36 1000000000000000000	0 123 7LIEEXZX4KXS	"0000" là số 0: in `0`. Cùng cơ số: chỉ cần bỏ các chữ số 0 ở đầu. 10^{18} viết trong cơ số 36 có 12 chữ số.



Hình 18.1. *Đổi $(7EA)_{16}$ sang cơ số 8: Horner cho giá trị 2026, chia liên tiếp cho 8 rồi đọc ngược các số dư.*

CHÚ Ý

Giá trị tới 10^{18} vừa `long long`. Trong sơ đồ Horner, mọi giá trị trung gian đều không vượt quá giá trị cuối cùng nên không tràn. Còn nếu cộng $d_i \cdot a^i$ thì với xâu dài 100 kí tự (nhiều chữ số 0 ở đầu), lũy thừa a^i **tràn** dù chữ số tương ứng bằng 0.

Số 0 phải in ra `0`: vòng `while (v > 0)` sẽ in xâu rỗng; hãy dùng `do ... while`.

Đổi kí tự sang giá trị: `c - '0'` với chữ số, `c - 'A' + 10` với chữ cái; chiều ngược lại tra xâu `"0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"`.

HƯỚNG SUY NGHĨ

- Đi qua giá trị thật của số: cơ số a → giá trị bằng Horner, rồi giá trị → cơ số b bằng chia liên tiếp và đọc ngược các số dư.

BÀI 18.2	BIT TRONG ĐOẠN	★★☆ TRUNG BÌNH	
Chương trình **BITDOAN.CPP**	Dữ liệu vào **BITDOAN.INP**	Kết quả **BITDOAN.OUT**	Giới hạn **1 giây · 256 MB**

Kiến thức: *Đếm tổng số bit 1 theo cột, tính tuần hoàn của cột bit (Tập 1, Bài 18).*

Gọi popcount(x) là số bit 1 trong biểu diễn nhị phân của x. Chẳng hạn popcount(12) = popcount($(1100)_2$) = 2.

Bảng điện tử của nhà thi đấu hiển thị số bằng một hàng bóng đèn: số x được hiển thị bằng cách bật đúng các bóng ứng với bit 1 của x. Trong một màn trình diễn, bảng lần lượt hiển thị mọi số nguyên từ L tới R, mỗi số một giây. Mỗi giây, mỗi bóng đang sáng tiêu tốn một đơn vị điện.

Có Q màn trình diễn. Với mỗi màn, hãy tính tổng lượng điện tiêu tốn, tức popcount(L) + popcount(L + 1) + ... + popcount(R).

Dữ liệu vào

Dòng đầu chứa số nguyên Q. Mỗi dòng trong Q dòng tiếp theo chứa hai số nguyên L và R.

Kết quả

Gồm Q dòng, dòng thứ i là đáp số của màn trình diễn thứ i.

Ràng buộc

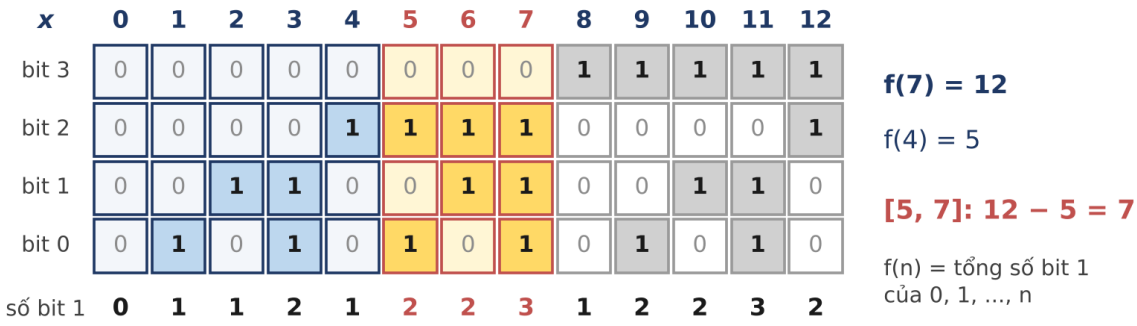
$1 \le Q \le 10^5$; $1 \le L \le R \le 10^{18}$.

Subtask	Điểm	Ràng buộc thêm
1	20%	$Q \le 10$, $R \le 10^6$.
2	30%	$R \le 10^{17}$.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

BITDOAN.INP	BITDOAN.OUT	Giải thích
2 1 12 5 7	22 7	popcount của 1, 2, ..., 12 lần lượt là 1, 1, 2, 1, 2, 2, 3, 1, 2, 2, 3, 2: tổng **22**. $(101)_2$, $(110)_2$, $(111)_2$: 2 + 2 + 3 = **7**.

BITDOAN.INP	BITDOAN.OUT	Giải thích
2 1 100000000000000000 100000000000000000 100000000000000000	29761222783429247000 24	Đáp số lớn hơn 2^{64} = 18 446 744 073 709 551 616: tràn cả `unsigned long long`. 10^{18} có 60 chữ số nhị phân, trong đó có 24 bit 1.



Hình 18.2. *Bảng nhị phân của 0..12. Tổng số bit 1 trên đoạn [5, 7] (ô vàng) bằng f(7) − f(4), với f(n) là tổng số bit 1 của 0..n (ô xanh và ô vàng).*

CHÚ Ý

Với R ≤ 10^{17}, f(R) ≤ f(10^{17}) = 2 804 216 299 269 586 964 vẫn vừa `long long`. Nhưng f(10^{18}) ≈ 2,98 · 10^{19} > 2^{64}, và ví dụ 2 cho thấy cả hiệu f(R) − f(L − 1) cũng có thể vượt 2^{64}: phải dùng `unsigned __int128` và tự viết hàm in.

Dùng `1LL << k` chứ không phải `1 << k`: hằng 1 kiểu `int` dịch quá 31 bit là sai.

Cẩn thận với L = 1: cần f(0) = 0. Viết f nhận n ≥ 0 (hoặc n = −1) cho đúng.

HƯỚNG SUY NGHĨ

- Đặt f(n) = popcount(0) + popcount(1) + ... + popcount(n), tính bằng cách đếm theo từng cột bit như bài DEMBIT của Tập 1. Đáp số của mỗi truy vấn là f(R) − f(L − 1), giống tổng đoạn là hiệu hai tổng tiền tố.
- Mỗi truy vấn chỉ cần khoảng 2 · 60 bước, nên 10^5 truy vấn chạy rất nhanh.

BÀI 18.3 SỐ CÓ K BIT 1

★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
KBIT.CPP	KBIT.INP	KBIT.OUT	1 giây · 256 MB

Kiến thức: *Biểu diễn nhị phân, duyệt bit từ cao xuống kết hợp tổ hợp C(n, k) (Tập 1, Bài 18 và Bài 13).*

Minh rất thích những số có đúng k bit 1 trong biểu diễn nhị phân. Chẳng hạn các số nguyên dương có đúng 2 bit 1, xếp tăng dần, là 3, 5, 6, 9, 10, 12, 17, ... Minh có Q câu hỏi, mỗi câu thuộc một trong hai loại:

- `1 L R k`: có bao nhiêu số nguyên x với $L \le x \le R$ có đúng k bit 1?
- `2 k m`: số nguyên dương nhỏ thứ m có đúng k bit 1 là số nào?

Dữ liệu vào

Dòng đầu chứa số nguyên Q. Mỗi dòng trong Q dòng tiếp theo mô tả một câu hỏi theo một trong hai dạng trên.

Kết quả

Gồm Q dòng, dòng thứ i là câu trả lời cho câu hỏi thứ i.

Ràng buộc

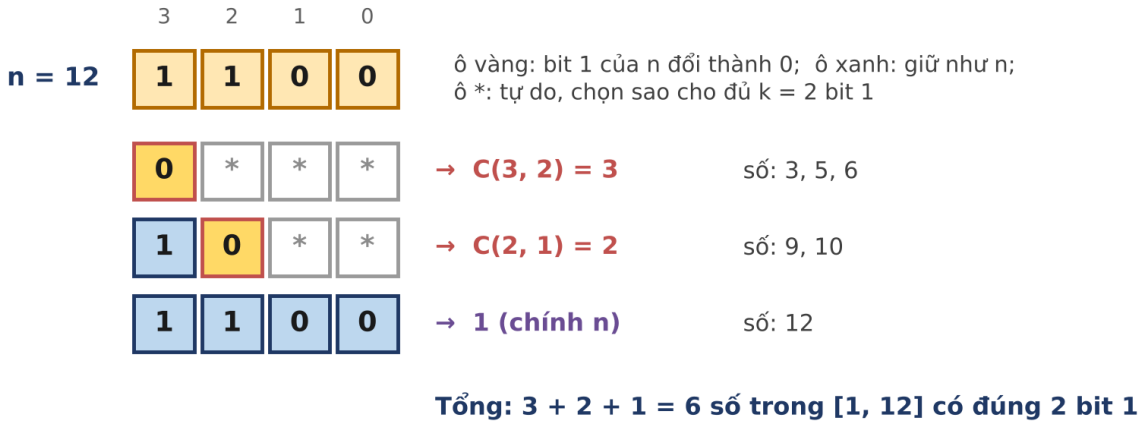
$1 \le Q \le 10^5$. Câu hỏi loại 1: $0 \le L \le R \le 10^{18}$, $0 \le k \le 60$. Câu hỏi loại 2: $1 \le k \le 59$, $m \ge 1$ và dữ liệu bảo đảm đáp số không vượt quá 10^{18}.

Subtask	Điểm	Ràng buộc thêm
1	20%	$R \le 10^5$; đáp số mọi câu hỏi loại 2 không vượt quá 10^5.
2	30%	Chỉ có câu hỏi loại 1.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

KBIT.INP	KBIT.OUT	Giải thích
3 1 1 12 2 2 2 4 1 0 10 0	6 9 1	Trong [1, 12] có **6** số có đúng 2 bit 1: 3, 5, 6, 9, 10, 12. Số thứ 4 của dãy 3, 5, 6, 9, ... là **9** = $(1001)_2$. Chỉ có số 0 không có bit 1 nào.

KBIT.INP	KBIT.OUT	Giải thích
2 1 1 1000000000000000000 30 2 59 1	103477184498507572 576460752303423487	Không thể duyệt 10^{18} số: đáp số phải được tính bằng tổ hợp. Số nhỏ nhất có 59 bit 1 là $2^{59} - 1$: 59 bit 1 liền nhau.



Hình 18.3. *Đếm các số $x \le 12 = (1100)_2$ có đúng 2 bit 1: mỗi bit 1 của n cho một nhóm số "giống n ở các bit cao hơn, bit này bằng 0, các bit thấp hơn tùy ý".*

CHÚ Ý

Loại 1 với k = 0: chỉ số 0 có 0 bit 1, nên đáp số là 1 nếu L = 0 và 0 nếu L ≥ 1.

C(60, 30) = 118 264 581 564 861 424 ≈ $1{,}18 \cdot 10^{17}$ vẫn vừa `long long`. Dựng bảng C bằng tam giác Pascal; công thức qua giai thừa sẽ tràn (20! đã gần chạm trần).

Đáp số loại 1 là g(R) − g(L − 1), với g(n) là số các x trong [0, n] có đúng k bit 1. Khi L = 0 thì g(−1) = 0.

Đừng quên tính chính n vào g(n) nếu n có đúng k bit 1.

HƯỚNG SUY NGHĨ

- Tính g(n): duyệt các bit của n từ cao xuống, đếm số bit 1 đã gặp là t. Gặp bit i bằng 1, xét các số giống n ở mọi bit cao hơn i nhưng có bit i bằng 0: i bit thấp hơn chọn tùy ý, nên có C(i, k − t) số như vậy.
- Loại 2: dựng đáp số từ bit cao xuống. Ở bit i, nếu còn cần k' bit 1 thì có C(i, k') số mà bit i bằng 0. Nếu m không vượt quá số đó, bit i bằng 0; ngược lại bit i bằng 1, m giảm đi C(i, k') và k' giảm đi 1.

BÀI 19 Tổng tiền tố và số dư

ÔN NHANH KIẾN THỨC · Tập 1, Bài 19

- **Tổng tiền tố:** $S_0 = 0$, $S_i = S_{i-1} + a_i$; tổng đoạn [l, r] bằng $S_r - S_{l-1}$. Đừng quên S_0: nó đại diện cho các đoạn bắt đầu từ vị trí 1.
- **Đoạn chia hết:** tổng đoạn [l, r] chia hết cho k $\Leftrightarrow$ $S_r \equiv S_{l-1} \pmod k$. Đếm đoạn = đếm cặp tiền tố cùng số dư: $\Sigma\, c_d\,(c_d - 1)/2$, hoặc cộng cnt[s] **trước** khi tăng, O(n + k).
- **Đoạn dài nhất:** lưu vị trí đầu tiên first[d] của mỗi số dư d (first[0] = 0); đoạn dài nhất kết thúc tại r là [first[s] + 1, r].
- Số dư làm chỉ số mảng phải không âm: `((x % k) + k) % k`. Số đoạn con có thể tới n(n + 1)/2: dùng `long long`.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
19.1	**TỔNG BẰNG KHÔNG**	★☆☆ CƠ BẢN	Đổi "đếm đoạn" thành "đếm cặp"; gom nhóm bằng sắp xếp.
19.2	**ĐOẠN DÀI NHẤT**	★★☆ TRUNG BÌNH	Tối ưu độ dài đoạn bằng vị trí xuất hiện đầu tiên; số dư không âm; chọn đáp án theo luật phụ.
19.3	**HÌNH CHỮ NHẬT CHIA HẾT**	★★★ NÂNG CAO	Hạ bài toán 2 chiều xuống 1 chiều; xóa mảng đếm thông minh.

BÀI 19.1 TỔNG BẰNG KHÔNG ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`TONG0.CPP`	`TONG0.INP`	`TONG0.OUT`	1 giây · 256 MB

Kiến thức: *Tổng tiền tố, đếm cặp tiền tố bằng nhau (Tập 1, Bài 19).*

Cửa hàng của cô Lan ghi sổ thu chi trong n ngày liên tiếp: ngày thứ i cửa hàng lãi a_i nghìn đồng ($a_i < 0$ nghĩa là lỗ $|a_i|$ nghìn đồng). Một **chuỗi ngày hòa vốn** là một dãy ngày liên tiếp có tổng tiền lãi bằng 0.

Hãy đếm số chuỗi ngày hòa vốn, tức số cặp (l, r) với $1 \le l \le r \le n$ và $a_l + a_{l+1} + \ldots + a_r = 0$.

Dữ liệu vào

Dòng đầu chứa số nguyên n. Dòng thứ hai chứa n số nguyên $a_1, a_2, \ldots, a_n$.

Kết quả

Một số nguyên là số chuỗi ngày hòa vốn.

Ràng buộc

$1 \le n \le 10^6$; $|a_i| \le 10^9$.

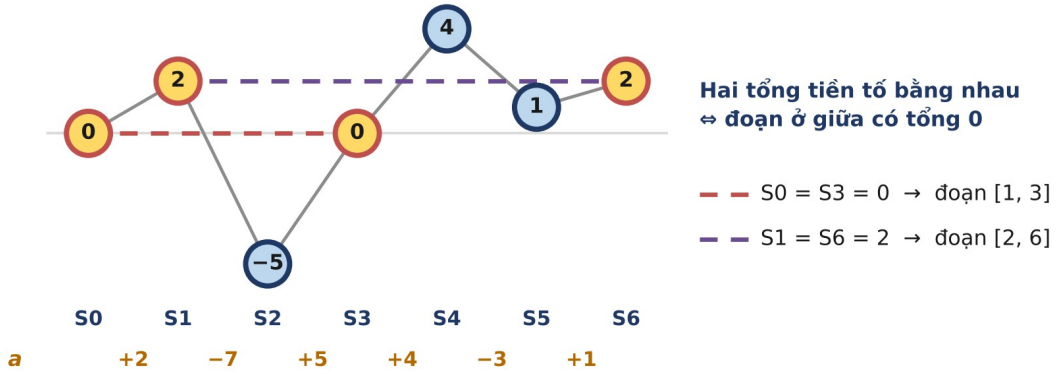
Subtask	Điểm	Ràng buộc thêm
1	30%	$n \le 3000$.
2	30%	$\|a_i\| \le 10$.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

TONG0.INP	TONG0.OUT	Giải thích
6 2 -7 5 4 -3 1	2	Tổng tiền tố: 0, 2, −5, 0, 4, 1, 2. $S_0 = S_3$ cho đoạn [1, 3] (2 − 7 + 5 = 0); $S_1 = S_6$ cho đoạn [2, 6] (−7 + 5 + 4 − 3 + 1 = 0).

TONG0.INP	TONG0.OUT	Giải thích
4 0 0 0 0	10	Mọi đoạn con đều có tổng 0: có 4 · 5 / 2 = **10** đoạn.

TONG0.INP	TONG0.OUT	Giải thích
5 1 -1 1 -1 1	6	Tổng tiền tố 0, 1, 0, 1, 0, 1: ba số 0 và ba số 1, cho C(3, 2) + C(3, 2) = **6** đoạn.



Hình 19.1. *Đồ thị tổng tiền tố của ví dụ 1: mỗi cặp điểm cùng độ cao (cùng giá trị S) cho một đoạn con có tổng bằng 0.*

CHÚ Ý

Số chuỗi có thể tới $n(n+1)/2 = 500\,000\,500\,000$ (khi mọi $a_i = 0$): dùng `long long`.

Tổng tiền tố có thể tới 10^{15} về trị tuyệt đối: vượt `int`, và quá rộng để làm chỉ số một mảng đếm (trừ khi $|a_i|$ nhỏ như ở subtask 2).

Thiếu $S_0 = 0$ thì mọi đoạn bắt đầu từ ngày 1 bị bỏ sót: ở ví dụ 1 em chỉ đếm được 1 đoạn.

HƯỚNG SUY NGHĨ

- Đoạn $[l, r]$ có tổng $0 \Leftrightarrow S_r = S_{l-1}$. Như vậy cần đếm số cặp chỉ số $i < j$ trong $0..n$ có $S_i = S_j$.
- Sắp xếp $n + 1$ tổng tiền tố: các giá trị bằng nhau đứng liền nhau thành nhóm; nhóm có c phần tử cho $c(c-1)/2$ cặp.

BÀI 19.2	ĐOẠN DÀI NHẤT	★★☆ TRUNG BÌNH

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
DOANDAI.CPP	DOANDAI.INP	DOANDAI.OUT	1 giây · 256 MB

Kiến thức: *Tổng tiền tố theo mô-đun, vị trí đầu tiên của mỗi số dư (Tập 1, Bài 19, Mở rộng 2).*

Trong một trò chơi, ngày thứ i người chơi nhận được a_i điểm (a_i có thể âm hoặc bằng 0). Cửa hàng đổi quà chỉ nhận những chuỗi ngày liên tiếp có tổng điểm **chia hết cho k** (tổng âm hoặc bằng 0 cũng được tính, miễn là chia hết cho k).

An muốn đổi quà bằng một chuỗi ngày dài nhất có thể. Hãy tìm đoạn con a_l, ..., a_r có tổng chia hết cho k và có độ dài $r - l + 1$ lớn nhất. Nếu có nhiều đoạn dài nhất, chọn đoạn có l nhỏ nhất.

Dữ liệu vào

Dòng đầu chứa hai số nguyên n và k. Dòng thứ hai chứa n số nguyên a_1, a_2, ..., a_n.

Kết quả

Nếu không có đoạn con nào có tổng chia hết cho k, in một số 0. Ngược lại, in hai số nguyên: độ dài của đoạn tìm được và chỉ số l của nó.

Ràng buộc

$1 \le n \le 10^6$; $1 \le k \le 10^6$; $|a_i| \le 10^9$.

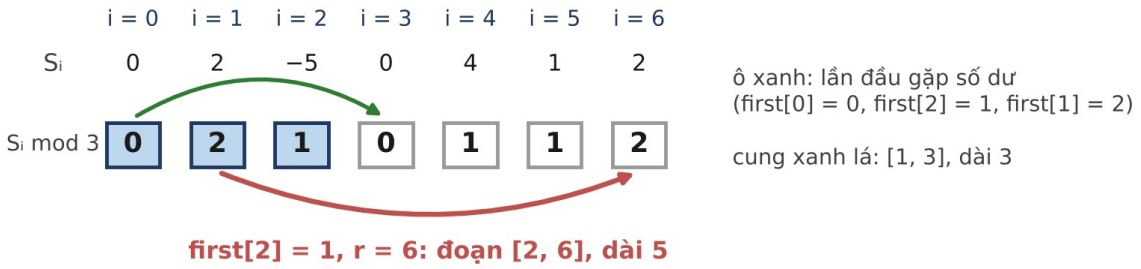
Subtask	Điểm	Ràng buộc thêm
1	30%	$n \le 3000$.
2	30%	$a_i \ge 0$.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

DOANDAI.INP	DOANDAI.OUT	Giải thích
6 3 2 -7 5 4 -3 1	5 2	Số dư (mod 3) của các tổng tiền tố 0, 2, −5, 0, 4, 1, 2 là 0, 2, 1, 0, 1, 1, 2. S_1 và S_6 cùng dư 2: đoạn [2, 6] dài 5, tổng −7 + 5 + 4 − 3 + 1 = 0.

DOANDAI.INP	DOANDAI.OUT	Giải thích
3 5 1 1 1	0	Các tổng 1, 2, 3 đều không chia hết cho 5.

DOANDAI.INP	DOANDAI.OUT	Giải thích
6 6 -1 2 7 -9 5 -2	3 2	Hai đoạn dài 3 là [2, 4] (tổng 0) và [4, 6] (tổng −6); chọn l = 2. Ở đây $S_1 = S_4 = -1$; trong C++ `-1 % 6` bằng −1, số dư đúng là 5.



Hình 19.2. *Ví dụ 1: với mỗi r, đoạn dài nhất kết thúc tại r bắt đầu ngay sau vị trí đầu tiên có cùng số dư.*

CHÚ Ý

Với r tăng dần, chỉ cập nhật đáp án khi độ dài **lớn hơn hẳn**. Khi đó trong các đoạn cùng độ dài, đoạn có r nhỏ nhất, cũng là đoạn có l nhỏ nhất, được giữ lại.

Nhớ first[0] = 0 cho S_0, và mọi số dư phải được đưa về [0, k) trước khi làm chỉ số mảng.

Đọc 10^6 số: dùng `scanf` hoặc tắt đồng bộ `cin`. Không cần lưu cả dãy.

HƯỚNG SUY NGHĨ

- Đoạn [l, r] hợp lệ $\Leftrightarrow S_{l-1} \equiv S_r \pmod k$. Với r cố định, đoạn dài nhất ứng với chỉ số l − 1 **nhỏ nhất** có cùng số dư với S_r.
- Chỉ cần mảng first[0..k − 1], khởi tạo −1 (chưa gặp), và một lượt duyệt O(n + k).

BÀI 19.3 — HÌNH CHỮ NHẬT CHIA HẾT

★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
HCNCHIAHET.CPP	HCNCHIAHET.INP	HCNCHIAHET.OUT	1 giây · 256 MB

Kiến thức: *Tổng tiền tố hai chiều quy về một chiều, đếm cặp tiền tố cùng số dư (Tập 1, Bài 19).*

Một mảnh vườn hình chữ nhật được chia thành m × n ô vuông; ô ở hàng i, cột j có độ màu mỡ $a_{i,j}$ (số âm nghĩa là đất xấu). Một **hình chữ nhật con** gồm mọi ô (i, j) với $i_1 \le i \le i_2$ và $j_1 \le j \le j_2$, trong đó $1 \le i_1 \le i_2 \le m$, $1 \le j_1 \le j_2 \le n$.

Hãy đếm số hình chữ nhật con có tổng độ màu mỡ chia hết cho k.

Dữ liệu vào

Dòng đầu chứa ba số nguyên m, n, k. Dòng thứ i trong m dòng tiếp theo chứa n số nguyên $a_{i,1}$, $a_{i,2}$, ..., $a_{i,n}$.

Kết quả

Một số nguyên là số hình chữ nhật con có tổng chia hết cho k.

Ràng buộc

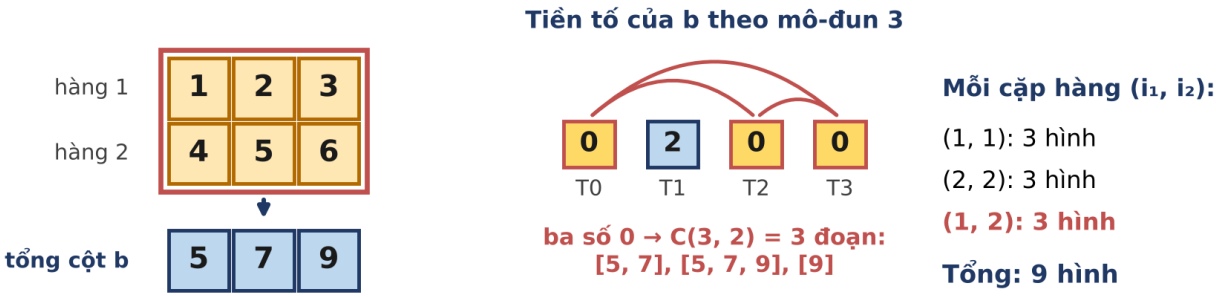
$1 \le m, n \le 400$; $1 \le k \le 10^6$; $|a_{i,j}| \le 10^9$.

Subtask	Điểm	Ràng buộc thêm
1	20%	m, n ≤ 30.
2	30%	k ≤ 100.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

HCNCHIAHET.INP	HCNCHIAHET.OUT	Giải thích
2 3 3 1 2 3 4 5 6	9	Chỉ hàng 1: [1, 2], [1, 2, 3], [3]. Chỉ hàng 2: [4, 5], [4, 5, 6], [6]. Cả hai hàng: tổng các cột là 5, 7, 9; các đoạn [5, 7], [5, 7, 9], [9] có tổng 12, 21, 9. Tổng cộng **9** hình.

HCNCHIAHET.INP	HCNCHIAHET.OUT	Giải thích
3 3 4 -1 2 -3 4 -5 6 -7 8 -9	8	Bốn hình vuông 2 × 2 có tổng 0, cả cột 1 (tổng −4), ô (2, 1) = 4, cả hàng 3 (tổng −8) và ô (3, 2) = 8: **8** hình.



Hình 19.3. *Cố định cặp hàng (1, 2): các hình chữ nhật con giữa hai hàng này ứng với các đoạn con của dãy tổng cột b, và được đếm như bài DOANCHIA.*

CHÚ Ý

Số hình chữ nhật con tới $(400 \cdot 401 / 2)^2 = 6\,432\,040\,000 > 2^{31}$: dùng `long long` cho đáp số.

Có $m(m + 1)/2 = 80\,200$ cặp hàng. Nếu với mỗi cặp em xóa cả mảng đếm kích thước $k = 10^6$ thì mất $8 \cdot 10^{10}$ thao tác. Chỉ xóa những ô vừa dùng: đi lại dãy số dư một lần nữa và gán 0.

Đưa từng $a_{i,j}$ về số dư không âm ngay khi đọc. Sau đó mọi tổng đều nằm trong $[0, k)$: cộng xong chỉ cần trừ k nếu vượt, không cần phép `%` chậm.

HƯỚNG SUY NGHĨ

- Cố định hàng trên i_1 và hàng dưới i_2. Đặt $b_j = a_{i1,j} + \ldots + a_{i2,j}$ (tổng cột j giữa hai hàng). Mỗi hình chữ nhật con giữa hai hàng ấy là một đoạn con của dãy b.
- Khi tăng i_2 thêm 1, cập nhật mọi b_j trong $O(n)$. Trên dãy b, đếm đoạn con có tổng chia hết cho k bằng mảng đếm số dư: $O(n)$. Tổng cộng $O(m^2 \cdot n) \approx 3{,}2 \cdot 10^7$ bước.

BÀI 20 Điểm nguyên và hình học số nguyên

ÔN NHANH KIẾN THỨC · Tập 1, Bài 20

- **Điểm nguyên trên đoạn thẳng:** đoạn nối hai điểm nguyên có gcd(|dx|, |dy|) + 1 điểm nguyên, cách đều nhau bởi bước (dx / g, dy / g). Quy ước gcd(d, 0) = d làm công thức đúng cả với đoạn đứng, đoạn ngang.
- **Công thức dây giày:** $2S = |\Sigma (x_i\, y_{i+1} - x_{i+1}\, y_i)|$, luôn là số nguyên với đỉnh nguyên; tính 2S, đừng chia 2 sớm.
- **Định lí Pick:** $S = I + B/2 - 1$, với $B = \Sigma \gcd(|dx|, |dy|)$ theo các cạnh; suy ra $I = (2S - B + 2)/2$. Chỉ đúng với đa giác không tự cắt, có mọi đỉnh nguyên.
- Hiệu tọa độ tới $2 \cdot 10^9$ đã vượt `int`; tích của hai hiệu tọa độ tới 10^{18} có thể tràn khi cộng trừ: ước lượng trước, dùng `__int128` khi cần.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
20.1	**TRỒNG CÂY**	★☆☆ CƠ BẢN	Áp dụng công thức gcd cho từng đoạn; không đếm trùng đỉnh chung.
20.2	**ĐA GIÁC LƯỚI**	★★☆ TRUNG BÌNH	Kết hợp diện tích và số điểm trên biên để đếm điểm bên trong.
20.3	**TAM GIÁC NGUYÊN**	★★★ NÂNG CAO	Ước lượng mọi đại lượng trung gian; tính toán và in số `__int128`.

BÀI 20.1 TRỒNG CÂY ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
TRONGCAY.CPP	**TRONGCAY.INP**	**TRONGCAY.OUT**	**1 giây · 256 MB**

Kiến thức: *Số điểm nguyên trên đoạn thẳng gcd(|dx|, |dy|) + 1 (Tập 1, Bài 20).*

Con đường đi bộ trong công viên là một đường gấp khúc $P_1P_2...P_n$ nối lần lượt n điểm nguyên $P_i(x_i, y_i)$. Ban quản lí trồng một cây tại **mỗi điểm nguyên** nằm trên con đường, kể cả các điểm P_i. Hỏi phải trồng bao nhiêu cây?

Con đường không tự cắt và không tự chạm: hai đoạn liên tiếp $P_{i-1}P_i$ và P_iP_{i+1} chỉ có đúng một điểm chung là P_i, còn hai đoạn không liên tiếp không có điểm chung nào.

Dữ liệu vào

Dòng đầu chứa số nguyên n. Dòng thứ i trong n dòng tiếp theo chứa hai số nguyên x_i, y_i.

Kết quả

Một số nguyên là số cây cần trồng.

Ràng buộc

$2 \le n \le 10^5$; $|x_i|, |y_i| \le 10^9$.

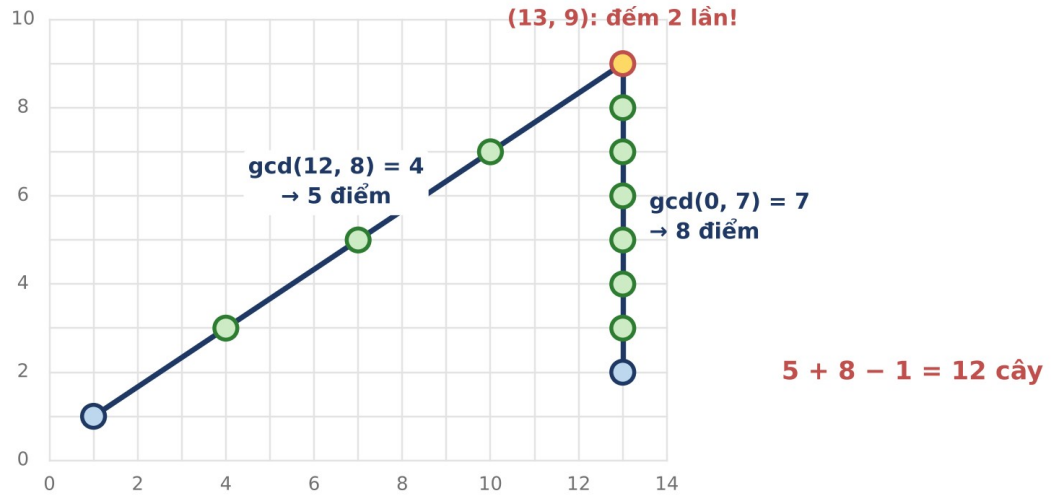
Subtask	Điểm	Ràng buộc thêm
1	30%	$n \le 100$; $\|x_i\|, \|y_i\| \le 100$.
2	30%	Mỗi đoạn của đường gấp khúc song song với một trục tọa độ.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

TRONGCAY.INP	TRONGCAY.OUT	Giải thích
3 1 1 13 9 13 2	12	Đoạn đầu: gcd(12, 8) + 1 = 5 điểm. Đoạn sau thẳng đứng: gcd(0, 7) + 1 = 8 điểm. Điểm (13, 9) được đếm hai lần: 5 + 8 − 1 = **12** cây.

TRONGCAY.INP	TRONGCAY.OUT	Giải thích
2 -1000000000 -1000000000 1000000000 1000000000	2000000001	gcd($2 \cdot 10^9$, $2 \cdot 10^9$) + 1: hiệu tọa độ đã vượt `int`.

TRONGCAY.INP	TRONGCAY.OUT	Giải thích
4 0 0 2 0 5 0 5 3	9	Ba đoạn: 3 + 4 + 4 điểm, trừ 2 đỉnh chung: **9** cây. Hai đoạn đầu cùng nằm trên trục hoành vẫn chỉ chung nhau điểm (2, 0).



Hình 20.1. *Ví dụ 1: các cây (chấm tròn) chia mỗi đoạn thành những bước bằng nhau; đỉnh chung (13, 9) chỉ được trồng một cây.*

CHÚ Ý

Đỉnh P_i ($2 \le i \le n-1$) là đầu mút chung của hai đoạn, đừng đếm hai lần. Gọn nhất: mỗi đoạn góp $\gcd(|dx|, |dy|)$ điểm (không tính đầu đoạn), rồi cộng thêm 1 cho P_1.

Lấy trị tuyệt đối của dx, dy trước khi tính gcd; dx có thể tới $2 \cdot 10^9$ nên phải dùng `long long`.

Đáp số có thể tới khoảng $10^5 \cdot 2 \cdot 10^9 = 2 \cdot 10^{14}$.

HƯỚNG SUY NGHĨ

- Dùng kết quả của bài DIEMNGUYEN (Tập 1, Bài 20) cho từng đoạn. Nhờ điều kiện không tự chạm, hai đoạn chỉ có thể chung nhau một đỉnh.

BÀI 20.2	ĐA GIÁC LƯỚI	★★☆ TRUNG BÌNH	
Chương trình **DAGIAC.CPP**	Dữ liệu vào **DAGIAC.INP**	Kết quả **DAGIAC.OUT**	Giới hạn **1 giây · 256 MB**

Kiến thức: *Công thức dây giày và định lí Pick (Tập 1, Bài 20, Mở rộng 1–2).*

Khu vườn của trường có dạng đa giác n đỉnh P_1, P_2, ..., P_n, mọi đỉnh đều là điểm nguyên, được liệt kê theo thứ tự đi quanh biên (cùng chiều hoặc ngược chiều kim đồng hồ). Đa giác không tự cắt, không tự chạm và có diện tích dương; có thể có ba đỉnh liên tiếp thẳng hàng.

Tại mỗi điểm nguyên nằm trên biên, nhà trường dựng một cột hàng rào; tại mỗi điểm nguyên nằm **hẳn bên trong**, nhà trường trồng một khóm hoa. Hãy đếm số khóm hoa và số cột rào.

Dữ liệu vào

Dòng đầu chứa số nguyên n. Dòng thứ i trong n dòng tiếp theo chứa hai số nguyên x_i, y_i là tọa độ đỉnh P_i.

Kết quả

Hai số nguyên: số điểm nguyên nằm hẳn bên trong đa giác và số điểm nguyên nằm trên biên của đa giác.

Ràng buộc

$3 \le n \le 10^5$; $|x_i|, |y_i| \le 10^6$.

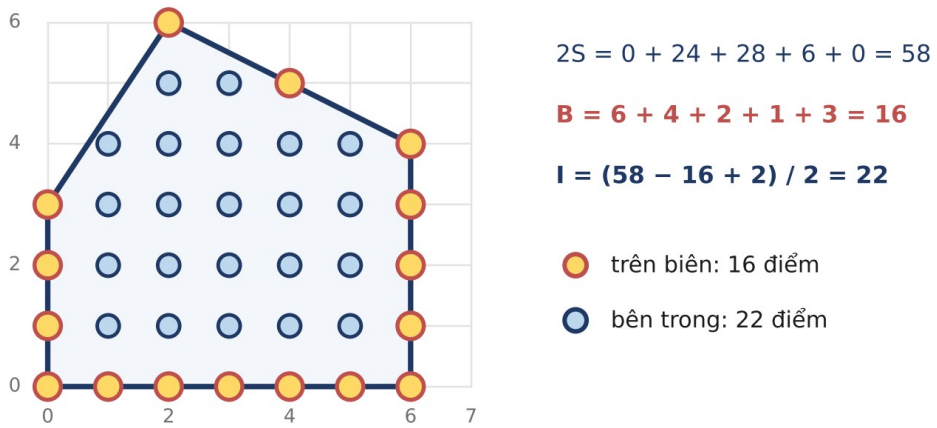
Subtask	Điểm	Ràng buộc thêm
1	20%	$n \le 100$; $\|x_i\|, \|y_i\| \le 100$.
2	30%	Mọi cạnh của đa giác song song với một trục tọa độ.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

DAGIAC.INP	DAGIAC.OUT	Giải thích
5 0 0 6 0 6 4 2 6 0 3	22 16	2S = 0 + 24 + 28 + 6 + 0 = 58, B = 6 + 4 + 2 + 1 + 3 = 16. Theo Pick: I = (58 − 16 + 2)/2 = **22**.

DAGIAC.INP	DAGIAC.OUT	Giải thích
4 0 0 0 4 2 2 4 0	3 12	Các đỉnh đi theo chiều kim đồng hồ nên tổng dây giày bằng −16: lấy trị tuyệt đối, 2S = 16. Đỉnh (2, 2) nằm giữa cạnh huyền không làm sai công thức: B = 4 + 2 + 2 + 4 = 12, I = (16 − 12 + 2)/2 = **3**.

DAGIAC.INP	DAGIAC.OUT	Giải thích
3 0 0 3 1 1 0	0 3	Tam giác rất mỏng có 2S = 1: chỉ có ba đỉnh trên biên, không có điểm nào bên trong.



Hình 20.2. *Ví dụ 1: 16 điểm trên biên (vàng), 22 điểm bên trong (xanh); diện tích 29 = 22 + 16/2 − 1.*

CHÚ Ý

Tổng dây giày âm khi các đỉnh được liệt kê theo chiều kim đồng hồ. Lấy trị tuyệt đối **sau khi** cộng hết, không lấy trị tuyệt đối từng số hạng.

Mỗi tích $x_i \cdot y_{i+1}$ tới 10^{12} và tổng tới cỡ 10^{17}: dùng `long long`. Đừng dùng `double` cho diện tích.

$B = \Sigma \gcd(|dx|, |dy|)$ theo **mọi** cạnh, kể cả cạnh cuối P_nP_1; không cộng thêm 1 cho mỗi cạnh vì mỗi đỉnh thuộc hai cạnh.

HƯỚNG SUY NGHĨ

- Trong một vòng lặp qua các cạnh, tính đồng thời 2S (dây giày) và B (tổng gcd). Định lí Pick cho ngay $I = (2S - B + 2)/2$.

BÀI 20.3	TAM GIÁC NGUYÊN	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
TAMGIAC.CPP	TAMGIAC.INP	TAMGIAC.OUT	1 giây · 256 MB

Kiến thức: *Định lí Pick với tọa độ rất lớn, số nguyên 128 bit (Tập 1, Bài 20 và Bài 17).*

Cho Q tam giác, mỗi tam giác có ba đỉnh là điểm nguyên không thẳng hàng. Với mỗi tam giác, hãy đếm số điểm nguyên nằm **hẳn bên trong** tam giác (không tính các điểm nằm trên cạnh).

Tọa độ các đỉnh có thể rất lớn, tới 10^{18}, nên số điểm cần đếm có thể vượt xa 2^{64}.

Dữ liệu vào

Dòng đầu chứa số nguyên Q. Tiếp theo là Q nhóm, mỗi nhóm gồm ba dòng mô tả một tam giác; mỗi dòng chứa hai số nguyên x, y là tọa độ một đỉnh.

Kết quả

Gồm Q dòng, dòng thứ i là số điểm nguyên nằm hẳn bên trong tam giác thứ i.

Ràng buộc

$1 \le Q \le 10^5$; $|x|, |y| \le 10^{18}$; ba đỉnh của mỗi tam giác không thẳng hàng.

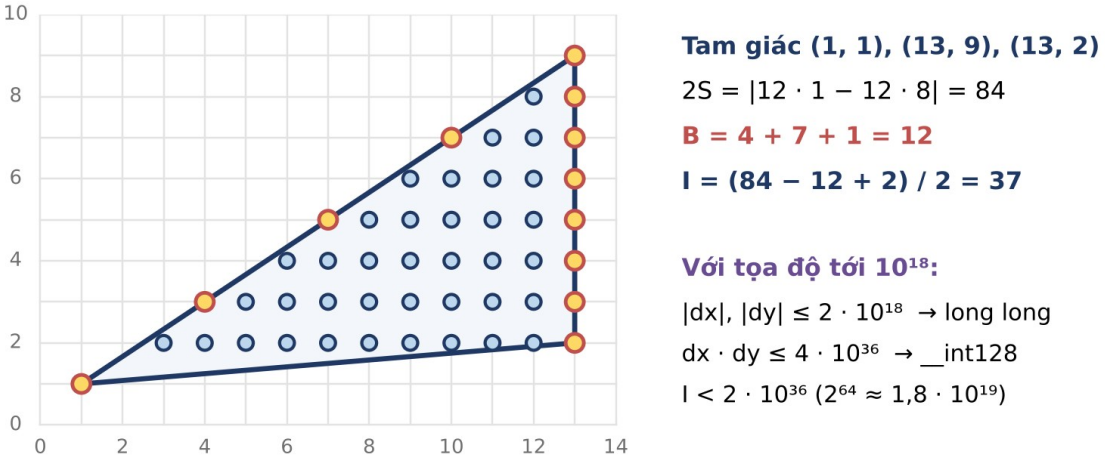
Subtask	Điểm	Ràng buộc thêm
1	20%	$\|x\|, \|y\| \le 100$.
2	30%	$\|x\|, \|y\| \le 10^9$.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

TAMGIAC.INP	TAMGIAC.OUT	Giải thích
2 0 0 4 0 0 4 1 1 13 9 13 2	3 37	Tam giác (0, 0), (4, 0), (0, 4): 2S = 16, B = 4 + 4 + 4 = 12, I = (16 − 12 + 2)/2 = **3**, đó là (1, 1), (1, 2), (2, 1). Tam giác thứ hai: 2S = 84, B = 4 + 7 + 1 = 12, I = **37** (Hình 20.3).

TAMGIAC.INP	TAMGIAC.OUT
2 0 0 1000000000000000000 0 0 1000000000000000000 0 0 1000000000000000000 1000000000000000000 1000000000000000000 999999999999999999	499999999999999998500000000000000001 0

Giải thích ví dụ 2. Tam giác thứ nhất: $2S = 10^{36}$, $B = 3 \cdot 10^{18}$, nên $I = (10^{36} - 3 \cdot 10^{18} + 2)/2$ có 36 chữ số: vượt xa $2^{64} \approx 1{,}8 \cdot 10^{19}$ nhưng vẫn nhỏ hơn $2^{127} \approx 1{,}7 \cdot 10^{38}$. Tam giác thứ hai rất "mỏng": $2S = |10^{18} \cdot (10^{18} - 1) - 10^{18} \cdot 10^{18}| = 10^{18}$, hai tích đều cỡ 10^{36} và tràn `long long` dù hiệu của chúng chỉ là 10^{18}. $B = 10^{18} + 1 + 1$ nên $I = 0$.



Hình 20.3. Tam giác thứ hai của ví dụ 1: 12 điểm trên biên, 37 điểm bên trong. Bên phải: độ lớn của các đại lượng khi tọa độ tới 10^{18}.

CHÚ Ý

Hiệu tọa độ tới $2 \cdot 10^{18}$ vẫn vừa `long long` (tới $9{,}2 \cdot 10^{18}$), nhưng tích hai hiệu tới $4 \cdot 10^{36}$: chỉ `__int128` (tới $1{,}7 \cdot 10^{38}$) chứa nổi.

Viết `(__int128)ux * vy`, không viết `(__int128)(ux * vy)`: ở cách sau phép nhân đã tràn trên 64 bit trước khi ép kiểu.

`cout` và `printf` không in được `__int128`: tự viết hàm in đệ quy như ở bài BITDOAN.

Đừng dùng `double` hay `long double`: chúng chỉ giữ khoảng 15 hoặc 18 chữ số có nghĩa.

HƯỚNG SUY NGHĨ

- Định lí Pick: $I = (2S - B + 2)/2$, với $2S = |(x_2 - x_1)(y_3 - y_1) - (x_3 - x_1)(y_2 - y_1)|$ và B là tổng gcd của ba cạnh.
- Ở subtask 2 mọi thứ vừa `long long` ($|2S| \le 8 \cdot 10^{18}$). Ở subtask 3 hãy ước lượng lại từng đại lượng: hiệu, tích, tổng gcd, kết quả.

PHỤ LỤC A. KIỂM TRA TRƯỚC KHI NỘP BÀI

Rất nhiều điểm bị mất không phải vì thuật toán sai mà vì những lỗi nhỏ. Trước khi nộp, hãy rà lại danh sách dưới đây; các lỗi này đã xuất hiện trong khung CHÚ Ý của nhiều bài tập.

ĐỌC GHI DỮ LIỆU

- ☐ Tên tệp đúng chính tả, viết hoa đúng như đề (NGAYTHU.INP, NGAYTHU.OUT).
- ☐ Đọc nhanh (`scanf` hoặc `ios::sync_with_stdio(false)`) khi dữ liệu có từ 10^5 số trở lên.
- ☐ Số cho dưới dạng **xâu** (hàng nghìn chữ số) phải đọc vào `string`, không đọc vào kiểu số.
- ☐ In đúng khuôn dạng: số dòng, thứ tự, dấu cách; in −1 / NO đúng như yêu cầu.

TRÀN SỐ

- ☐ Mọi biến có thể vượt $2 \cdot 10^9$ đều là `long long`; tích hai số tới 10^{18} dùng `__int128`.
- ☐ Lấy dư **sau mỗi phép tính** khi đề yêu cầu mô-đun; tích hai số dư < $10^9 + 7$ vừa `long long`.
- ☐ BCNN: chia trước, nhân sau; kiểm tra vượt giới hạn trước khi nhân.
- ☐ Vòng lặp `for (d = 1; d * d <= n; d++)`: d phải là `long long` khi n tới 10^{18}.

SỐ DƯ VÀ PHÉP CHIA

- ☐ Số âm: `a % m` có thể âm trong C++; dùng `((a % m) + m) % m`.
- ☐ Không chia trong đồng dư: nhân với nghịch đảo (Tập 1, Bài 10), và chỉ khi gcd(b, m) = 1.
- ☐ Kết quả trừ theo mô-đun phải cộng thêm m trước khi lấy dư.
- ☐ m = 1: mọi số dư đều bằng 0. Đã thử chưa?

TRƯỜNG HỢP ĐẶC BIỆT

- ☐ n = 0, n = 1, n là số nguyên tố, n là số chính phương, n là lũy thừa của 2.
- ☐ a = 0 hoặc b = 0 trong gcd, lũy thừa 0^0 (thường quy ước bằng 1).
- ☐ Sàng: các số 0 và 1 không phải số nguyên tố; số 2 là số nguyên tố chẵn duy nhất.
- ☐ Nhiều bộ dữ liệu trong một tệp: đã khởi tạo lại các mảng, biến đếm chưa?

KIỂM TRA CUỐI CÙNG

- ☐ Chạy lại toàn bộ ví dụ trong đề.
- ☐ Tự tạo một test lớn nhất để đo thời gian.
- ☐ Đối chiếu với lời giải ngây thơ trên test ngẫu nhiên nhỏ (Phụ lục B).
- ☐ Thử test lớn nhất có kết quả đã biết (ví dụ $n = 10^{18}$) để kiểm tra tràn số.

PHỤ LỤC B. TỰ KIỂM TRA BẰNG CÁCH ĐỐI CHIẾU

Mọi lời giải mẫu của bộ sách đều được kiểm tra bằng kĩ thuật **đối chiếu** (còn gọi là "chấm trâu"). Ý tưởng rất đơn giản: viết một chương trình **ngây thơ** (thường là vét cạn, chậm nhưng chắc chắn đúng), sinh thật nhiều bộ dữ liệu **nhỏ** ngẫu nhiên, chạy cả hai chương trình và so sánh kết quả. Chỉ cần một bộ cho kết quả khác nhau là em đã có trong tay một test nhỏ làm lộ lỗi sai.

Ba chương trình cần có:

- `GEN.CPP`: sinh ngẫu nhiên một bộ dữ liệu nhỏ và in ra màn hình.
- `TRAU.CPP`: lời giải ngây thơ.
- `SOL.CPP`: lời giải cần kiểm tra.

Lưu ý: khi đối chiếu, hãy đặt các chương trình ở thư mục **không có** tệp `.INP` của bài, để kiểu đọc "có tệp thì đọc tệp" chuyển sang đọc từ bàn phím (luồng chuẩn).

GEN.CPP: sinh ngẫu nhiên một bộ dữ liệu nhỏ cho bài 2.2 (một số n ≤ 1000)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main(int argc, char* argv[]) {
    mt19937 rng(atoi(argv[1]));     // hạt giống lấy từ tham số dòng lệnh
    int n = rng() % 1000 + 1;       // số nhỏ: vét cạn chạy tức thì
    printf("%d\n", n);
    return 0;
}
```

DOICHIEU.CPP: chạy 1000 test, dừng ở test đầu tiên có kết quả khác nhau

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    for (int t = 1; t <= 1000; t++) {
        string seed = to_string(t);
        system(("gen " + seed + " > test.inp").c_str());
        system("sol < test.inp > sol.out");
        system("trau < test.inp > trau.out");
        // Windows dùng "fc", Linux dùng "cmp -s"
        if (system("fc sol.out trau.out > nul") != 0) {
            printf("Sai o test %d, xem tep test.inp\n", t);
            return 0;
        }
    }
    printf("Ca 1000 test deu khop!\n");
    return 0;
}
```

CHÚ Ý

Với bài có nhiều đáp án đúng (đề bài ghi "in cách bất kì"), không thể so sánh trực tiếp hai tệp kết quả. Hãy viết một chương trình **kiểm tra**: đọc dữ liệu vào và kết quả của em, rồi kiểm tra kết quả có hợp lệ không và có tối ưu bằng kết quả của chương trình trâu không.

Test ngẫu nhiên nên **nhỏ** (số nhỏ, dãy ngắn): khi phát hiện sai, em sẽ chạy tay được ngay trên test đó. Với bài số học, Python tính được số nguyên lớn chính xác, nên một dòng Python cũng có thể là "chương trình trâu" rất tốt.

PHỤ LỤC C. BẢNG THEO DÕI TIẾN ĐỘ

Sau mỗi lần làm bài, hãy ghi lại ngày làm, số điểm (theo subtask) và một dòng ghi chú về điều học được hoặc lỗi đã mắc. Đọc lại cột ghi chú trước mỗi kì thi.

Chương 1. CHIA HẾT VÀ ƯỚC SỐ

Bài	Tên bài	Mức độ	Ngày làm	Điểm	Ghi chú
1.1	NGÀY TRONG TUẦN	★☆☆			
1.2	CHỮ SỐ CUỐI	★★☆			
1.3	VIẾT LIỀN	★★★			
2.1	XẾP HÀNG	★☆☆			
2.2	ƯỚC TĂNG DẦN	★★☆			
2.3	SỐ BẠN BÈ	★★★			
3.1	CẮT GẠCH	★☆☆			
3.2	BỎ MỘT SỐ	★★☆			
3.3	ĐÈN NHÁY	★★★			
4.1	ĐONG NƯỚC	★☆☆			
4.2	MUA HÀNG	★★☆			
4.3	ĐỒNG HỒ HỎNG	★★★			

Chương 2. SỐ NGUYÊN TỐ

Bài	Tên bài	Mức độ	Ngày làm	Điểm	Ghi chú
5.1	NGUYÊN TỐ GẦN NHẤT	★☆☆			
5.2	TỔNG HAI SỐ NGUYÊN TỐ	★★☆			
5.3	NGUYÊN TỐ 64 BIT	★★★			
6.1	ĐẾM SỐ NGUYÊN TỐ	★☆☆			
6.2	SỐ NGUYÊN TỐ SINH ĐÔI	★★☆			
6.3	SỐ CÓ ĐÚNG BA ƯỚC	★★★			
7.1	SỐ ƯỚC	★☆☆			
7.2	CHỮ SỐ 0 CỦA GIAI THỪA	★★☆			

Bài	Tên bài	Mức độ	Ngày làm	Điểm	Ghi chú
7.3	BCNN CỦA DÃY	★★★			
8.1	NGUYÊN TỐ TRÊN ĐOẠN	★☆☆			
8.2	KHOẢNG CÁCH LỚN NHẤT	★★☆			
8.3	SỐ ƯỚC TRÊN ĐOẠN	★★★			

Chương 3. SỐ HỌC ĐỒNG DƯ

Bài	Tên bài	Mức độ	Ngày làm	Điểm	Ghi chú
9.1	LŨY THỪA	★☆☆			
9.2	SỐ MŨ KHỔNG LỒ	★★☆			
9.3	TỔNG LŨY THỪA	★★★			
10.1	XÁC SUẤT	★☆☆			
10.2	NGHỊCH ĐẢO DÃY	★★☆			
10.3	PHƯƠNG TRÌNH ĐỒNG DƯ	★★★			
11.1	PHI	★☆☆			
11.2	PHÂN SỐ TỐI GIẢN	★★☆			
11.3	THÁP LŨY THỪA	★★★			
12.1	ĐIỂM BINH	★☆☆			
12.2	HAI NGỌN ĐÈN	★★☆			
12.3	LỊCH CAN CHI	★★★			

Chương 4. TỔ HỢP VÀ ĐẾM

Bài	Tên bài	Mức độ	Ngày làm	Điểm	Ghi chú
13.1	CHIA KẸO	★☆☆			
13.2	ĐƯỜNG ĐI TRÊN LƯỚI	★★☆			
13.3	TỔ HỢP KHỔNG LỒ	★★★			
14.1	BỘI SỐ	★☆☆			
14.2	NGUYÊN TỐ CÙNG NHAU	★★☆			
14.3	TẶNG QUÀ	★★★			

Bài	Tên bài	Mức độ	Ngày làm	Điểm	Ghi chú
15.1	LÁT GẠCH	★☆☆			
15.2	TRIBONACCI	★★☆			
15.3	ĐƯỜNG ĐI ĐỘ DÀI K	★★★			
16.1	TỔNG ƯỚC	★☆☆			
16.2	TỔNG DƯ	★★☆			
16.3	HAI THƯƠNG	★★★			

Chương 5. KĨ THUẬT SỐ HỌC TRONG LẬP TRÌNH

Bài	Tên bài	Mức độ	Ngày làm	Điểm	Ghi chú
17.1	TỔNG LỚN	★☆☆			
17.2	LŨY THỪA ĐẦY ĐỦ	★★☆			
17.3	TÍCH LỚN	★★★			
18.1	ĐỔI CƠ SỐ	★☆☆			
18.2	BIT TRONG ĐOẠN	★★☆			
18.3	SỐ CÓ K BIT 1	★★★			
19.1	TỔNG BẰNG KHÔNG	★☆☆			
19.2	ĐOẠN DÀI NHẤT	★★☆			
19.3	HÌNH CHỮ NHẬT CHIA HẾT	★★★			
20.1	TRỒNG CÂY	★☆☆			
20.2	ĐA GIÁC LƯỚI	★★☆			
20.3	TAM GIÁC NGUYÊN	★★★			